```
        }
        else if (units.equals("foot")
                   || units.equals("feet") || units.equals("ft")) {
            inches += measurement * 12;
        }
        else if (units.equals("yard")
                   || units.equals("yards") || units.equals("yd")) {
            inches += measurement * 36;
        }
        else if (units.equals("mile")
                   || units.equals("miles") || units.equals("mi")) {
            inches += measurement * 12 * 5280;
        }
        else {
            TextIO.putln("Error: \"" + units
                               + "\" is not a legal unit of measure.");
            return -1;
        }

        /* Look ahead to see whether the next thing on the line is
           the end-of-line. */

        skipBlanks();
        ch = TextIO.peek();

    }  // end while

    return inches;

} // end readMeasurement()
```

The source code for the complete program can be found in the file *LengthConverter2.java*.

8.3 Exceptions and try..catch

GETTING A PROGRAM TO WORK under ideal circumstances is usually a lot easier than making the program ***robust***. A robust program can survive unusual or "exceptional" circumstances without crashing. One approach to writing robust programs is to anticipate the problems that might arise and to include tests in the program for each possible problem. For example, a program will crash if it tries to use an array element `A[i]`, when `i` is not within the declared range of indices for the array `A`. A robust program must anticipate the possibility of a bad index and guard against it. One way to do this is to write the program in a way that ensures that the index is in the legal range. Another way is to test whether the index value is legal before using it in the array. This could be done with an `if` statement:

```
if (i < 0 || i >= A.length) {
   ...  // Do something to handle the out-of-range index, i
}
else {
   ...  // Process the array element, A[i]
}
```

There are some problems with this approach. It is difficult and sometimes impossible to anticipate all the possible things that might go wrong. It's not always clear what to do when an error is detected. Furthermore, trying to anticipate all the possible problems can turn what would otherwise be a straightforward program into a messy tangle of `if` statements.

8.3.1 Exceptions and Exception Classes

We have already seen that Java (like its cousin, C++) provides a neater, more structured alternative method for dealing with errors that can occur while a program is running. The method is referred to as ***exception handling***. The word "exception" is meant to be more general than "error." It includes any circumstance that arises as the program is executed which is meant to be treated as an exception to the normal flow of control of the program. An exception might be an error, or it might just be a special case that you would rather not have clutter up your elegant algorithm.

When an exception occurs during the execution of a program, we say that the exception is ***thrown***. When this happens, the normal flow of the program is thrown off-track, and the program is in danger of crashing. However, the crash can be avoided if the exception is ***caught*** and handled in some way. An exception can be thrown in one part of a program and caught in a different part. An exception that is not caught will generally cause the program to crash. (More exactly, the thread that throws the exception will crash. In a multithreaded program, it is possible for other threads to continue even after one crashes. We will cover threads in Section 8.5. In particular, GUI programs are multithreaded, and parts of the program might continue to function even while other parts are non-functional because of exceptions.)

By the way, since Java programs are executed by a Java interpreter, having a program crash simply means that it terminates abnormally and prematurely. It doesn't mean that the Java interpreter will crash. In effect, the interpreter catches any exceptions that are not caught by the program. The interpreter responds by terminating the program. In many other programming languages, a crashed program will sometimes crash the entire system and freeze the computer until it is restarted. With Java, such system crashes should be impossible—which means that when they happen, you have the satisfaction of blaming the system rather than your own program.

Exceptions were introduced in Section 3.7, along with the `try..catch` statement, which is used to catch and handle exceptions. However, that section did not cover the complete syntax of `try..catch` or the full complexity of exceptions. In this section, we cover these topics in full detail.

* * *

When an exception occurs, the thing that is actually "thrown" is an object. This object can carry information (in its instance variables) from the point where the exception occurs to the point where it is caught and handled. This information always includes the ***subroutine call stack***, which is a list of the subroutines that were being executed when the exception was thrown. (Since one subroutine can call another, several subroutines can be active at the same time.) Typically, an exception object also includes an error message describing what happened to cause the exception, and it can contain other data as well. All exception objects must belong to a subclass of the standard class `java.lang.Throwable`. In general, each different type of exception is represented by its own subclass of *Throwable*, and these subclasses are arranged in a fairly complex class hierarchy that shows the relationship among various types of exception. *Throwable* has two direct subclasses, *Error* and *Exception*. These two subclasses in turn have

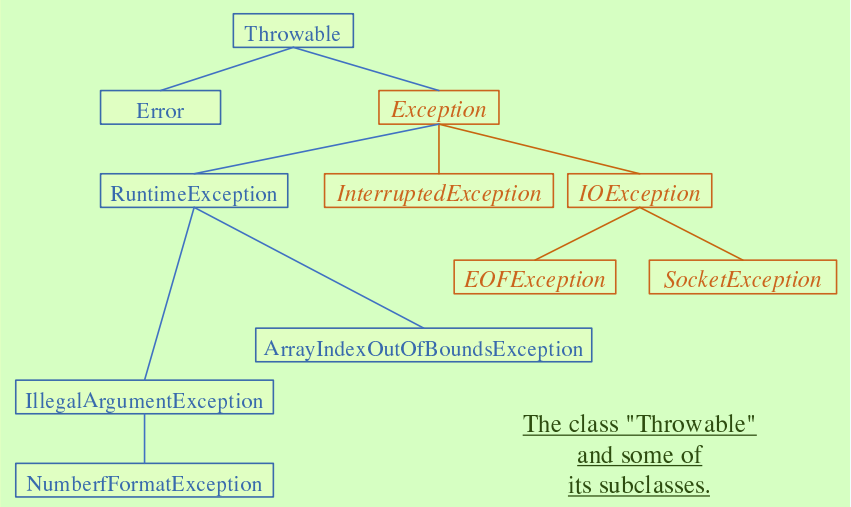
many other predefined subclasses. In addition, a programmer can create new exception classes to represent new types of exception.

Most of the subclasses of the class *Error* represent serious errors within the Java virtual machine that should ordinarily cause program termination because there is no reasonable way to handle them. In general, you should not try to catch and handle such errors. An example is a *ClassFormatError*, which occurs when the Java virtual machine finds some kind of illegal data in a file that is supposed to contain a compiled Java class. If that class was being loaded as part of the program, then there is really no way for the program to proceed.

On the other hand, subclasses of the class *Exception* represent exceptions that are meant to be caught. In many cases, these are exceptions that might naturally be called "errors," but they are errors in the program or in input data that a programmer can anticipate and possibly respond to in some reasonable way. (However, you should avoid the temptation of saying, "Well, I'll just put a thing here to catch all the errors that might occur, so my program won't crash." If you don't have a reasonable way to respond to the error, it's best just to let the program crash, because trying to go on will probably only lead to worse things down the road—in the worst case, a program that gives an incorrect answer without giving you any indication that the answer might be wrong!)

The class *Exception* has its own subclass, *RuntimeException*. This class groups together many common exceptions, including all those that have been covered in previous sections. For example, *IllegalArgumentException* and *NullPointerException* are subclasses of *RuntimeException*. A *RuntimeException* generally indicates a bug in the program, which the programmer should fix. *RuntimeExceptions* and *Errors* share the property that a program can simply ignore the possibility that they might occur. ("Ignoring" here means that you are content to let your program crash if the exception occurs.) For example, a program does this every time it uses an array reference like `A[i]` without making arrangements to catch a possible *ArrayIndexOutOfBoundsException*. For all other exception classes besides *Error*, *RuntimeException*, and their subclasses, exception-handling is "mandatory" in a sense that I'll discuss below.

The following diagram is a class hierarchy showing the class `Throwable` and just a few of its subclasses. Classes that require mandatory exception-handling are shown in italic:



The class *Throwable* includes several instance methods that can be used with any exception object. If `e` is of type *Throwable* (or one of its subclasses), then `e.getMessage()` is a function

that returns a *String* that describes the exception. The function `e.toString()`, which is used by the system whenever it needs a string representation of the object, returns a *String* that contains the name of the class to which the exception belongs as well as the same string that would be returned by `e.getMessage()`. And `e.printStackTrace()` writes a stack trace to standard output that tells which subroutines were active when the exception occurred. A stack trace can be very useful when you are trying to determine the cause of the problem. (Note that if an exception is **not** caught by the program, then the system automatically prints the stack trace to standard output.)

8.3.2 The try Statement

To catch exceptions in a Java program, you need a `try` statement. We have been using such statements since Section 3.7, but the full syntax of the `try` statement is more complicated than what was presented there. The `try` statements that we have used so far had a syntax similar to the following example:

```
try {
    double determinant = M[0][0]*M[1][1] - M[0][1]*M[1][0];
    System.out.println("The determinant of M is " + determinant);
}
catch ( ArrayIndexOutOfBoundsException e ) {
   System.out.println("M is the wrong size to have a determinant.");
   e.printStackTrace();
}
```

Here, the computer tries to execute the block of statements following the word "`try`". If no exception occurs during the execution of this block, then the "`catch`" part of the statement is simply ignored. However, if an exception of type *ArrayIndexOutOfBoundsException* occurs, then the computer jumps immediately to the `catch` clause of the `try` statement. This block of statements is said to be an ***exception handler*** for *ArrayIndexOutOfBoundsException*. By handling the exception in this way, you prevent it from crashing the program. Before the body of the `catch` clause is executed, the object that represents the exception is assigned to the variable `e`, which is used in this example to print a stack trace.

However, the full syntax of the `try` statement allows more than one `catch` clause. This makes it possible to catch several different types of exception with one `try` statement. In the above example, in addition to the possible *ArrayIndexOutOfBoundsException*, there is a possible *NullPointerException* which will occur if the value of `M` is `null`. We can handle both possible exceptions by adding a second `catch` clause to the `try` statement:

```
try {
    double determinant = M[0][0]*M[1][1] - M[0][1]*M[1][0];
    System.out.println("The determinant of M is " + determinant);
}
catch ( ArrayIndexOutOfBoundsException e ) {
   System.out.println("M is the wrong size to have a determinant.");
}
catch ( NullPointerException e ) {
   System.out.print("Programming error!  M doesn't exist." + );
}
```

Here, the computer tries to execute the statements in the `try` clause. If no error occurs, both of the `catch` clauses are skipped. If an *ArrayIndexOutOfBoundsException* occurs, the computer

executes the body of the first `catch` clause and skips the second one. If a *NullPointerException* occurs, it jumps to the second `catch` clause and executes that. 1

Note that both *ArrayIndexOutOfBoundsException* and *NullPointerException* are subclasses of *RuntimeException*. It's possible to catch all *RuntimeExceptions* with a single `catch` clause. For example: 2

```
try {
    double determinant = M[0][0]*M[1][1] - M[0][1]*M[1][0];
    System.out.println("The determinant of M is " + determinant);
}
catch ( RuntimeException err ) {
   System.out.println("Sorry, an error has occurred.");
   System.out.println("The error was: " + err);
}
```
3

The `catch` clause in this `try` statement will catch any exception belonging to class *RuntimeException* or to any of its subclasses. This shows why exception classes are organized into a class hierarchy. It allows you the option of casting your net narrowly to catch only a specific type of exception. Or you can cast your net widely to catch a wide class of exceptions. Because of subclassing, when there are multiple `catch` clauses in a `try` statement, it is possible that a given exception might match several of those `catch` clauses. For example, an exception of type *NullPointerException* would match `catch` clauses for *NullPointerException*, *RuntimeException*, *Exception*, or *Throwable*. In this case, only the **first** `catch` clause that matches the exception is executed. 4

The example I've given here is not particularly realistic. You are not very likely to use exception-handling to guard against null pointers and bad array indices. This is a case where careful programming is better than exception handling: Just be sure that your program assigns a reasonable, non-`null` value to the array `M`. You would certainly resent it if the designers of Java forced you to set up a `try..catch` statement every time you wanted to use an array! This is why handling of potential *RuntimeExceptions* is not mandatory. There are just too many things that might go wrong! (This also shows that exception-handling does not solve the problem of program robustness. It just gives you a tool that will in many cases let you approach the problem in a more organized way.) 5

* * * 6

I have still not completely specified the syntax of the `try` statement. There is one additional element: the possibility of a ***finally clause*** at the end of a `try` statement. The complete syntax of the `try` statement can be described as: 7

```
try {
   ⟨statements⟩
}
⟨optional-catch-clauses⟩
⟨optional-finally-clause⟩
```
8

Note that the `catch` clauses are also listed as optional. The `try` statement can include zero or more `catch` clauses and, optionally, a `finally` clause. The `try` statement **must** include one or the other. That is, a `try` statement can have either a `finally` clause, or one or more `catch` clauses, or both. The syntax for a `catch` clause is 9

```
catch ( ⟨exception-class-name⟩ ⟨variable-name⟩ ) {
   ⟨statements⟩
}
```
10

and the syntax for a `finally` clause is

```
finally {
   ⟨statements⟩
}
```

The semantics of the `finally` clause is that the block of statements in the `finally` clause is guaranteed to be executed as the last step in the execution of the try statement, whether or not any exception occurs and whether or not any exception that does occur is caught and handled. The `finally` clause is meant for doing essential cleanup that under no circumstances should be omitted. One example of this type of cleanup is closing a network connection. Although you don't yet know enough about networking to look at the actual programming in this case, we can consider some pseudocode:

```
try {
   open a network connection
}
catch ( IOException e ) {
   report the error
   return  // Don't continue if connection can't be opened!
}

// At this point, we KNOW that the connection is open.

try {
   communicate over the connection
}
catch ( IOException e ) {
   handle the error
}
finally {
   close the connection
}
```

The `finally` clause in the second `try` statement ensures that the network connection will definitely be closed, whether or not an error occurs during the communication. The first `try` statement is there to make sure that we don't even try to communicate over the network unless we have successfully opened a connection. The pseudocode in this example follows a general pattern that can be used to robustly obtain a resource, use the resource, and then release the resource.

8.3.3 Throwing Exceptions

There are times when it makes sense for a program to deliberately throw an exception. This is the case when the program discovers some sort of exceptional or error condition, but there is no reasonable way to handle the error at the point where the problem is discovered. The program can throw an exception in the hope that some other part of the program will catch and handle the exception. This can be done with a ***throw statement***. You have already seen an example of this in Subsection 4.3.5. In this section, we cover the `throw` statement more fully. The syntax of the `throw` statement is:

```
throw ⟨exception-object⟩ ;
```

The ⟨*exception-object*⟩ must be an object belonging to one of the subclasses of `Throwable`. Usually, it will in fact belong to one of the subclasses of `Exception`. In most cases, it will be a newly constructed object created with the `new` operator. For example:

```
throw new ArithmeticException("Division by zero");
```

The parameter in the constructor becomes the error message in the exception object; if `e` refers to the object, the error message can be retrieved by calling `e.getMessage()`. (You might find this example a bit odd, because you might expect the system itself to throw an *ArithmeticException* when an attempt is made to divide by zero. So why should a programmer bother to throw the exception? Recall that if the numbers that are being divided are of type **int**, then division by zero will indeed throw an *ArithmeticException*. However, no arithmetic operations with floating-point numbers will ever produce an exception. Instead, the special value `Double.NaN` is used to represent the result of an illegal operation. In some situations, you might prefer to throw an *ArithmeticException* when a real number is divided by zero.)

An exception can be thrown either by the system or by a `throw` statement. The exception is processed in exactly the same way in either case. Suppose that the exception is thrown inside a `try` statement. If that `try` statement has a `catch` clause that handles that type of exception, then the computer jumps to the `catch` clause and executes it. The exception has been ***handled***. After handling the exception, the computer executes the `finally` clause of the `try` statement, if there is one. It then continues normally with the rest of the program, which follows the `try` statement. If the exception is not immediately caught and handled, the processing of the exception will continue.

When an exception is thrown during the execution of a subroutine and the exception is not handled in the same subroutine, then that subroutine is terminated (after the execution of any pending `finally` clauses). Then the routine that called that subroutine gets a chance to handle the exception. That is, if the subroutine was called inside a `try` statement that has an appropriate `catch` clause, then **that** `catch` clause will be executed and the program will continue on normally from there. Again, if the second routine does not handle the exception, then it also is terminated and the routine that called **it** (if any) gets the next shot at the exception. The exception will crash the program only if it passes up through the entire chain of subroutine calls without being handled. (In fact, even this is not quite true: In a multithreaded program, only the thread in which the exception occurred is terminated.)

A subroutine that might generate an exception can announce this fact by adding a clause "`throws` ⟨*exception-class-name*⟩" to the header of the routine. For example:

```
/**
 * Returns the larger of the two roots of the quadratic equation
 * A*x*x + B*x + C = 0, provided it has any roots.  If A == 0 or
 * if the discriminant, B*B - 4*A*C, is negative, then an exception
 * of type IllegalArgumentException is thrown.
 */
static public double root( double A, double B, double C )
                                throws IllegalArgumentException {
   if (A == 0) {
     throw new IllegalArgumentException("A can't be zero.");
   }
   else {
      double disc = B*B - 4*A*C;
      if (disc < 0)
         throw new IllegalArgumentException("Discriminant < zero.");
```

```
        return  (-B + Math.sqrt(disc)) / (2*A);
    }
}
```

As discussed in the previous section, the computation in this subroutine has the preconditions that `A != 0` and `B*B-4*A*C >= 0`. The subroutine throws an exception of type *IllegalArgumentException* when either of these preconditions is violated. When an illegal condition is found in a subroutine, throwing an exception is often a reasonable response. If the program that called the subroutine knows some good way to handle the error, it can catch the exception. If not, the program will crash—and the programmer will know that the program needs to be fixed.

A `throws` clause in a subroutine heading can declare several different types of exception, separated by commas. For example:

```
void processArray(int[] A) throws NullPointerException,
                                  ArrayIndexOutOfBoundsException { ...
```

8.3.4 Mandatory Exception Handling

In the preceding example, declaring that the subroutine `root()` can throw an *IllegalArgumentException* is just a courtesy to potential readers of this routine. This is because handling of *IllegalArgumentExceptions* is not "mandatory." A routine can throw an *IllegalArgumentException* without announcing the possibility. And a program that calls that routine is free either to catch or to ignore the exception, just as a programmer can choose either to catch or to ignore an exception of type *NullPointerException*.

For those exception classes that require mandatory handling, the situation is different. If a subroutine can throw such an exception, that fact **must** be announced in a `throws` clause in the routine definition. Failing to do so is a syntax error that will be reported by the compiler.

On the other hand, suppose that some statement in the body of a subroutine can generate an exception of a type that requires mandatory handling. The statement could be a `throw` statement, which throws the exception directly, or it could be a call to a subroutine that can throw the exception. In either case, the exception **must** be handled. This can be done in one of two ways: The first way is to place the statement in a `try` statement that has a `catch` clause that handles the exception; in this case, the exception is handled within the subroutine, so that any caller of the subroutine will never see the exception. The second way is to declare that the subroutine can throw the exception. This is done by adding a "`throws`" clause to the subroutine heading, which alerts any callers to the possibility that an exception might be generated when the subroutine is executed. The caller will, in turn, be forced either to handle the exception in a `try` statement or to declare the exception in a `throws` clause in its own header.

Exception-handling is mandatory for any exception class that is not a subclass of either *Error* or *RuntimeException*. Exceptions that require mandatory handling generally represent conditions that are outside the control of the programmer. For example, they might represent bad input or an illegal action taken by the user. There is no way to **avoid** such errors, so a robust program has to be prepared to handle them. The design of Java makes it impossible for programmers to ignore the possibility of such errors.

Among the exceptions that require mandatory handling are several that can occur when using Java's input/output routines. This means that you can't even use these routines unless you understand something about exception-handling. Chapter 11 deals with input/output and uses mandatory exception-handling extensively.

8.3.5 Programming with Exceptions

Exceptions can be used to help write robust programs. They provide an organized and structured approach to robustness. Without exceptions, a program can become cluttered with `if` statements that test for various possible error conditions. With exceptions, it becomes possible to write a clean implementation of an algorithm that will handle all the normal cases. The exceptional cases can be handled elsewhere, in a `catch` clause of a `try` statement.

When a program encounters an exceptional condition and has no way of handling it immediately, the program can throw an exception. In some cases, it makes sense to throw an exception belonging to one of Java's predefined classes, such as *IllegalArgumentException* or *IOException*. However, if there is no standard class that adequately represents the exceptional condition, the programmer can define a new exception class. The new class must extend the standard class *Throwable* or one of its subclasses. In general, if the programmer does **not** want to require mandatory exception handling, the new class will extend *RuntimeException* (or one of its subclasses). To create a new exception class that **does** require mandatory handling, the programmer can extend one of the other subclasses of *Exception* or can extend *Exception* itself.

Here, for example, is a class that extends *Exception*, and therefore requires mandatory exception handling when it is used:

```
public class ParseError extends Exception {
   public ParseError(String message) {
         // Create a ParseError object containing
         // the given message as its error message.
      super(message);
   }
}
```

The class contains only a constructor that makes it possible to create a *ParseError* object containing a given error message. (The statement "`super(message)`" calls a constructor in the superclass, *Exception*. See Subsection 5.6.3.) Of course the class inherits the `getMessage()` and `printStackTrace()` routines from its superclass. If `e` refers to an object of type `ParseError`, then the function call `e.getMessage()` will retrieve the error message that was specified in the constructor. But the main point of the *ParseError* class is simply to exist. When an object of type *ParseError* is thrown, it indicates that a certain type of error has occurred. (***Parsing***, by the way, refers to figuring out the syntax of a string. A *ParseError* would indicate, presumably, that some string that is being processed by the program does not have the expected form.)

A `throw` statement can be used in a program to throw an error of type *ParseError*. The constructor for the *ParseError* object must specify an error message. For example:

```
throw new ParseError("Encountered an illegal negative number.");
```

or

```
throw new ParseError("The word '" + word
                              + "' is not a valid file name.");
```

If the `throw` statement does not occur in a `try` statement that catches the error, then the subroutine that contains the `throw` statement must declare that it can throw a *ParseError* by adding the clause "`throws ParseError`" to the subroutine heading. For example,

```
void getUserData() throws ParseError {
   . . .
}
```

This would not be required if *ParseError* were defined as a subclass of *RuntimeException* instead of *Exception*, since in that case exception handling for *ParseErrors* would not be mandatory.

A routine that wants to handle *ParseErrors* can use a `try` statement with a `catch` clause that catches *ParseErrors*. For example:

```
try {
   getUserData();
   processUserData();
}
catch (ParseError pe) {
   . . .  // Handle the error
}
```

Note that since *ParseError* is a subclass of *Exception*, a `catch` clause of the form "`catch (Exception e)`" would also catch *ParseErrors*, along with any other object of type *Exception*.

Sometimes, it's useful to store extra data in an exception object. For example,

```
class ShipDestroyed extends RuntimeException {
   Ship ship;  // Which ship was destroyed.
   int where_x, where_y;  // Location where ship was destroyed.
   ShipDestroyed(String message, Ship s, int x, int y) {
         // Constructor creates a ShipDestroyed object
         // carrying an error message plus the information
         // that the ship s was destroyed at location (x,y)
         // on the screen.
       super(message);
       ship = s;
       where_x = x;
       where_y = y;
   }
}
```

Here, a *ShipDestroyed* object contains an error message and some information about a ship that was destroyed. This could be used, for example, in a statement:

```
if ( userShip.isHit() )
   throw new ShipDestroyed("You've been hit!", userShip, xPos, yPos);
```

Note that the condition represented by a *ShipDestroyed* object might not even be considered an error. It could be just an expected interruption to the normal flow of a game. Exceptions can sometimes be used to handle such interruptions neatly.

* * *

The ability to throw exceptions is particularly useful in writing general-purpose subroutines and classes that are meant to be used in more than one program. In this case, the person writing the subroutine or class often has no reasonable way of handling the error, since that person has no way of knowing exactly how the subroutine or class will be used. In such circumstances, a novice programmer is often tempted to print an error message and forge ahead, but this is almost never satisfactory since it can lead to unpredictable results down the line. Printing an error message and terminating the program is almost as bad, since it gives the program no chance to handle the error.

The program that calls the subroutine or uses the class needs to know that the error has occurred. In languages that do not support exceptions, the only alternative is to return some special value or to set the value of some variable to indicate that an error has occurred. For

example, the `readMeasurement()` function in Subsection 8.2.2 returns the value `-1` if the user's input is illegal. However, this only does any good if the main program bothers to test the return value. It is very easy to be lazy about checking for special return values every time a subroutine is called. And in this case, using `-1` as a signal that an error has occurred makes it impossible to allow negative measurements. Exceptions are a cleaner way for a subroutine to react when it encounters an error.

It is easy to modify the `readMeasurement()` subroutine to use exceptions instead of a special return value to signal an error. My modified subroutine throws a *ParseError* when the user's input is illegal, where *ParseError* is the subclass of *Exception* that was defined above. (Arguably, it might be reasonable to avoid defining a new class by using the standard exception class *IllegalArgumentException* instead.) The changes from the original version are shown in italic:

```
/**
 * Reads the user's input measurement from one line of input.
 * Precondition:   The input line is not empty.
 * Postcondition:  If the user's input is legal, the measurement
 *                 is converted to inches and returned.
 * @throws ParseError if the user's input is not legal.
 */
static double readMeasurement() throws ParseError {

   double inches;  // Total number of inches in user's measurement.

   double measurement;  // One measurement,
                        //   such as the 12 in "12 miles."
   String units;        // The units specified for the measurement,
                        //   such as "miles."

   char ch;  // Used to peek at next character in the user's input.

   inches = 0;  // No inches have yet been read.

   skipBlanks();
   ch = TextIO.peek();

   /* As long as there is more input on the line, read a measurement and
      add the equivalent number of inches to the variable, inches.  If an
      error is detected during the loop, end the subroutine immediately
      by throwing a ParseError. */

   while (ch != '\n') {

       /* Get the next measurement and the units.  Before reading
          anything, make sure that a legal value is there to read. */

       if ( ! Character.isDigit(ch) ) {
           throw new ParseError("Expected to find a number, but found " + ch);
       }
       measurement = TextIO.getDouble();

       skipBlanks();
       if (TextIO.peek() == '\n') {
          throw new ParseError("Missing unit of measure at end of line.");
       }
       units = TextIO.getWord();
       units = units.toLowerCase();
```

```
        /* Convert the measurement to inches and add it to the total. */
        if (units.equals("inch")
                || units.equals("inches") || units.equals("in")) {
            inches += measurement;
        }
        else if (units.equals("foot")
                    || units.equals("feet") || units.equals("ft")) {
            inches += measurement * 12;
        }
        else if (units.equals("yard")
                    || units.equals("yards") || units.equals("yd")) {
            inches += measurement * 36;
        }
        else if (units.equals("mile")
                    || units.equals("miles") || units.equals("mi")) {
            inches += measurement * 12 * 5280;
        }
        else {
            throw new ParseError("\"" + units
                                 + "\" is not a legal unit of measure.");
        }

        /* Look ahead to see whether the next thing on the line is
           the end-of-line. */

        skipBlanks();
        ch = TextIO.peek();

    }  // end while

    return inches;

} // end readMeasurement()
```

In the main program, this subroutine is called in a `try` statement of the form

```
try {
   inches = readMeasurement();
}
catch (ParseError e) {
   . . .  // Handle the error.
}
```

The complete program can be found in the file *LengthConverter3.java*. From the user's point of view, this program has exactly the same behavior as the program *LengthConverter2* from the previous section. Internally, however, the programs are significantly different, since LengthConverter3 uses exception-handling.

8.4 Assertions

In this short section, we look at ***assertions***, another feature of the Java programming language that can be used to aid in the development of correct and robust programs.

Recall that a precondition is a condition that must be true at a certain point in a program, for the execution of the program to continue correctly from that point. In the case where

there is a chance that the precondition might not be satisfied—for example, if it depends on input from the user—then it's a good idea to insert an `if` statement to test it. But then the question arises, What should be done if the precondition does not hold? One option is to throw an exception. This will terminate the program, unless the exception is caught and handled elsewhere in the program.

In many cases, of course, instead of using an `if` statement to *test* whether a precondition holds, a programmer tries to write the program in a way that will *guarantee* that the precondition holds. In that case, the test should not be necessary, and the `if` statement can be avoided. The problem is that programmers are not perfect. In spite of the programmer's intention, the program might contain a bug that screws up the precondition. So maybe it's a good idea to check the precondition—at least during the debugging phase of program development.

Similarly, a postcondition is a condition that is true at a certain point in the program as a consequence of the code that has been executed before that point. Assuming that the code is correctly written, a postcondition is guaranteed to be true, but here again testing whether a desired postcondition is **actually** true is a way of checking for a bug that might have screwed up the postcondition. This is somthing that might be desirable during debugging.

The programming languages C and C++ have always had a facility for adding what are called ***assertions*** to a program. These assertions take the form "`assert(`⟨*condition*⟩`)`", where ⟨*condition*⟩ is a **boolean**-valued expression. This condition expresses a precondition or postcondition that should hold at that point in the program. When the computer encounters an assertion during the execution of the program, it evaluates the condition. If the condition is false, the program is terminated. Otherwise, the program continues normally. This allows the programmer's belief that the condition is true to be tested; if if it not true, that indicates that the part of the program that preceded the assertion contained a bug. One nice thing about assertions in C and C++ is that they can be "turned off" at compile time. That is, if the program is compiled in one way, then the assertions are included in the compiled code. If the program is compiled in another way, the assertions are not included. During debugging, the first type of compilation is used. The release version of the program is compiled with assertions turned off. The release version will be more efficient, because the computer won't have to evaluate all the assertions.

Although early versions of Java did not have assertions, an assertion facility similar to the one in C/C++ has been available in Java since version 1.4. As with the C/C++ version, Java assertions can be turned on during debugging and turned off during normal execution. In Java, however, assertions are turned on and off at run time rather than at compile time. An assertion in the Java source code is always included in the compiled class file. When the program is run in the normal way, these assertions are ignored; since the condition in the assertion is not evaluated in this case, there is little or no performance penalty for having the assertions in the program. When the program is being debugged, it can be run with assertions enabled, as discussed below, and then the assertions can be a great help in locating and identifying bugs.

* * *

An ***assertion statement*** in Java takes one of the following two forms:

```
assert ⟨condition⟩ ;
```

or

```
assert ⟨condition⟩ : ⟨error-message⟩ ;
```

where ⟨*condition*⟩ is a **boolean**-valued expression and ⟨*error-message*⟩ is a string or an expression of type *String*. The word "`assert`" is a reserved word in Java, which cannot be used as an

identifier. An assertion statement can be used anyplace in Java where a statement is legal.

If a program is run with assertions disabled, an assertion statement is equivalent to an empty statement and has no effect. When assertions are enabled and an assertion statement is encountered in the program, the ⟨*condition*⟩ in the assertion is evaluated. If the value is `true`, the program proceeds normally. If the value of the condition is `false`, then an exception of type `java.lang.AssertionError` is thrown, and the program will crash (unless the error is caught by a `try` statement). If the `assert` statement includes an ⟨*error-message*⟩, then the error message string becomes the message in the *AssertionError*.

So, the statement "`assert` ⟨*condition*⟩ `:` ⟨*error-message*⟩`;`" is similar to

```
if ( ⟨condition⟩ == false )
    throw new AssertionError( ⟨error-message⟩ );
```

except that the `if` statement is executed whenever the program is run, and the `assert` statement is executed only when the program is run with assertions enabled.

The question is, when to use assertions instead of exceptions? The general rule is to use assertions to test conditions that should definitely be true, if the program is written correctly. Assertions are useful for testing a program to see whether or not it is correct and for finding the errors in an incorrect program. After testing and debugging, when the program is used in the normal way, the assertions in the program will be ignored. However, if a problem turns up later, the assertions are still there in the program to be used to help locate the error. If someone writes to you to say that your program doesn't work when he does such-and-such, you can run the program with assertions enabled, do such-and-such, and hope that the assertions in the program will help you locate the point in the program where it goes wrong.

Consider, for example, the `root()` method from Subsection 8.3.3 that calculates a root of a quadratic equation. If you believe that your program will always call this method with legal arguments, then it would make sense to write the method using assertions instead of exceptions:

```
/**
 * Returns the larger of the two roots of the quadratic equation
 * A*x*x + B*x + C = 0, provided it has any roots.
 * Precondition: A != 0 and B*B - 4*A*C >= 0.
 */
static public double root( double A, double B, double C )  {
   assert A != 0 : "Leading coefficient of quadratic equation cannot be zero.";
   double disc = B*B - 4*A*C;
   assert disc >= 0 : "Discriminant of quadratic equation cannot be negative.";
   return  (-B + Math.sqrt(disc)) / (2*A);
}
```

The assertions are not checked when the program is run in the normal way. If you are correct in your belief that the method is never called with illegal arguments, then checking the conditions in the assertions would be unnecessary. If your belief is not correct, the problem should turn up during testing or debugging, when the program is run with the assertions enabled.

If the `root()` method is part of a software library that you expect other people to use, then the situation is less clear. Sun's Java documentation advises that assertions should **not** be used for checking the contract of public methods: If the caller of a method violates the contract by passing illegal parameters, then an exception should be thrown. This will enforce the contract whether or not assertions are enabled. (However, while it's true that Java programmers *expect* the contract of a method to be enforced with exceptions, there are reasonable arguments for using assertions instead, in some cases.)

On the other hand, it never hurts to use an assertion to check a postcondition of a method. A postcondition is something that is supposed to be true after the method has executed, and it can be tested with an `assert` statement at the end of the method. If the postcodition is false, there is a bug in the method itself, and that is something that needs to be found during the development of the method.

* * *

To have any effect, assertions must be **enabled** when the program is run. How to do this depends on what programming environment you are using. (See Section 2.6 for a discussion of programming environments.) In the usual command line environment, assertions are enabled by adding the option `-enableassertions` to the `java` command that is used to run the program. For example, if the class that contains the main program is *RootFinder*, then the command

```
java -enableassertions RootFinder
```

will run the program with assertions enabled. The `-enableassertions` option can be abbreviated to `-ea`, so the command can alternatively be written as

```
java -ea RootFinder
```

In fact, it is possible to enable assertions in just part of a program. An option of the form "`-ea:`⟨*class-name*⟩" enables only the assertions in the specified class. Note that there are no spaces between the `-ea`, the ":", and the name of the class. To enable all the assertions in a package and in its sub-packages, you can use an option of the form "`-ea:`⟨*package-name*⟩`...`". To enable assertions in the "default package" (that is, classes that are not specified to belong to a package, like almost all the classes in this book), use "`-ea:...`". For example, to run a Java program named "MegaPaint" with assertions enabled for every class in the packages named "paintutils" and "drawing", you would use the command:

```
java  -ea:paintutils...  -ea:drawing...  MegaPaint
```

If you are using the Eclipse integrated development environment, you can specify the `-ea` option by creating a ***run configuration***. Right-click the name of the main program class in the Package Explorer pane, and select "Run As" from the pop-up menu and then "Run..." from the submenu. This will open a dialog box where you can manage run configurations. The name of the project and of the main class will be already be filled in. Click the "Arguments" tab, and enter `-ea` in the box under "VM Arguments". The contents of this box are added to the `java` command that is used to run the program. You can enter other options in this box, including more complicated `enableassertions` options such as `-ea:paintutils...`. When you click the "Run" button, the options will be applied. Furthermore, they will be applied whenever you run the program, unless you change the run configuration or add a new configuration. Note that it is possible to make two run configurations for the same class, one with assertions enabled and one with assertions disabled.

8.5 Introduction to Threads

LIKE PEOPLE, COMPUTERS can ***multitask***. That is, they can be working on several different tasks at the same time. A computer that has just a single central processing unit can't literally do two things at the same time, any more than a person can, but it can still switch its attention back and forth among several tasks. Furthermore, it is increasingly common for computers to have more than one processing unit, and such computers can literally work on several tasks simultaneously. It is likely that from now on, most of the increase in computing power will

come from adding additional processors to computers rather than from increasing the speed of individual processors. To use the full power of these multiprocessing computers, a programmer must do ***parallel programming***, which means writing a program as a set of several tasks that can be executed simultaneously. Even on a single-processor computer, parallel programming techniques can be useful, since some problems can be tackled most naturally by breaking the solution into a set of simultaneous tasks that cooperate to solve the problem.

In Java, a single task is called a ***thread***. The term "thread" refers to a "thread of control" or "thread of execution," meaning a sequence of instructions that are executed one after another—the thread extends through time, connecting each instruction to the next. In a multithreaded program, there can be many threads of control, weaving through time in parallel and forming the complete fabric of the program. (Ok, enough with the metaphor, already!) Every Java program has at least one thread; when the Java virtual machine runs your program, it creates a thread that is responsible for executing the `main` routine of the program. This main thread can in turn create other threads that can continue even after the main thread has terminated. In a GUI program, there is at least one additional thread, which is responsible for handling events and drawing components on the screen. This GUI thread is created when the first window is opened. So in fact, you have already done parallel programming! When a `main` routine opens a window, both the main thread and the GUI thread can continue to run in parallel. Of course, parallel programming can be used in much more interesting ways.

Unfortunately, parallel programming is even more difficult than ordinary, single-threaded programming. When several threads are working together on a problem, a whole new category of errors is possible. This just means that techniques for writing correct and robust programs are even more important for parallel programming than they are for normal programming. (That's one excuse for having this section in this chapter—another is that we will need threads at several points in future chapters, and I didn't have another place in the book where the topic fits more naturally.) Since threads are a difficult topic, you will probably not fully understand everything in this section the first time through the material. Your understanding should improve as you encounter more examples of threads in future sections.

8.5.1 Creating and Running Threads

In Java, a thread is represented by an object belonging to the class `java.lang.Thread` (or to a subclass of this class). The purpose of a *Thread* object is to execute a single method. The method is executed in its own thread of control, which can run in parallel with other threads. When the execution of the method is finished, either because the method terminates normally or because of an uncaught exception, the thread stops running. Once this happens, there is no way to restart the thread or to use the same *Thread* object to start another thread.

There are two ways to program a thread. One is to create a subclass of *Thread* and to define the method `public void run()` in the subclass. This `run()` method defines the task that will be performed by the thread; that is, when the thread is started, it is the `run()` method that will be executed in the thread. For example, here is a simple, and rather useless, class that defines a thread that does nothing but print a message on standard output:

```
public class NamedThread extends Thread {
   private String name;  // The name of this thread.
   public NamedThread(String name) {  // Constructor gives name to thread.
      this.name = name;
   }
   public void run() {  // The run method prints a message to standard output.
```

```
        System.out.println("Greetings from thread '" + name + "'!");
    }
}
```

To use a *NamedThread*, you must of course create an object belonging to this class. For example,

```
NamedThread greetings = new NamedThread("Fred");
```

However, creating the object does not automatically start the thread running. To do that, you must call the `start()` method in the thread object. For the example, this would be done with the statement

```
greetings.start();
```

The purpose of the `start()` method is to create a new thread of control that will execute the *Thread* object's `run()` method. The new thread runs in parallel with the thread in which the `start()` method was called, along with any other threads that already existed. This means that the code in the `run()` method will execute at the same time as the statements that follow the call to `greetings.start()`. Consider this code segment:

```
NamedThread greetings = new NamedThread("Fred");
greetings.start();
System.out.println("Thread has been started.");
```

After `greetings.start()` is executed, there are two threads. One of them will print "Thread has been started." while the other one wants to print "Greetings from thread 'Fred'!". It is important to note that *these messages can be printed in either order*. The two threads run simultaneously and will compete for access to standard output, so that they can print their messages. Whichever thread happens to be the first to get access will be the first to print its message. In a normal, single-threaded program, things happen in a definite, predictable order from beginning to end. In a multi-threaded program, there is a fundamental indeterminancy. You can't be sure what order things will happen in. This indeterminacy is what makes parallel programming so difficult!

Note that calling `greetings.start()` is **very** different from calling `greetings.run()`. Calling `greetings.run()` will execute the `run()` method in the same thread, rather than creating a new thread. This means that all the work of the `run()` will be done before the computer moves on to the statement that follows the call to `greetings.run()` in the program. There is no parallelism and no indeterminacy.

* * *

I mentioned that there are two ways to program a thread. The first way was to define a subclass of *Thread*. The second is to define a class that implements the interface `java.lang.Runnable`. The *Runnable* interface defines a single method, `public void run()`. An object that implements the *Runnable* interface can be passed as a parameter to the constructor of an object of type *Thread*. When that thread's `start` method is called, the thread will execute the `run()` method in the *Runnable* object. For example, as an alternative to the *NamedThread* class, we could define the class:

```
public class NamedRunnable implements Runnable {
   private String name;  // The name of this thread.
   public NamedRunnable(String name) {  // Constructor gives name to object.
      this.name = name;
   }
```

```
   public void run() {  // The run method prints a message to standard output.
      System.out.println("Greetings from thread '" + name +"'!");
   }
}
```

To use this version of the class, we would create a *NamedRunnable* object and use that object to create an object of type *Thread*:

```
NamedRunnable greetings = new NamedRunnable("Fred");
Thread greetingsThread = new Thread(greetings);
greetingsThread.start();
```

Finally, I'll note that it is sometimes convenient to define a thread using an anonymous inner class (Subsection 5.7.3). For example:

```
Thread greetingsFromFred = new Thread() {
   public void run() {
      System.out.println("Greetings from Fred!");
   }
};
greetingsFromFred.start();
```

* * *

To help you understand how multiple threads are executed in parallel, we consider the sample program *ThreadTest1.java*. This program creates several threads. Each thread performs exactly the same task. The task is to count the number of integers less than 1000000 that are prime. (The particular task that is done is not important for our purposes here.) On my computer, this task takes a little more than one second of processing time. The threads that perform this task are defined by the following static nested class:

```
/**
 * When a thread belonging to this class is run it will count the
 * number of primes between 2 and 1000000.  It will print the result
 * to standard output, along with its ID number and the elapsed
 * time between the start and the end of the computation.
 */
private static class CountPrimesThread extends Thread {
   int id;  // An id number for this thread; specified in the constructor.
   public CountPrimesThread(int id) {
      this.id = id;
   }
   public void run() {
      long startTime = System.currentTimeMillis();
      int count = countPrimes(2,1000000); // Counts the primes.
      long elapsedTime = System.currentTimeMillis() - startTime;
      System.out.println("Thread " + id + " counted " +
            count + " primes in " + (elapsedTime/1000.0) + " seconds.");
   }
}
```

The main program asks the user how many threads to run, and then creates and starts the specified number of threads:

```
public static void main(String[] args) {
   int numberOfThreads = 0;
   while (numberOfThreads < 1 || numberOfThreads > 25) {
      System.out.print("How many threads do you want to use  (1 to 25) ?  ");
      numberOfThreads = TextIO.getlnInt();
      if (numberOfThreads < 1 || numberOfThreads > 25)
         System.out.println("Please enter a number between 1 and 25 !");
   }
   System.out.println("\nCreating " + numberOfThreads
                                          + " prime counting threads...");
   CountPrimesThread[] worker = new CountPrimesThread[numberOfThreads];
   for (int i = 0; i < numberOfThreads; i++)
      worker[i] = new CountPrimesThread( i );
   for (int i = 0; i < numberOfThreads; i++)
      worker[i].start();
   System.out.println("Threads have been created and started.");
}
```

1

It would be a good idea for you to compile and run the program or to try the applet version, which can be found in the on-line version of this section. 2

When I ran the program with one thread, it took 1.18 seconds for my computer to do the computation. When I ran it using six threads, the output was: 3

```
Creating 6 prime counting threads...
Threads have been created and started.
Thread 1 counted 78498 primes in 6.706 seconds.
Thread 4 counted 78498 primes in 6.693 seconds.
Thread 0 counted 78498 primes in 6.838 seconds.
Thread 2 counted 78498 primes in 6.825 seconds.
Thread 3 counted 78498 primes in 6.893 seconds.
Thread 5 counted 78498 primes in 6.859 seconds.
```

4

The second line was printed immediately after the first. At this point, the main program has ended but the six threads continue to run. After a pause of about seven seconds, all six threads completed at about the same time. The order in which the threads complete is not the same as the order in which they were started, and the order is indeterminate. That is, if the program is run again, the order in which the threads complete will probably be different. 5

On my computer, six threads take about six times longer than one thread. This is because my computer has only one processor. Six threads, all doing the same task, take six times as much processing as one thread. With only one processor to do the work, the total elapsed time for six threads is about six times longer than the time for one thread. On a computer with two processors, the computer can work on two tasks at the same time, and six threads might complete in as little as three times the time it takes for one thread. On a computer with six or more processors, six threads might take no more time than a single thread. Because of overhead and other reasons, the actual speedup will probably be smaller than this analysis indicates, but on a multiprocessor machine, you should see a definite speedup. What happens when you run the program on your own computer? How many processors do you have? 6

Whenever there are more threads to be run than there are processors to run them, the computer divides its attention among all the runnable threads by switching rapidly from one thread to another. That is, each processor runs one thread for a while then switches to another thread and runs that one for a while, and so on. Typically, these “context switches” occur about 100 times or more per second. The result is that the computer makes progress on all 7

the tasks, and it looks to the user as if all the tasks are being executed simultaneously. This is why in the sample program, in which each thread has the same amount of work to do, all the threads complete at about the same time: Over any time period longer than a fraction of a second, the computer's time is divided approximately equally among all the threads.

When you do parallel programming in order to spread the work among several processors, you might want to take into account the number of available processors. You might, for example, want to create one thread for each processor. In Java, you can find out the number of processors by calling the function

```
Runtime.getRuntime().availableProcessors()
```

which returns an **int** giving the number of processors that are available to the Java Virtual Machine. In some cases, this might be less than the actual number of processors in the computer.

8.5.2 Operations on Threads

The *Thread* class includes several useful methods in addition to the `start()` method that was discussed above. I will mention just a few of them.

If `thrd` is an object of type *Thread*, then the **boolean**-valued function `thrd.isAlive()` can be used to test whether or not the thread is alive. A thread is "alive" between the time it is started and the time when it terminates. After the thread has terminated it is said to be "dead". (The rather gruesome metaphor is also used when we refer to "killing" or "aborting" a thread.)

The static method `Thread.sleep(milliseconds)` causes the thread that executes this method to "sleep" for the specified number of milliseconds. A sleeping thread is still alive, but it is not running. While a thread is sleeping, the computer will work on any other runnable threads (or on other programs). `Thread.sleep()` can be used to insert a pause in the execution of a thread. The `sleep` method can throw an exception of type *InterruptedException*, which is an exception class that requires mandatory exception handling (see Subsection 8.3.4). In practice, this means that the `sleep` method is usually used in a `try..catch` statement that catches the potential *InterruptedException*:

```
try {
   Thread.sleep(lengthOfPause);
}
catch (InterruptedException e) {
}
```

One thread can interrupt another thread to wake it up when it is sleeping or paused for some other reason. A *Thread*, `thrd`, can be interrupted by calling its method `thrd.interrupt()`, but you are not likely to do this until you start writing rather advanced applications, and you are not likely to need to do anything in response to an *InterruptedException* (except to `catch` it). It's unfortunate that you have to worry about it at all, but that's the way that mandatory exception handling works.

Sometimes, it's necessary for one thread to wait for anther thread to die. This is done with the `join()` method from the *Thread* class. Suppose that `thrd` is a *Thread*. Then, if another thread calls `thrd.join()`, that other thread will go to sleep until `thrd` terminates. If `thrd` is already dead when `thrd.join()` is called, then it simply has no effect—the thread that called `thrd.join()` proceeds immediately. The method `join()` can throw an `InterruptedException`, which must be handled. As an example, the following code starts several threads, waits for them all to terminate, and then outputs the elapsed time:

```
CountPrimesThread[] worker = new CountPrimesThread[numberOfThreads];
long startTime = System.currentTimeMillis();
for (int i = 0; i < numberOfThreads; i++) {
   worker[i] = new CountPrimesThread();
   worker[i].start();
}
for (int i = 0; i < numberOfThreads; i++) {
   try {
      worker[i].join();  // Sleep until worker[i] has terminated.
   }
   catch (InterruptedException e) {
   }
}
// At this point, all the worker threads have terminated.
long elapsedTime = System.currentTimeMillis() - startTime;
System.out.println("Elapsed time:  " + (elapsedTime/1000.0) + " seconds.");
```

An observant reader will note that this code assumes that no *InterruptedException* will occur. To be absolutely sure that the thread `worker[i]` has terminated in an environment where *InterruptedExceptions* are possible, you would have to do something like:

```
while (worker[i].isAlive()) {
   try {
      worker[i].join();
   }
   catch (InterruptedException e) {
   }
}
```

8.5.3 Mutual Exclusion with "synchronized"

Programming several threads to carry out independent tasks is easy. The real difficulty arises when threads have to interact in some way. One way that threads interact is by sharing resources. When two threads need access to the same resource, such as a variable or a window on the screen, some care must be taken that they don't try to use the same resource at the same time. Otherwise, the situation could be something like this: Imagine several cooks sharing the use of just one measuring cup, and imagine that Cook A fills the measuring cup with milk, only to have Cook B grab the cup before Cook A has a chance to empty the milk into his bowl. There has to be some way for Cook A to claim exclusive rights to the cup while he performs the two operations: Add-Milk-To-Cup and Empty-Cup-Into-Bowl.

Something similar happens with threads, even with something as simple as adding one to a counter. The statement

```
count = count + 1;
```

is actually a sequence of three operations:

```
Step 1.  Get the value of count
Step 2.  Add 1 to the value.
Step 3.  Store the new value in count
```

Suppose that several threads perform these three steps. Remember that it's possible for two threads to run at the same time, and even if there is only one processor, it's possible for that processor to switch from one thread to another at any point. Suppose that while one thread is between Step 2 and Step 3, another thread starts executing the same sequence of steps. Since the first thread has not yet stored the new value in `count`, the second thread reads the **old** value of `count` and adds one to that old value. After both threads have executed Step 3, the value of `count` has gone up only by 1 instead of by 2! This type of problem is called a ***race condition***. This occurs when one thread is in the middle of a multi-step operation, and another thread changes some value or condition that the first thread is depending upon. (The first thread is "in a race" to complete all the steps before it is interrupted by another thread.) Another example of a race condition can occur in an `if` statement. Suppose the following statement, which is meant to avoid a division-by-zero error is executed by a thread:

```
if ( A != 0 )
   B = C / A;
```

If the variable `A` is shared by several threads, and if nothing is done to guard against the race condition, then it is possible that a second thread will change the value of `A` to zero between the time that the first thread checks the condition `A != 0` and the time that it does the division. This means that the thread ends up dividing by zero, even though it just checked that `A` was not zero!

To fix the problem of race conditions, there has to be some way for a thread to get ***exclusive access*** to a shared resource. This is not a trivial thing to implement, but Java provides a high level and relatively easy-to-use approach to exclusive access. It's done with ***synchronized methods*** and with the ***synchronized statement***. These are used to protect shared resources by making sure that only one thread at a time will try to access the resource. Synchronization in Java actually provides only ***mutual exclusion***, which means that exclusive access to a resource is only guaranteed if **every** thread that needs access to that resource uses synchronization. Synchronization is like a cook leaving a note that says, "I'm using the measuring cup." This will get the cook exclusive access to the cup—but only if all the cooks agree to check the note before trying to grab the cup.

Because this is a difficult topic, I will start with a simple example. Suppose that we want to avoid the race condition that occurs when several threads all want to add 1 to a counter. We can do this by defining a class to represent the counter and by using synchronized methods in that class:

```
public class ThreadSafeCounter {

   private int count = 0;  // The value of the counter.

   synchronized public void increment() {
      count = count + 1;
   }

   synchronized public int getValue() {
      return count;
   }

}
```

If `tsc` is of type *ThreadSafeCounter*, then any thread can call `tsc.increment()` to add 1 to the counter in a completely safe way. The fact that `tsc.increment()` is `synchronized` means that only one thread can be in this method at a time; once a thread starts executing this

method, it is guaranteed that it will finish executing it without having another thread change the value of `tsc.count` in the meantime. There is no possibility of a race condition. Note that the guarantee depends on the fact that `count` is a `private` variable. This forces all access to `tsc.count` to occur in the `synchronized` methods that are provided by the class. If `count` were `public`, it would be possible for a thread to bypass the synchronization by, for example, saying `tsc.count++`. This could change the value of `count` while another thread is in the middle of the `tsc.increment()`. Synchronization does **not** guarantee exclusive access; it only guarantees **mutual exclusion** among all the threads that are properly synchronized.

The *ThreadSafeCounter* class does not prevent all possible race conditions that might arise when using a counter. Consider the `if` statement:

```
if ( tsc.getValue() == 0 )
   doSomething();
```

where `doSomething()` is some method that requires the value of the counter to be zero. There is still a race condition here, which occurs if a second thread increments the counter between the time the first thread tests `tsc.getValue() == 0` and the time it executes `doSomething()`. The first thread needs exclusive access to the counter during the execution of the whole `if` statement. (The synchronization in the *ThreadSafeCounter* class only gives it exclusive access during the time it is evaluating `tsc.getValue()`.) We can solve the race condition by putting the `if` statement in a `synchronized` statement:

```
synchronized(tsc) {
   if ( tsc.getValue() == 0 )
      doSomething();
}
```

Note that the `synchronized` statement takes an object—`tsc` in this case—as a kind of parameter. The syntax of the `synchronized` statement is:

```
synchronized( ⟨object⟩ ) {
   ⟨statements⟩
}
```

In Java, mutual exclusion is always associated with an object; we say that the synchronization is "on" that object. For example, the `if` statement above is "synchronized on `tsc`." A synchronized instance method, such as those in the class *ThreadSafeCounter*, is synchronized on the object that contains the instance method. In fact, adding the `synchronized` modifier to the definition of an instance method is pretty much equivalent to putting the body of the method in a `synchronized` statement, `synchronized(this) {...}`. It is also possible to have synchronized static methods; a synchronized static method is synchronized on a special class object that represents the class that contains the static method.

The real rule of synchronization in Java is: **Two threads cannot be synchronized on the same object at the same time**; that is, they cannot simultaneously be executing code segments that are synchronized on that object. If one thread is synchronized on an object, and a second thread tries to synchronize on the same object, the second thread is forced to wait until the first thread has finished with the object. This is implemented using something called a ***lock***. Every object has a lock, and that lock can be "held" by only one thread at a time. To enter a synchronized statement or synchronized method, a thread must obtain the associated object's lock. If the lock is available, then the thread obtains the lock and immediately begins executing the synchronized code. It releases the lock after it finishes executing the synchronized code. If Thread A tries to obtain a lock that is already held by Thread B, then Thread A has

to wait until Thread B releases the lock. In fact, Thread A will go to sleep, and will not be awoken until the lock becomes available.

* * *

As a simple example of shared resources, we return to the prime-counting problem. Suppose that we want to count all the primes in a given range of integers, and suppose that we want to divide the work up among several threads. Each thread will be assigned part of the range of integers and will count the primes in its assigned range. At the end of its computation, the thread has to add its count to the overall total number of primes found. The variable that represents the total is shared by all the threads. If each thread just says

```
total = total + count;
```

then there is a (small) chance that two threads will try to do this at the same time and that the final total will be wrong. To prevent this race condition, access to `total` has to be synchronized. My program uses a synchronized method to add the counts to the total:

```
synchronized private static void addToTotal(int x) {
   total = total + x;
   System.out.println(total + " primes found so far.");
}
```

The source code for the program can be found in *ThreadTest2.java*. This program counts the primes in the range 3000001 to 6000000. (The numbers are rather arbitrary.) The `main()` routine in this program creates between 1 and 5 threads and assigns part of the job to each thread. It then waits for all the threads to finish, using the `join()` method as described above, and reports the total elapsed time. If you run the program on a multiprocessor computer, it should take less time for the program to run when you use more than one thread. You can compile and run the program or try the equivalent applet in the on-line version of this section.

* * *

Synchronization can help to prevent race conditions, but it introduces the possibility of another type of error, ***deadlock***. A deadlock occurs when a thread waits forever for a resource that it will never get. In the kitchen, a deadlock might occur if two very simple-minded cooks both want to measure a cup of milk at the same time. The first cook grabs the measuring cup, while the second cook grabs the milk. The first cook needs the milk, but can't find it because the second cook has it. The second cook needs the measuring cup, but can't find it because the first cook has it. Neither cook can continue and nothing more gets done. This is deadlock. Exactly the same thing can happen in a program, for example if there are two threads (like the two cooks) both of which need to obtain locks on the same two objects (like the milk and the measuring cup) before they can proceed. Deadlocks can easily occur, unless great care is taken to avoid them. Fortunately, we won't be looking at any examples that require locks on more than one object, so we will avoid that source of deadlock.

8.5.4 Wait and Notify

Threads can interact with each other in other ways besides sharing resources. For example, one thread might produce some sort of result that is needed by another thread. This imposes some restriction on the order in which the threads can do their computations. If the second thread gets to the point where it needs the result from the first thread, it might have to stop and wait for the result to be produced. Since the second thread can't continue, it might as well go to sleep. But then there has to be some way to notify the second thread when the result is

ready, so that it can wake up and continue its computation. Java, of course, has a way to do this kind of waiting and notification: It has `wait()` and `notify()` methods that are defined as instance methods in class *Object* and so can be used with any object. The reason why `wait()` and `notify()` should be associated with objects is not obvious, so don't worry about it at this point. It does, at least, make it possible to direct different notifications to a different recipients, depending on which object's `notify()` method is called.

The general idea is that when a thread calls a `wait()` method in some object, that thread goes to sleep until the `notify()` method in the same object is called. It will have to be called, obviously, by another thread, since the thread that called `wait()` is sleeping. A typical pattern is that Thread A calls `wait()` when it needs a result from Thread B, but that result is not yet available. When Thread B has the result ready, it calls `notify()`, which will wake Thread A up so that it can use the result. It is not an error to call `notify()` when no one is waiting; it just has no effect. To implement this, Thread A will execute code simlar to the following, where `obj` is some object:

```
if ( resultIsAvailable() == false )
   obj.wait();  // wait for noification that the result is available
useTheResult();
```

while Thread B does something like:

```
generateTheResult();
obj.notify();  // send out a notification that the result is available
```

Now, there is a really nasty race condition in this code. The two threads might execute their code in the following order:

```
1.  Thread A checks resultIsAvailable() and finds that the result is not ready,
        so it decides to execute the obj.wait() statement, but before it does,
2.  Thread B finishes generating the result and calls obj.notify()
3.  Thread A calls obj.wait() to wait for notification that the result is ready.
```

In Step 3, Thread A is waiting for a notification that will never come, because `notify()` has already been called. This is a kind of deadlock that can leave Thread A waiting forever. Obviously, we need some kind of synchronization. The solution is to enclose both Thread A's code and Thread B's code in `synchronized` statements, and it is very natural to synchronize on the same object, `obj`, that is used for the calls to `wait()` and `notify()`. In fact, since synchronization is almost always needed when `wait()` and `notify()` are used, Java makes it an absolute requirement. In Java, a thread can legally call `obj.wait()` or `obj.notify()` only if that thread holds the synchronization lock associated with the object `obj`. If it does not hold that lock, then an exception is thrown. (The exception is of type *IllegalMonitorStateException*, which does not require mandatory handling and which is typically not caught.) One further complication is that the `wait()` method can throw an *InterruptedException* and so should be called in a `try` statement that handles the exception.

To make things more definite, lets consider a ***producer/consumer*** problem where one thread produces a result that is consumed by another thread. Assume that there is a shared variable named `sharedResult` that is used to transfer the result from the producer to the consumer. When the result is ready, the producer sets the variable to a non-null value. The producer can check whether the result is ready by testing whether the value of `sharedResult` is null. We will use a variable named `lock` for synchronization. The the code for the producer thread could have the form:

```
makeResult = generateTheResult();  // Not synchronized!
synchronized(lock) {
   sharedResult = makeResult;
   lock.notify();
}
```

while the consumer would execute code such as:

```
synchronized(lock) {
   while ( sharedResult == null ) {
      try {
         lock.wait();
      }
      catch (InterruptedException e) {
      }
   }
   useResult = sharedResult;
}
useTheResult(useResult);  // Not synchronized!
```

The calls to `generateTheResult()` and `useTheResult()` are not synchronized, which allows them to run in parallel with other threads that might also synchronize on `lock`. Since `sharedResult` is a shared variable, all references to `sharedResult` should be synchronized, so the references to `sharedResult` must be inside the `synchronized` statements. The goal is to do as little as possible (but not less) in synchronized code segments.

If you are uncommonly alert, you might notice something funny: `lock.wait()` does not finish until `lock.notify()` is executed, but since both of these methods are called in `synchronized` statements that synchronize on the same object, shouldn't it be impossible for both methods to be running at the same time? In fact, `lock.wait()` is a special case: When the consumer thread calls `lock.wait()`, it gives up the lock that it holds on the synchronization object, `lock`. This gives the producer thread a chance to execute the `synchronized(lock)` block that contains the `lock.notify()` statement. After the producer thread exits from this block, the lock is returned to the consumer thread so that it can continue.

The producer/consumer pattern can be generalized and made more useful without making it any more complex. In the general case, multiple results are produced by one or more producer threads and are consumed by one or more consumer threads. Instead of having just one `sharedResult` object, we keep a list of objects that have been produced but not yet consumed. Producer threads add objects to this list. Consumer threads remove objects from this list. The only time when a thread is blocked from running is when a consumer thread tries to get a result from the list, and no results are available. It is easy to encapsulate the whole producer/consumer pattern in a class (where I assume that there is a class *ResultType* that represents the result objects):

```
/**
 * An object of type ProducerConsumer represents a list of results
 * that are available for processing.  Results are added to the list
 * by calling the produce method and are remove by calling consume.
 * If no result is available when consume is called, the method will
 * not return until a result becomes available.
 */
private static class ProducerConsumer {

   private ArrayList<ResultType> items = new ArrayList<ResultType>();
```

```
   // This ArrayList holds results that have been produced and are waiting
   // to be consumed.  See Subsection 7.3.3 for information on ArrayList.

   public void produce(ResultType item) {
      synchronized(items) {
         items.add(item); // Add item to the list of results.
         items.notify();  // Notify any thread waiting in consume() method.
      }
   }

   public ResultType consume() {
      ResultType item;
      synchronized(items) {
         // If no results are available, wait for notification from produce().
         while (items.size() == 0) {
            try {
               items.wait();
            }
            catch (InterruptedException e) {
            }
         }
         // At this point, we know that at least one result is available.
         item = items.remove(0);
      }
      return item;
   }

}
```

For an example of a program that uses a *ProducerConsumer* class, see *ThreadTest3.java*. This program performs the same task as *ThreadTest2.java*, but the threads communicate using the producer/consumer pattern instead of with a shared variable.

Going back to our kitchen analogy for a moment, consider a restaurant with several waiters and several cooks. If we look at the flow of customer orders into the kitchen, the waiters "produce" the orders and leave them in a pile. The orders are "consumed" by the cooks; whenever a cook needs a new order to work on, she picks one up from the pile. The pile of orders, or course, plays the role of the list of result objects in the producer/consumer pattern. Note that the only time that a cook has to wait is when she needs a new order to work on, and there are no orders in the pile. The cook must wait until one of the waiters places an order in the pile. We can complete the analogy by imagining that the waiter rings a bell when he places the order in the pile—ringing the bell is like calling the `notify()` method to notify the cooks that an order is available.

A final note on `notify`: It is possible for several threads to be waiting for notification. A call to `obj.notify()` will wake only one of the threads that is waiting on `obj`. If you want to wake all threads that are waiting on `obj`, you can call `obj.notifyAll()`. And a final note on `wait`: There is an another version of `wait()` that takes a number of milliseconds as a parameter. A thread that calls `obj.wait(milliseconds)` will wait only up to the specified number of milliseconds for a notification. If a notification doesn't occur during that period, the thread will wake up and continue without the notification. In practice, this feature is most often used to let a waiting thread wake periodically while it is waiting in order to perform some periodic task, such as causing a message "Waiting for computation to finish" to blink.

8.5.5 Volatile Variables

And a final note on communication among threads: In general, threads communicate by sharing variables and accessing those variables in synchronized methods or synchronized statements. However, synchronization is fairly expensive computationally, and excessive use of it should be avoided. So in some cases, it can make sense for threads to refer to shared variables without synchronizing their access to those variables.

However, a subtle problem arises when the value of a shared variable is set is one thread and used in another. Because of the way that threads are implemented in Java, the second thread might not see the changed value of the variable immediately. That is, it is possible that a thread will continue to see the **old** value of the shared variable for some time after the value of the variable has been changed by another thread. This is because threads are allowed to ***cache*** shared data. That is, each thread can keep its own local copy of the shared data. When one thread changes the value of a shared variable, the local copies in the caches of other threads are not immediately changed, so the other threads continue to see the old value.

When a synchronized method or statement is entered, threads are forced to update their caches to the most current values of the variables in the cache. So, using shared variables in synchronized code is always safe.

It is still possible to use a shared variable **outside** of synchronized code, but in that case, the variable must be declared to be ***volatile***. The `volatile` keyword is a modifier that can be added to a variable declaration, as in

```
private volatile int count;
```

If a variable is declared to be `volatile`, no thread will keep a local copy of that variable in its cache. Instead, the thread will always use the official, main copy of the variable. This means that any change made to the variable will immediately be available to all threads. This makes it safe for threads to refer to `volatile` shared variables even outside of synchronized code. (Remember, though, that synchronization is still the only way to prevent race conditions.)

When the `volatile` modifier is applied to an object variable, only the variable itself is declared to be volatile, not the contents of the object that the variable points to. For this reason, `volatile` is generally only used for variables of simple types such as primitive types and enumerated types.

A typical example of using volatile variables is to send a signal from one thread to another that tells the second thread to terminate. The two threads would share a variable

```
volatile boolean terminate = false;
```

The run method of the second thread would check the value of `terminate` frequently and end when the value of `terminate` becomes true:

```
public void run() {
   while (true) {
      if (terminate)
         return;
      .
      .  // Do some work
      .
   }
}
```

This thread will run until some other thread sets the value of `terminate` to true. Something like this is really the only clean way for one thread to cause another thread to die.

(By the way, you might be wondering why threads should use local data caches in the first place, since it seems to complicate things unnecessarily. Caching is allowed because of the structure of multiprocessing computers. In many multiprocessing computers, each processor has some local memory that is directly connected to the processor. A thread's cache is stored in the local memory of the processor on which the thread is running. Access to this local memory is much faster than access to other memory, so it is more efficient for a thread to use a local copy of a shared variable rather than some "master copy" that is stored in non-local memory.)

8.6 Analysis of Algorithms

THIS CHAPTER HAS CONCENTRATED mostly on correctness of programs. In practice, another issue is also important: ***efficiency***. When analyzing a program in terms of efficiency, we want to look at questions such as, "How long does it take for the program to run?" and "Is there another approach that will get the answer more quickly?" Efficiency will always be less important than correctness; if you don't care whether a program works correctly, you can make it run very quickly indeed, but no one will think it's much of an achievement! On the other hand, a program that gives a correct answer after ten thousand years isn't very useful either, so efficiency is often an important issue.

The term "efficiency" can refer to efficient use of almost any resource, including time, computer memory, disk space, or network bandwidth. In this section, however, we will deal exclusively with time efficiency, and the major question that we want to ask about a program is, how long does it take to perform its task?

It really makes little sense to classify an individual program as being "efficient" or "inefficient." It makes more sense to compare two (correct) programs that perform the same task and ask which one of the two is "more efficient," that is, which one performs the task more quickly. However, even here there are difficulties. The running time of a program is not well-defined. The run time can be different depending on the number and speed of the processors in the computer on which it is run and, in the case of Java, on the design of the Java Virtual Machine which is used to interpret the program. It can depend on details of the compiler which is used to translate the program from high-level language to machine language. Furthermore, the run time of a program depends on the size of the problem which the program has to solve. It takes a sorting program longer to sort 10000 items than it takes it to sort 100 items. When the run times of two programs are compared, it often happens that Program A solves small problems faster than Program B, while Program B solves large problems faster than Program A, so that it is simply not the case that one program is faster than the other in all cases.

In spite of these difficulties, there is a field of computer science dedicated to analyzing the efficiency of programs. The field is known as ***Analysis of Algorithms***. The focus is on algorithms, rather than on programs as such, to avoid having to deal with multiple implementations of the same algorithm written in different languages, compiled with different compilers, and running on different computers. Analysis of Algorithms is a mathematical field that abstracts away from these down-and-dirty details. Still, even though it is a theoretical field, every working programmer should be aware of some of its techniques and results. This section is a very brief introduction to some of those techniques and results. Because this is not a mathematics book, the treatment will be rather informal.

One of the main techniques of analysis of algorithms is ***asymptotic analysis***. The term "asymptotic" here means basically "the tendency in the long run." An asymptotic analysis of

an algorithm's run time looks at the question of how the run time depends on the size of the problem. The analysis is asymptotic because it only considers what happens to the run time as the size of the problem increases without limit; it is not concerned with what happens for problems of small size or, in fact, for problems of any fixed finite size. Only what happens in the long run, as the problem size increases without limit, is important. Showing that Algorithm A is asymptotically faster than Algorithm B doesn't necessarily mean that Algorithm A will run faster than Algorithm B for problems of size 10 or size 1000 or even size 1000000—it only means that if you keep increasing the problem size, you will eventually come to a point where Algorithm A is faster than Algorithm B. An asymptotic analysis is only a first approximation, but in practice it often gives important and useful information.

* * *

Central to asymptotic analysis is ***Big-Oh notation***. Using this notation, we might say, for example, that an algorithm has a running time that is $\mathcal{O}(n^2)$ or $\mathcal{O}(n)$ or $\mathcal{O}(\log(n))$. These notations are read "Big-Oh of n squared," "Big-Oh of n," and "Big-Oh of log n" (where log is a logarithm function). More generally, we can refer to $\mathcal{O}(f(n))$ ("Big-Oh of f of n"), where f(n) is some function that assigns a positive real number to every positive integer n. The "n" in this notation refers to the size of the problem. Before you can even begin an asymptotic analysis, you need some way to measure problem size. Usually, this is not a big issue. For example, if the problem is to sort a list of items, then the problem size can be taken to be the number of items in the list. When the input to an algorithm is an integer, as in the case of algorithm that checks whether a given positive integer is prime, the usual measure of the size of a problem is the number of bits in the input integer rather than the integer itself. More generally, the number of bits in the input to a problem is often a good measure of the size of the problem.

To say that the running time of an algorithm is $\mathcal{O}(f(n))$ means that for large values of the problem size, n, the running time of the algorithm is no bigger than some constant times f(n). (More rigorously, there is a number C and a positive integer M such that whenever n is greater than M, the run time is less than or equal to C*f(n).) The constant takes into account details such as the speed of the computer on which the algorithm is run; if you use a slower computer, you might have to use a bigger constant in the formula, but changing the constant won't change the basic fact that the run time is $\mathcal{O}(f(n))$. The constant also makes it unnecessary to say whether we are measuring time in seconds, years, CPU cycles, or any other unit of measure; a change from one unit of measure to another is just multiplication by a constant. Note also that $\mathcal{O}(f(n))$ doesn't depend at all on what happens for small problem sizes, only on what happens in the long run as the problem size increases without limit.

To look at a simple example, consider the problem of adding up all the numbers in an array. The problem size, n, is the length of the array. Using `A` as the name of the array, the algorithm can be expressed in Java as:

```
total = 0;
for (int i = 0; i < n; i++)
   total = total + A[i];
```

This algorithm performs the same operation, `total = total + A[i]`, n times. The total time spent on this operation is a*n, where a is the time it takes to perform the operation once. Now, this is not the only thing that is done in the algorithm. The value of `i` is incremented and is compared to n each time through the loop. This adds an additional time of b*n to the run time, for some constant b. Furthermore, `i` and `total` both have to be initialized to zero; this adds some constant amount c to the running time. The exact running time would then be (a+b)*n+c, where the constants a, b, and c depend on factors such as how the code is compiled

and what computer it is run on. Using the fact that c is less than or equal to c*n for any positive integer n, we can say that the run time is less than or equal to (a+b+c)*n. That is, the run time is less than or equal to a constant times n. By definition, this means that the run time for this algorithm is $\mathcal{O}(n)$.

If this explanation is too mathematical for you, we can just note that for large values of n, the c in the formula (a+b)*n+c is insignificant compared to the other term, (a+b)*n. We say that c is a "lower order term." When doing asymptotic analysis, lower order terms can be discarded. A rough, but correct, asymptotic analysis of the algorithm would go something like this: Each iteration of the `for` loop takes a certain constant amount of time. There are n iterations of the loop, so the total run time is a constant times n, plus lower order terms (to account for the initialization). Disregarding lower order terms, we see that the run time is $\mathcal{O}(n)$.

* * *

Note that to say that an algorithm has run time $\mathcal{O}(f(n))$ is to say that its run time is no bigger than some constant times n (for large values of n). $\mathcal{O}(f(n))$ puts an **upper limit** on the run time. However, the run time could be smaller, even much smaller. For example, if the run time is $\mathcal{O}(n)$, it would also be correct to say that the run time is $\mathcal{O}(n^2)$ or even $\mathcal{O}(n^{10})$. If the run time is less than a constant times n, then it is certainly less than the same constant times n^2 or n^{10}.

Of course, sometimes it's useful to have a **lower limit** on the run time. That is, we want to be able to say that the run time is greater than or equal to some constant times f(n) (for large values of n). The notation for this is $\Omega(f(n))$, read "Omega of f of n." "Omega" is the name of a letter in the Greek alphabet, and Ω is the upper case version of that letter. (To be technical, saying that the run time of an algorithm is $\Omega(f(n))$ means that there is a positive number C and a positive integer M such that whenever n is greater than M, the run time is greater than or equal to C*f(n).) $\mathcal{O}(f(n))$ tells you something about the maximum amount of time that you might have to wait for an algorithm to finish; $\Omega(f(n))$ tells you something about the minimum time.

The algorithm for adding up the numbers in an array has a run time that is $\Omega(n)$ as well as $\mathcal{O}(n)$. When an algorithm has a run time that is both $\Omega(f(n))$ and $\mathcal{O}(f(n))$, its run time is said to be $\Theta(f(n))$, read "Theta of f of n." (Theta is another letter from the Greek alphabet.) To say that the run time of an algorithm is $\Theta(f(n))$ means that for large values of n, the run time is between a*f(n) and b*f(n), where a and b are constants (with b greater than a, and both greater than 0).

Let's look at another example. Consider the algorithm that can be expressed in Java in the following method:

```
/**
 * Sorts the n array elements A[0], A[1], ..., A[n-1] into increasing order.
 */
public static simpleBubbleSort( int[] A, int n ) {
   for (int i = 0; i < n; i++) {
         // Do n passes through the array...
      for (int j = 0; j < n-1; j++) {
         if ( A[j] > A[j+1] ) {
               // A[j] and A[j+1] are out of order, so swap them
            int temp = A[j];
            A[j] = A[j+1];
            A[j+1] = temp;
```

```
            }
        }
    }
}
```

Here, the parameter n represents the problem size. The outer `for` loop in the method is executed n times. Each time the outer `for` loop is executed, the inner `for` loop is exectued n-1 times, so the `if` statement is executed n*(n-1) times. This is n^2-n, but since lower order terms are not significant in an asymptotic analysis, it's good enough to say that the `if` statement is executed about n^2 times. In particular, the test `A[j] > A[j+1]` is executed about n^2 times, and this fact by itself is enough to say that the run time of the algorithm is $\Omega(n^2)$, that is, the run time is at least some constant times n^2. Furthermore, if we look at other operations—the assignment statements, incrementing `i` and `j`, etc.—none of them are executed more than n^2 times, so the run time is also $\mathcal{O}(n^2)$, that is, the run time is no more than some constant times n^2. Since it is both $\Omega(n^2)$ and $\mathcal{O}(n^2)$, the run time of the simpleBubbleSort algorithm is $\Theta(n^2)$.

You should be aware that some people use the notation $\mathcal{O}(f(n))$ as if it meant $\Theta(f(n))$. That is, when they say that the run time of an algorithm is $\mathcal{O}(f(n))$, they mean to say that the run time is about **equal** to a constant times f(n). For that, they should use $\Theta(f(n))$. Properly speaking, $\mathcal{O}(f(n))$ means that the run time is less than a constant times f(n), possibly much less.

* * *

So far, my analysis has ignored an important detail. We have looked at how run time depends on the problem size, but in fact the run time usually depends not just on the size of the problem but on the specific data that has to be processed. For example, the run time of a sorting algorithm can depend on the initial order of the items that are to be sorted, and not just on the number of items.

To account for this dependency, we can consider either the ***worst case*** run time analysis or the ***average case*** run time analysis of an algorithm. For a worst case run time analysis, we consider all possible problems of size n and look at the **longest** possible run time for all such problems. For an average case analysis, we consider all possible problems of size n and look at the **average** of the run times for all such problems. Usually, the average case analysis assumes that all problems of size n are equally likely to be encountered, although this is not always realistic—or even possible in the case where there is an infinite number of different problems of a given size.

In many cases, the average and the worst case run times are the same to within a constant multiple. This means that as far as asymptotic analysis is concerned, they are the same. That is, if the average case run time is $\mathcal{O}(f(n))$ or $\Theta(f(n))$, then so is the worst case. However, later in the book, we will encounter a few cases where the average and worst case asymptotic analyses differ.

* * *

So, what do you really have to know about analysis of algorithms to read the rest of this book? We will not do any rigorous mathematical analysis, but you should be able to follow informal discussion of simple cases such as the examples that we have looked at in this section. Most important, though, you should have a feeling for exactly what it means to say that the running time of an algorithm is $\mathcal{O}(f(n))$ or $\Theta(f(n))$ for some common functions f(n). The main point is that these notations do not tell you anything about the actual numerical value of the running time of the algorithm for any particular case. They do not tell you anything at all

about the running time for small values of n. What they do tell you is something about the ***rate of growth*** of the running time as the size of the problem increases.

Suppose you compare two algorithm that solve the same problem. The run time of one algorithm is $\Theta(n^2)$, while the run time of the second algorithm is $\Theta(n^3)$. What does this tell you? If you want to know which algorithm will be faster for some particular problem of size, say, 100, nothing is certain. As far as you can tell just from the asymptotic analysis, either algorithm could be faster for that particular case—or in **any** particular case. But what you can say for sure is that if you look at larger and larger problems, you will come to a point where the $\Theta(n^2)$ algorithm is faster than the $\Theta(n^3)$ algorithm. Furthermore, as you continue to increase the problem size, the relative advantage of the $\Theta(n^2)$ algorithm will continue to grow. There will be values of n for which the $\Theta(n^2)$ algorithm is a thousand times faster, a million times faster, a billion times faster, and so on. This is because for any positive constants a and b, the function $a*n^3$ **grows faster** than the function $b*n^2$ as n gets larger. (Mathematically, the limit of the ratio of $a*n^3$ to $b*n^2$ is infinite as n approaches infinity.)

This means that for "large" problems, a $\Theta(n^2)$ algorithm will definitely be faster than a $\Theta(n^3)$ algorithm. You just don't know—based on the asymptotic analysis alone—exactly how large "large" has to be. In practice, in fact, it is likely that the $\Theta(n^2)$ algorithm will be faster even for fairly small values of n, and absent other information you would generally prefer a $\Theta(n^2)$ algorithm to a $\Theta(n^3)$ algorithm.

So, to understand and apply asymptotic analysis, it is essential to have some idea of the rates of growth of some common functions. For the power functions n, n^2, n^3, n^4, . . . , the larger the exponent, the greater the rate of growth of the function. Exponential functions such as 2^n and 10^n, where the n is in the exponent, have a growth rate that is faster than that of any power function. In fact, exponential functions grow so quickly that an algorithm whose run time grows exponentially is almost certainly impractical even for relatively modest values of n, because the running time is just too long. Another function that often turns up in asymptotic analysis is the logarithm function, log(n). There are actually many different logarithm functions, but the one that is usually used in computer science is the so-called logarithm to the base two, which is defined by the fact that $\log(2^x) = x$ for any number x. (Usually, this function is written $\log_2(n)$, but I will leave out the subscript 2, since I will only use the base-two logarithm in this book.) The logarithm function grows very slowly. The growth rate of log(n) is much smaller than the growth rate of n. The growth rate of n*log(n) is a little larger than the growth rate of n, but much smaller than the growth rate of n^2. The following table should help you understand the differences among the rates of grows of various functions:

n	log(n)	n*log(n)	n^2	n / log(n)
16	4	64	256	4.0
64	6	384	4096	10.7
256	8	2048	65536	32.0
1024	10	10240	1048576	102.4
1000000	20	19931568	1000000000000	50173.1
1000000000	30	29897352854	1000000000000000000	33447777.3

The reason that log(n) shows up so often is because of its association with multiplying and dividing by two: Suppose you start with the number n and divide it by 2, then divide by 2 again, and so on, until you get a number that is less than or equal to 1. Then the number of divisions is equal (to the nearest integer) to log(n).

As an example, consider the binary search algorithm from Subsection 7.4.1. This algorithm searches for an item in a sorted array. The problem size, n, can be taken to be the length of the array. Each step in the binary search algorithm divides the number of items still under consideration by 2, and the algorithm stops when the number of items under consideration is less than or equal to 1 (or sooner). It follows that the number of steps for an array of length n is at most log(n). This means that the worst-case run time for binary search is $\Theta(\log(n))$. (The average case run time is also $\Theta(\log(n))$.) By comparison, the linear search algorithm, which was also presented in Subsection 7.4.1 has a run time that is $\Theta(n)$. The Θ notation gives us a quantitative way to express and to understand the fact that binary search is "much faster" than linear search. 1

In binary search, each step of the algorithm divides the problem size by 2. It often happens that some operation in an algorithm (not necessarily a single step) divides the problem size by 2. Whenever that happens, the logarithm function is likely to show up in an asymptotic analysis of the run time of the algorithm. 2

Analysis of Algorithms is a large, fascinating field. We will only use a few of the most basic ideas from this field, but even those can be very helpful for understanding the differences among algorithms. 3

Exercises for Chapter 8

1. Write a program that uses the following subroutine, from Subsection 8.3.3, to solve equations specified by the user.

```
/**
 * Returns the larger of the two roots of the quadratic equation
 * A*x*x + B*x + C = 0, provided it has any roots.  If A == 0 or
 * if the discriminant, B*B - 4*A*C, is negative, then an exception
 * of type IllegalArgumentException is thrown.
 */
static public double root( double A, double B, double C )
                                 throws IllegalArgumentException {
    if (A == 0) {
      throw new IllegalArgumentException("A can't be zero.");
    }
    else {
       double disc = B*B - 4*A*C;
       if (disc < 0)
          throw new IllegalArgumentException("Discriminant < zero.");
       return  (-B + Math.sqrt(disc)) / (2*A);
    }
}
```

Your program should allow the user to specify values for `A`, `B`, and `C`. It should call the subroutine to compute a solution of the equation. If no error occurs, it should print the root. However, if an error occurs, your program should catch that error and print an error message. After processing one equation, the program should ask whether the user wants to enter another equation. The program should continue until the user answers no.

2. As discussed in Section 8.1, values of type **int** are limited to 32 bits. Integers that are too large to be represented in 32 bits cannot be stored in an **int** variable. Java has a standard class, `java.math.BigInteger`, that addresses this problem. An object of type *BigInteger* is an integer that can be arbitrarily large. (The maximum size is limited only by the amount of memory on your computer.) Since *BigIntegers* are objects, they must be manipulated using instance methods from the *BigInteger* class. For example, you can't add two *BigIntegers* with the `+` operator. Instead, if `N` and `M` are variables that refer to *BigIntegers*, you can compute the sum of `N` and `M` with the function call `N.add(M)`. The value returned by this function is a new *BigInteger* object that is equal to the sum of `N` and `M`.

The `BigInteger` class has a constructor `new BigInteger(str)`, where `str` is a string. The string must represent an integer, such as "3" or "39849823783783283733". If the string does not represent a legal integer, then the constructor throws a *NumberFormatException*.

There are many instance methods in the *BigInteger* class. Here are a few that you will find useful for this exercise. Assume that `N` and `M` are variables of type `BigInteger`.

- `N.add(M)` — a function that returns a *BigInteger* representing the sum of `N` and `M`.
- `N.multiply(M)` — a function that returns a *BigInteger* representing the result of multiplying `N` times `M`.

- `N.divide(M)` — a function that returns a *BigInteger* representing the result of dividing `N` by `M`, discarding the remainder.
- `N.signum()` — a function that returns an ordinary **int**. The returned value represents the sign of the integer `N`. The returned value is 1 if `N` is greater than zero. It is -1 if `N` is less than zero. And it is 0 if `N` is zero.
- `N.equals(M)` — a function that returns a **boolean** value that is `true` if `N` and `M` have the same integer value.
- `N.toString()` — a function that returns a *String* representing the value of `N`.
- `N.testBit(k)` — a function that returns a **boolean** value. The parameter `k` is an integer. The return value is `true` if the `k`-th bit in `N` is 1, and it is `false` if the `k`-th bit is 0. Bits are numbered from right to left, starting with 0. Testing "`if (N.testBit(0))`" is an easy way to check whether `N` is even or odd. `N.testBit(0)` is `true` if and only if `N` is an odd number.

For this exercise, you should write a program that prints `3N+1` sequences with starting values specified by the user. In this version of the program, you should use *BigIntegers* to represent the terms in the sequence. You can read the user's input into a *String* with the `TextIO.getln()` function. Use the input value to create the *BigInteger* object that represents the starting point of the `3N+1` sequence. Don't forget to catch and handle the *NumberFormatException* that will occur if the user's input is not a legal integer! You should also check that the input number is greater than zero.

If the user's input is legal, print out the `3N+1` sequence. Count the number of terms in the sequence, and print the count at the end of the sequence. Exit the program when the user inputs an empty line.

3. A Roman numeral represents an integer using letters. Examples are XVII to represent 17, MCMLIII for 1953, and MMMCCCIII for 3303. By contrast, ordinary numbers such as 17 or 1953 are called Arabic numerals. The following table shows the Arabic equivalent of all the single-letter Roman numerals:

M	1000	X	10
D	500	V	5
C	100	I	1
L	50		

When letters are strung together, the values of the letters are just added up, with the following exception. When a letter of smaller value is followed by a letter of larger value, the smaller value is subtracted from the larger value. For example, IV represents 5 - 1, or 4. And MCMXCV is interpreted as M + CM + XC + V, or 1000 + (1000 - 100) + (100 - 10) + 5, which is 1995. In standard Roman numerals, no more than thee consecutive copies of the same letter are used. Following these rules, every number between 1 and 3999 can be represented as a Roman numeral made up of the following one- and two-letter combinations:

M	1000	X	10
CM	900	IX	9
D	500	V	5
CD	400	IV	4
C	100	I	1
XC	90		

```
L       50
XL      40
```

Write a class to represent Roman numerals. The class should have two constructors. One constructs a Roman numeral from a string such as "XVII" or "MCMXCV". It should throw a *NumberFormatException* if the string is not a legal Roman numeral. The other constructor constructs a Roman numeral from an **int**. It should throw a *NumberFormatException* if the **int** is outside the range 1 to 3999.

In addition, the class should have two instance methods. The method `toString()` returns the string that represents the Roman numeral. The method `toInt()` returns the value of the Roman numeral as an **int**.

At some point in your class, you will have to convert an **int** into the string that represents the corresponding Roman numeral. One way to approach this is to gradually "move" value from the Arabic numeral to the Roman numeral. Here is the beginning of a routine that will do this, where `number` is the **int** that is to be converted:

```
String roman = "";
int N = number;
while (N >= 1000) {
      // Move 1000 from N to roman.
   roman += "M";
   N -= 1000;
}
while (N >= 900) {
      // Move 900 from N to roman.
   roman += "CM";
   N -= 900;
}
.
.  // Continue with other values from the above table.
.
```

(You can save yourself a lot of typing in this routine if you use arrays in a clever way to represent the data in the above table.)

Once you've written your class, use it in a main program that will read both Arabic numerals and Roman numerals entered by the user. If the user enters an Arabic numeral, print the corresponding Roman numeral. If the user enters a Roman numeral, print the corresponding Arabic numeral. (You can tell the difference by using `TextIO.peek()` to peek at the first character in the user's input. If that character is a digit, then the user's input is an Arabic numeral. Otherwise, it's a Roman numeral.) The program should end when the user inputs an empty line.

4. The source code file file *Expr.java* defines a class, *Expr*, that can be used to represent mathematical expressions involving the variable `x`. The expression can use the operators `+`, `-`, `*`, `/`, and `^` (where `^` represents the operation of raising a number to a power). It can use mathematical functions such as `sin`, `cos`, `abs`, and `ln`. See the source code file for full details. The *Expr* class uses some advanced techniques which have not yet been covered in this textbook. However, the interface is easy to understand. It contains only a constructor and two public methods.

The constructor `new Expr(def)` creates an *Expr* object defined by a given expression. The parameter, `def`, is a string that contains the definition. For example,

`new Expr("x^2")` or `new Expr("sin(x)+3*x")`. If the parameter in the constructor call does not represent a legal expression, then the constructor throws an *IllegalArgumentException*. The message in the exception describes the error.

If `func` is a variable of type `Expr` and `num` is of type **double**, then `func.value(num)` is a function that returns the value of the expression when the number `num` is substituted for the variable `x` in the expression. For example, if `Expr` represents the expression `3*x+1`, then `func.value(5)` is `3*5+1`, or 16. If the expression is undefined for the specified value of `x`, then the special value `Double.NaN` is returned.

Finally, `func.toString()` returns the definition of the expression. This is just the string that was used in the constructor that created the expression object.

For this exercise, you should write a program that lets the user enter an expression. If the expression contains an error, print an error message. Otherwise, let the user enter some numerical values for the variable `x`. Print the value of the expression for each number that the user enters. However, if the expression is undefined for the specified value of `x`, print a message to that effect. You can use the **boolean**-valued function `Double.isNaN(val)` to check whether a number, `val`, is `Double.NaN`.

The user should be able to enter as many values of `x` as desired. After that, the user should be able to enter a new expression. In the on-line version of this exercise, there is an applet that simulates my solution, so that you can see how it works.

5. This exercise uses the class *Expr*, which was described in Exercise 8.4 and which is defined in the source code file *Expr.java*. For this exercise, you should write a GUI program that can graph a function, `f(x)`, whose definition is entered by the user. The program should have a text-input box where the user can enter an expression involving the variable `x`, such as `x^2` or `sin(x-3)/x`. This expression is the definition of the function. When the user presses return in the text input box, the program should use the contents of the text input box to construct an object of type *Expr*. If an error is found in the definition, then the program should display an error message. Otherwise, it should display a graph of the function. (Note: A `JTextField` generates an `ActionEvent` when the user presses return.)

The program will need a *JPanel* for displaying the graph. To keep things simple, this panel should represent a fixed region in the xy-plane, defined by `-5 <= x <= 5` and `-5 <= y <= 5`. To draw the graph, compute a large number of points and connect them with line segments. (This method does not handle discontinuous functions properly; doing so is very hard, so you shouldn't try to do it for this exercise.) My program divides the interval `-5 <= x <= 5` into 300 subintervals and uses the 301 endpoints of these subintervals for drawing the graph. Note that the function might be undefined at one of these `x`-values. In that case, you have to skip that point.

A point on the graph has the form `(x,y)` where `y` is obtained by evaluating the user's expression at the given value of `x`. You will have to convert these real numbers to the integer coordinates of the corresponding pixel on the canvas. The formulas for the conversion are:

```
a  =  (int)( (x + 5)/10 * width );
b  =  (int)( (5 - y)/10 * height );
```

where `a` and `b` are the horizontal and vertical coordinates of the pixel, and `width` and `height` are the width and height of the panel.

You can find an applet version of my solution in the on-line version of this exercise.

6. Exercise 3.2 asked you to find the integer in the range 1 to 10000 that has the largest number of divisors. Now write a program that uses multiple threads to solve the same problem. By using threads, your program will take less time to do the computation when it is run on a multiprocessor computer. At the end of the program, output the elapsed time, the integer that has the largest number of divisors, and the number of divisors that it has. The program can be modeled on the sample prime-counting program *ThreadTest2.java* from Subsection 8.5.3.

Quiz on Chapter 8

1

1. What does it mean to say that a program is *robust*? 2

2. Why do programming languages require that variables be declared before they are used? What does this have to do with correctness and robustness? 3

3. What is a *precondition*? Give an example. 4

4. Explain how preconditions can be used as an aid in writing correct programs. 5

5. Java has a predefined class called *Throwable*. What does this class represent? Why does it exist? 6

6. Write a method that prints out a `3N+1` sequence starting from a given integer, `N`. The starting value should be a parameter to the method. If the parameter is less than or equal to zero, throw an *IllegalArgumentException*. If the number in the sequence becomes too large to be represented as a value of type **int**, throw an *ArithmeticException*. 7

7. Rewrite the method from the previous question, using `assert` statements instead of exceptions to check for errors. What the difference between the two versions of the method when the program is run? 8

8. Some classes of exceptions require *mandatory exception handling*. Explain what this means. 9

9. Consider a subroutine `processData()` that has the header 10

```
static void processData() throws IOException
```

11

Write a `try..catch` statement that calls this subroutine and prints an error message if an *IOException* occurs. 12

10. Why should a subroutine throw an exception when it encounters an error? Why not just terminate the program? 13

11. Suppose that a program uses a single thread that takes 4 seconds to run. Now suppose that the program creates two threads and divides the same work between the two threads. What can be said about the expected execution time of the program that uses two threads? 14

12. Consider the *ThreadSafeCounter* example from Subsection 8.5.3: 15

```
public class ThreadSafeCounter {

   private int count = 0;  // The value of the counter.

   synchronized public void increment() {
      count = count + 1;
   }

   synchronized public int getValue() {
      return count;
   }

}
```

16

The `increment()` method is synchronized so that the caller of the method can complete the three steps of the operation "Get value of count," "Add 1 to value," "Store new value in count" without being interrupted by another thread. But `getValue()` consists of a single, simple step. Why is `getValue()` synchronized? (This is a deep and tricky question.) 1

Chapter 9

Linked Data Structures and Recursion [1]

In this chapter, we look at two advanced programming techniques, recursion and linked data structures, and some of their applications. Both of these techniques are related to the seemingly paradoxical idea of defining something in terms of itself. This turns out to be a remarkably powerful idea. [2]

A subroutine is said to be recursive if it calls itself, either directly or indirectly. That is, the subroutine is used in its own definition. Recursion can often be used to solve complex problems by reducing them to simpler problems of the same type. [3]

A reference to one object can be stored in an instance variable of another object. The objects are then said to be "linked." Complex data structures can be built by linking objects together. An especially interesting case occurs when an object contains a link to another object that belongs to the same class. In that case, the class is used in its own definition. Several important types of data structures are built using classes of this kind. [4]

9.1 Recursion [5]

At one time or another, you've probably been told that you can't define something in terms of itself. Nevertheless, if it's done right, defining something at least partially in terms of itself can be a very powerful technique. A ***recursive*** definition is one that uses the concept or thing that is being defined as part of the definition. For example: An "ancestor" is either a parent or an ancestor of a parent. A "sentence" can be, among other things, two sentences joined by a conjunction such as "and." A "directory" is a part of a disk drive that can hold files and directories. In mathematics, a "set" is a collection of elements, which can themselves be sets. A "statement" in Java can be a `while` statement, which is made up of the word "while", a boolean-valued condition, and a statement. [6]

Recursive definitions can describe very complex situations with just a few words. A definition of the term "ancestor" without using recursion might go something like "a parent, or a grandparent, or a great-grandparent, or a great-great-grandparent, and so on." But saying "and so on" is not very rigorous. (I've often thought that recursion is really just a rigorous way of saying "and so on.") You run into the same problem if you try to define a "directory" as "a file that is a list of files, where some of the files can be lists of files, where some of **those** files can be lists of files, and so on." Trying to describe what a Java statement can look like, without using recursion in the definition, would be difficult and probably pretty comical. [7]

Recursion can be used as a programming technique. A ***recursive subroutine*** is one that calls itself, either directly or indirectly. To say that a subroutine calls itself directly means that its definition contains a subroutine call statement that calls the subroutine that is being defined. To say that a subroutine calls itself indirectly means that it calls a second subroutine which in turn calls the first subroutine (either directly or indirectly). A recursive subroutine can define a complex task in just a few lines of code. In the rest of this section, we'll look at a variety of examples, and we'll see other examples in the rest of the book.

9.1.1 Recursive Binary Search

Let's start with an example that you've seen before: the binary search algorithm from Subsection 7.4.1. Binary search is used to find a specified value in a sorted list of items (or, if it does not occur in the list, to determine that fact). The idea is to test the element in the middle of the list. If that element is equal to the specified value, you are done. If the specified value is less than the middle element of the list, then you should search for the value in the first half of the list. Otherwise, you should search for the value in the second half of the list. The method used to search for the value in the first or second half of the list is binary search. That is, you look at the middle element in the half of the list that is still under consideration, and either you've found the value you are looking for, or you have to apply binary search to one half of the remaining elements. And so on! This is a recursive description, and we can write a recursive subroutine to implement it.

Before we can do that, though, there are two considerations that we need to take into account. Each of these illustrates an important general fact about recursive subroutines. First of all, the binary search algorithm begins by looking at the "middle element of the list." But what if the list is empty? If there are no elements in the list, then it is impossible to look at the middle element. In the terminology of Subsection 8.2.1, having a non-empty list is a "precondition" for looking at the middle element, and this is a clue that we have to modify the algorithm to take this precondition into account. What should we do if we find ourselves searching for a specified value in an empty list? The answer is easy: If the list is empty, we can be sure that the value does not occur in the list, so we can give the answer without any further work. An empty list is a ***base case*** for the binary search algorithm. A base case for a recursive algorithm is a case that is handled directly, rather than by applying the algorithm recursively. The binary search algorithm actually has another type of base case: If we find the element we are looking for in the middle of the list, we are done. There is no need for further recursion.

The second consideration has to do with the parameters to the subroutine. The problem is phrased in terms of searching for a value in a list. In the original, non-recursive binary search subroutine, the list was given as an array. However, in the recursive approach, we have to able to apply the subroutine recursively to just a **part** of the original list. Where the original subroutine was designed to search an entire array, the recursive subroutine must be able to search part of an array. The parameters to the subroutine must tell it what part of the array to search. This illustrates a general fact that in order to solve a problem recursively, it is often necessary to generalize the problem slightly.

Here is a recursive binary search algorithm that searches for a given value in part of an array of integers:

```
/**
 * Search in the array A in positions numbered loIndex to hiIndex,
 * inclusive, for the specified value.  If the value is found, return
 * the index in the array where it occurs. If the value is not found,
```

```
 * return -1.  Precondition:  The array must be sorted into increasing
 * order.
 */
static int binarySearch(int[] A, int loIndex, int hiIndex, int value) {

   if (loIndex > hiIndex) {
         // The starting position comes after the final index,
         // so there are actually no elements in the specified
         // range.  The value does not occur in this empty list!
      return -1;
   }

   else {
         // Look at the middle position in the list.  If the
         // value occurs at that position, return that position.
         // Otherwise, search recursively in either the first
         // half or the second half of the list.
      int middle = (loIndex + hiIndex) / 2;
      if (value == A[middle])
         return middle;
      else if (value < A[middle])
         return binarySearch(A, loIndex, middle - 1, value);
      else   // value must be > A[middle]
         return binarySearch(A, middle + 1, hiIndex, value);
   }

} // end binarySearch()
```

In this routine, the parameters `loIndex` and `hiIndex` specify the part of the array that is to be searched. To search an entire array, it is only necessary to call `binarySearch(A, 0, A.length - 1, value)`. In the two base cases—when there are no elements in the specified range of indices and when the value is found in the middle of the range—the subroutine can return an answer immediately, without using recursion. In the other cases, it uses a recursive call to compute the answer and returns that answer.

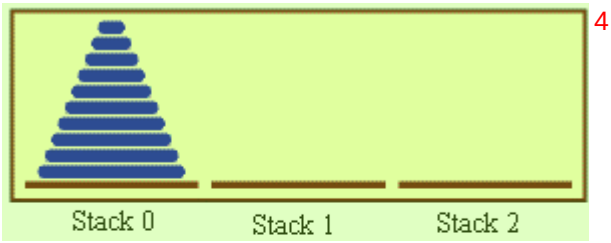
Most people find it difficult at first to convince themselves that recursion actually works. The key is to note two things that must be true for recursion to work properly: There must be one or more base cases, which can be handled without using recursion. And when recursion is applied during the solution of a problem, it must be applied to a problem that is in some sense smaller—that is, closer to the base cases—than the original problem. The idea is that if you can solve small problems and if you can reduce big problems to smaller problems, then you can solve problems of any size. Ultimately, of course, the big problems have to be reduced, possibly in many, many steps, to the very smallest problems (the base cases). Doing so might involve an immense amount of detailed bookkeeping. But the computer does that bookkeeping, not you! As a programmer, you lay out the big picture: the base cases and the reduction of big problems to smaller problems. The computer takes care of the details involved in reducing a big problem, in many steps, all the way down to base cases. Trying to think through this reduction in detail is likely to drive you crazy, and will probably make you think that recursion is hard. Whereas in fact, recursion is an elegant and powerful method that is often the simplest approach to solving a complex problem.

A common error in writing recursive subroutines is to violate one of the two rules: There must be one or more base cases, and when the subroutine is applied recursively, it must be applied to a problem that is smaller than the original problem. If these rules are violated, the

result can be an ***infinite recursion***, where the subroutine keeps calling itself over and over, without ever reaching a base case. Infinite recursion is similar to an infinite loop. However, since each recursive call to the subroutine uses up some of the computer's memory, a program that is stuck in an infinite recursion will run out of memory and crash before long. (In Java, the program will crash with an exception of type `StackOverflowError`.)

9.1.2 Towers of Hanoi

Binary search can be implemented with a `while` loop, instead of with recursion, as was done in Subsection 7.4.1. Next, we turn to a problem that is easy to solve with recursion but difficult to solve without it. This is a standard example known as "The Towers of Hanoi." The problem involves a stack of various-sized disks, piled up on a base in order of decreasing size. The object is to move the stack from one base to another, subject to two rules: Only one disk can be moved at a time, and no disk can ever be placed on top of a smaller disk. There is a third base that can be used as a "spare." The starting situation for a stack of ten disks is shown in the top half of the following picture. The situation after a number of moves have been made is shown in the bottom half of the picture. These pictures are from the applet at the end of Section 9.5, which displays an animation of the step-by-step solution of the problem.



The stacks after a number of moves.

The problem is to move ten disks from Stack 0 to Stack 1, subject to certain rules. Stack 2 can be used as a spare location. Can we reduce this to smaller problems of the same type, possibly generalizing the problem a bit to make this possible? It seems natural to consider the size of the problem to be the number of disks to be moved. If there are `N` disks in Stack 0, we know that we will eventually have to move the bottom disk from Stack 0 to Stack 1. But before we can do that, according to the rules, the first `N-1` disks must be on Stack 2. Once we've moved the `N`-th disk to Stack 1, we must move the other `N-1` disks from Stack 2 to Stack 1 to complete the solution. But moving `N-1` disks is the same type of problem as moving `N` disks, except that it's a smaller version of the problem. This is exactly what we need to do recursion! The problem has to be generalized a bit, because the smaller problems involve moving disks from Stack 0 to Stack 2 or from Stack 2 to Stack 1, instead of from Stack 0 to Stack 1. In the recursive subroutine that solves the problem, the stacks that serve as the source and destination

of the disks have to be specified. It's also convenient to specify the stack that is to be used as a spare, even though we could figure that out from the other two parameters. The base case is when there is only one disk to be moved. The solution in this case is trivial: Just move the disk in one step. Here is a version of the subroutine that will print out step-by-step instructions for solving the problem:

```
/**
 * Solve the problem of moving the number of disks specified
 * by the first parameter from the stack specified by the
 * second parameter to the stack specified by the third
 * parameter.  The stack specified by the fourth parameter
 * is available for use as a spare.  Stacks are specified by
 * number: 0, 1, or 2.
 */
static void TowersOfHanoi(int disks, int from, int to, int spare) {
   if (disks == 1) {
         // There is only one disk to be moved.  Just move it.
      System.out.println("Move a disk from stack number "
               + from + " to stack number " + to);
   }
   else {
         // Move all but one disk to the spare stack, then
         // move the bottom disk, then put all the other
         // disks on top of it.
      TowersOfHanoi(disks-1, from, spare, to);
      System.out.println("Move a disk from stack number "
               + from + " to stack number " + to);
      TowersOfHanoi(disks-1, spare, to, from);
   }
}
```

This subroutine just expresses the natural recursive solution. The recursion works because each recursive call involves a smaller number of disks, and the problem is trivial to solve in the base case, when there is only one disk. To solve the "top level" problem of moving `N` disks from Stack 0 to Stack 1, it should be called with the command `TowersOfHanoi(N,0,1,2)`. The subroutine is demonstrated by the sample program *TowersOfHanoi.java*.

Here, for example, is the output from the program when it is run with the number of disks set equal to 3:

```
Move a disk from stack number 0 to stack number 2
Move a disk from stack number 0 to stack number 1
Move a disk from stack number 2 to stack number 1
Move a disk from stack number 0 to stack number 2
Move a disk from stack number 1 to stack number 0
Move a disk from stack number 1 to stack number 2
Move a disk from stack number 0 to stack number 2
Move a disk from stack number 0 to stack number 1
Move a disk from stack number 2 to stack number 1
Move a disk from stack number 2 to stack number 0
Move a disk from stack number 1 to stack number 0
Move a disk from stack number 2 to stack number 1
Move a disk from stack number 0 to stack number 2
Move a disk from stack number 0 to stack number 1
Move a disk from stack number 2 to stack number 1
```

The output of this program shows you a mass of detail that you don't really want to think about! The difficulty of following the details contrasts sharply with the simplicity and elegance of the recursive solution. Of course, you really want to leave the details to the computer. It's much more interesting to watch the applet from Section 9.5, which shows the solution graphically. That applet uses the same recursive subroutine, except that the `System.out.println` statements are replaced by commands that show the image of the disk being moved from one stack to another.

There is, by the way, a story that explains the name of this problem. According to this story, on the first day of creation, a group of monks in an isolated tower near Hanoi were given a stack of 64 disks and were assigned the task of moving one disk every day, according to the rules of the Towers of Hanoi problem. On the day that they complete their task of moving all the disks from one stack to another, the universe will come to an end. But don't worry. The number of steps required to solve the problem for `N` disks is 2^N - 1, and 2^{64} - 1 days is over 50,000,000,000,000 years. We have a long way to go.

(In the terminology of Section 8.6, the Towers of Hanoi algorithm has a run time that is $\Theta(2^n)$, where n is the number of disks that have to be moved. Since the exponential function 2^n grows so quickly, the Towers of Hanoi problem can be solved in practice only for a small number of disks.)

* * *

By the way, in addtion to the graphical Towers of Hanoi applet at the end of this chapter, there are two other end-of-chapter applets in the on-line version of this text that use recursion. One is a maze-solving applet from the end of Section 11.5, and the other is a pentominos applet from the end of Section 10.5.

The Maze applet first builds a random maze. It then tries to solve the maze by finding a path through the maze from the upper left corner to the lower right corner. This problem is actually very similar to a "blob-counting" problem that is considered later in this section. The recursive maze-solving routine starts from a given square, and it visits each neighboring square and calls itself recursively from there. The recursion ends if the routine finds itself at the lower right corner of the maze.

The Pentominos applet is an implementation of a classic puzzle. A pentomino is a connected figure made up of five equal-sized squares. There are exactly twelve figures that can be made in this way, not counting all the possible rotations and reflections of the basic figures. The problem is to place the twelve pentominos on an 8-by-8 board in which four of the squares have already been marked as filled. The recursive solution looks at a board that has already been partially filled with pentominos. The subroutine looks at each remaining piece in turn. It tries to place that piece in the next available place on the board. If the piece fits, it calls itself recursively to try to fill in the rest of the solution. If that fails, then the subroutine goes on to the next piece. A generalized version of the pentominos applet with many more features can be found at http://math.hws.edu/xJava/PentominosSolver/.

The Maze applet and the Pentominos applet are fun to watch, and they give nice visual representations of recursion.

9.1.3 A Recursive Sorting Algorithm

Turning next to an application that is perhaps more practical, we'll look at a recursive algorithm for sorting an array. The selection sort and insertion sort algorithms, which were covered in Section 7.4, are fairly simple, but they are rather slow when applied to large arrays. Faster

sorting algorithms are available. One of these is ***Quicksort***, a recursive algorithm which turns out to be the fastest sorting algorithm in most situations. 2

The Quicksort algorithm is based on a simple but clever idea: Given a list of items, select any item from the list. This item is called the ***pivot***. (In practice, I'll just use the first item in the list.) Move all the items that are smaller than the pivot to the beginning of the list, and move all the items that are larger than the pivot to the end of the list. Now, put the pivot between the two groups of items. This puts the pivot in the position that it will occupy in the final, completely sorted array. It will not have to be moved again. We'll refer to this procedure as QuicksortStep. 3

To apply QuicksortStep to a list of numbers, select one of the numbers, 23 in this case. Arrange the numbers so that numbers less than 23 lie to its left and numbers greater than 23 lie to its right. 4

23 10 7 45 16 86 56 2 31 18

18 12 7 2 16 23 86 56 31 45

To finish sorting the list, sort the numbers to the left of 23, and sort the numbers to the right of 23. The nuber 23 itself is already in its final position and doesn't have to be moved again

QuicksortStep is not recursive. It is used as a subroutine by Quicksort. The speed of Quicksort depends on having a fast implementation of QuicksortStep. Since it's not the main point of this discussion, I present one without much comment. 5

```
/**
 * Apply QuicksortStep to the list of items in locations lo through hi
 * in the array A.  The value returned by this routine is the final
 * position of the pivot item in the array.
 */
static int quicksortStep(int[] A, int lo, int hi) {

   int pivot = A[lo];  // Get the pivot value.

   // The numbers hi and lo mark the endpoints of a range
   // of numbers that have not yet been tested.  Decrease hi
   // and increase lo until they become equal, moving numbers
   // bigger than pivot so that they lie above hi and moving
   // numbers less than the pivot so that they lie below lo.
   // When we begin, A[lo] is an available space, since it used
   // to hold the pivot.

   while (hi > lo) {

      while (hi > lo && A[hi] > pivot) {
            // Move hi down past numbers greater than pivot.
            // These numbers do not have to be moved.
         hi--;
      }

      if (hi == lo)
         break;
```

1

```
      // The number A[hi] is less than pivot.  Move it into
      // the available space at A[lo], leaving an available
      // space at A[hi].

      A[lo] = A[hi];
      lo++;

      while (hi > lo && A[lo] < pivot) {
            // Move lo up past numbers less than pivot.
            // These numbers do not have to be moved.
         lo++;
      }

      if (hi == lo)
         break;

      // The number A[lo] is greater than pivot.  Move it into
      // the available space at A[hi], leaving an available
      // space at A[lo].

      A[hi] = A[lo];
      hi--;

   } // end while

   // At this point, lo has become equal to hi, and there is
   // an available space at that position.  This position lies
   // between numbers less than pivot and numbers greater than
   // pivot.  Put pivot in this space and return its location.

   A[lo] = pivot;
   return lo;

}  // end QuicksortStep
```

With this subroutine in hand, Quicksort is easy. The Quicksort algorithm for sorting a list consists of applying QuicksortStep to the list, then applying Quicksort recursively to the items that lie to the left of the new position of the pivot and to the items that lie to the right of that position. Of course, we need base cases. If the list has only one item, or no items, then the list is already as sorted as it can ever be, so Quicksort doesn't have to do anything in these cases.

```
/**
 * Apply quicksort to put the array elements between
 * position lo and position hi into increasing order.
 */
static void quicksort(int[] A, int lo, int hi) {
   if (hi <= lo) {
         // The list has length one or zero.  Nothing needs
         // to be done, so just return from the subroutine.
      return;
   }
   else {
         // Apply quicksortStep and get the new pivot position.
         // Then apply quicksort to sort the items that
         // precede the pivot and the items that follow it.
      int pivotPosition = quicksortStep(A, lo, hi);
      quicksort(A, lo, pivotPosition - 1);
      quicksort(A, pivotPosition + 1, hi);
```

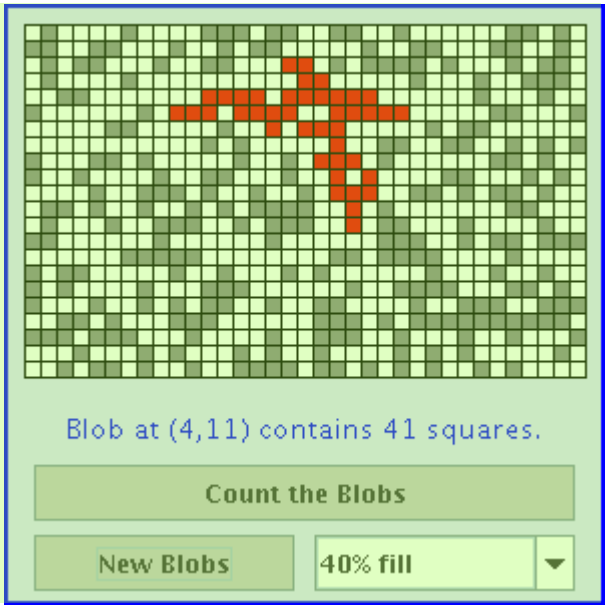
```
        }
    }
```

As usual, we had to generalize the problem. The original problem was to sort an array, but the recursive algorithm is set up to sort a specified part of an array. To sort an entire array, `A`, using the `quickSort()` subroutine, you would call `quicksort(A, 0, A.length - 1)`.

Quicksort is an interesting example from the point of view of the analysis of algorithms (Section 8.6), because its average case run time differs greatly from its worst case run time. Here is a very informal analysis, starting with the average case: Note that an application of quicksortStep divides a problem into two sub-problems. On the average, the subproblems will be of approximately the same size. A problem of size n is divided into two problems that are roughly of size n/2; these are then divided into four problems that are roughly of size n/4; and so on. Since the problem size is divided by 2 on each level, there will be approximately log(n) levels of subdivision. The amount of processing on each level is proportional to n. (On the top level, each element in the array is looked at and possibly moved. On the second level, where there are two subproblems, every element but one in the array is part of one of those two subproblems and must be looked at and possibly moved, so there is a total of about n steps in both subproblems combined. Similarly, on the third level, there are four subproblems and a total of about n steps in all four subproblems combined on that level. . . .) With a total of n steps on each level and approximately log(n) levels in the average case, the average case run time for Quicksort is Θ(n*log(n)). This analysis assumes that quicksortStep divides a problem into two approximately equal parts. However, in the worst case, each application of quicksortStep divides a problem of size n into a problem of size 0 and a problem of size n-1. This happens when the pivot element ends up at the beginning or end of the array. In this worst case, there are n levels of subproblems, and the worst-case run time is $\Theta(n^2)$. The worst case is very rare—it depends on the items in the array being arranged in a very special way, so the average performance of Quicksort can be very good even though it is not so good in certain rare cases. There are sorting algorithms that have both an average case and a worst case run time of Θ(n*log(n)). One example is MergeSort, which you can look up if you are interested.

9.1.4 Blob Counting

The program *Blobs.java* displays a grid of small white and gray squares. The gray squares are considered to be "filled" and the white squares are "empty." For the purposes of this example, we define a "blob" to consist of a filled square and all the filled squares that can be reached from it by moving up, down, left, and right through other filled squares. If the user clicks on any filled square in the program, the computer will count the squares in the blob that contains the clicked square, and it will change the color of those squares to red. The program has several controls. There is a "New Blobs" button; clicking this button will create a new random pattern in the grid. A pop-up menu specifies the approximate percentage of squares that will be filled in the new pattern. The more filled squares, the larger the blobs. And a button labeled "Count the Blobs" will tell you how many different blobs there are in the pattern. You can try an applet version of the program in the on-line version of the book. Here is a picture of the program after the user has clicked one of the filled squares:



Recursion is used in this program to count the number of squares in a blob. Without recursion, this would be a very difficult thing to implement. Recursion makes it relatively easy, but it still requires a new technique, which is also useful in a number of other applications.

The data for the grid of squares is stored in a two dimensional array of boolean values,

```
boolean[][]  filled;
```

The value of `filled[r][c]` is true if the square in row `r` and in column `c` of the grid is filled. The number of rows in the grid is stored in an instance variable named `rows`, and the number of columns is stored in `columns`. The program uses a recursive instance method named `getBlobSize()` to count the number of squares in the blob that contains the square in a given row `r` and column `c`. If there is no filled square at position `(r,c)`, then the answer is zero. Otherwise, `getBlobSize()` has to count all the filled squares that can be reached from the square at position `(r,c)`. The idea is to use `getBlobSize()` recursively to get the number of filled squares that can be reached from each of the neighboring positions, `(r+1,c)`, `(r-1,c)`, `(r,c+1)`, and `(r,c-1)`. Add up these numbers, and add one to count the square at `(r,c)` itself, and you get the total number of filled squares that can be reached from `(r,c)`. Here is an implementation of this algorithm, as stated. Unfortunately, it has a serious flaw: It leads to an infinite recursion!

```
int getBlobSize(int r, int c) {  // BUGGY, INCORRECT VERSION!!
      // This INCORRECT method tries to count all the filled
      // squares that can be reached from position (r,c) in the grid.
   if (r < 0 || r >= rows || c < 0 || c >= columns) {
         // This position is not in the grid, so there is
         // no blob at this position.  Return a blob size of zero.
      return 0;
   }
   if (filled[r][c] == false) {
        // This square is not part of a blob, so return zero.
      return 0;
   }
   int size = 1;  // Count the square at this position, then count the
```

```
                //   the blobs that are connected to this square
                //   horizontally or vertically.
   size += getBlobSize(r-1,c);
   size += getBlobSize(r+1,c);
   size += getBlobSize(r,c-1);
   size += getBlobSize(r,c+1);
   return size;
}  // end INCORRECT getBlobSize()
```

Unfortunately, this routine will count the same square more than once. In fact, it will try to count each square infinitely often! Think of yourself standing at position `(r,c)` and trying to follow these instructions. The first instruction tells you to move up one row. You do that, and then you apply the same procedure. As one of the steps in that procedure, you have to move **down** one row and apply the same procedure yet again. But that puts you back at position `(r,c)`! From there, you move up one row, and from there you move down one row.... Back and forth forever! We have to make sure that a square is only counted and processed once, so we don't end up going around in circles. The solution is to leave a trail of breadcrumbs—or on the computer a trail of boolean values—to mark the squares that you've already visited. Once a square is marked as visited, it won't be processed again. The remaining, unvisited squares are reduced in number, so definite progress has been made in reducing the size of the problem. Infinite recursion is avoided!

A second boolean array, `visited[r][c]`, is used to keep track of which squares have already been visited and processed. It is assumed that all the values in this array are set to false before `getBlobSize()` is called. As `getBlobSize()` encounters unvisited squares, it marks them as visited by setting the corresponding entry in the `visited` array to `true`. When `getBlobSize()` encounters a square that is already visited, it doesn't count it or process it further. The technique of "marking" items as they are encountered is one that used over and over in the programming of recursive algorithms. Here is the corrected version of `getBlobSize()`, with changes shown in italic:

```
/**
 * Counts the squares in the blob at position (r,c) in the
 * grid.  Squares are only counted if they are filled and
 * unvisited.  If this routine is called for a position that
 * has been visited, the return value will be zero.
 */
int getBlobSize(int r, int c) {
   if (r < 0 || r >= rows || c < 0 || c >= columns) {
         // This position is not in the grid, so there is
         // no blob at this position.  Return a blob size of zero.
      return 0;
   }
   if (filled[r][c] == false || visited[r][c] == true) {
        // This square is not part of a blob, or else it has
        // already been counted, so return zero.
      return 0;
   }
   visited[r][c] = true;   // Mark the square as visited so that
                           //    we won't count it again during the
                           //    following recursive calls.
   int size = 1;  // Count the square at this position, then count the
                  //   the blobs that are connected to this square
```

```
                //   horizontally or vertically.
   size += getBlobSize(r-1,c);
   size += getBlobSize(r+1,c);
   size += getBlobSize(r,c-1);
   size += getBlobSize(r,c+1);
   return size;
}  // end getBlobSize()
```

In the program, this method is used to determine the size of a blob when the user clicks on a square. After `getBlobSize()` has performed its task, all the squares in the blob are still marked as visited. The `paintComponent()` method draws visited squares in red, which makes the blob visible. The `getBlobSize()` method is also used for counting blobs. This is done by the following method, which includes comments to explain how it works:

```
/**
 * When the user clicks the "Count the Blobs" button, find the
 * number of blobs in the grid and report the number in the
 * message label.
 */
void countBlobs() {

   int count = 0; // Number of blobs.

   /* First clear out the visited array. The getBlobSize() method
      will mark every filled square that it finds by setting the
      corresponding element of the array to true.  Once a square
      has been marked as visited, it will stay marked until all the
      blobs have been counted.  This will prevent the same blob from
      being counted more than once. */

   for (int r = 0; r < rows; r++)
      for (int c = 0; c < columns; c++)
         visited[r][c] = false;

   /* For each position in the grid, call getBlobSize() to get the
      size of the blob at that position.  If the size is not zero,
      count a blob.  Note that if we come to a position that was part
      of a previously counted blob, getBlobSize() will return 0 and
      the blob will not be counted again. */

   for (int r = 0; r < rows; r++)
      for (int c = 0; c < columns; c++) {
         if (getBlobSize(r,c) > 0)
            count++;
      }

   repaint();  // Note that all the filled squares will be red,
               //   since they have all now been visited.

   message.setText("The number of blobs is " + count);

} // end countBlobs()
```

9.2 Linked Data Structures

EVERY USEFUL OBJECT contains instance variables. When the type of an instance variable is given by a class or interface name, the variable can hold a reference to another object. Such a reference is also called a pointer, and we say that the variable ***points to*** the object. (Of course, any variable that can contain a reference to an object can also contain the special value `null`, which points to nowhere.) When one object contains an instance variable that points to another object, we think of the objects as being "linked" by the pointer. Data structures of great complexity can be constructed by linking objects together.

9.2.1 Recursive Linking

Something interesting happens when an object contains an instance variable that can refer to another object of the same type. In that case, the definition of the object's class is recursive. Such recursion arises naturally in many cases. For example, consider a class designed to represent employees at a company. Suppose that every employee except the boss has a supervisor, who is another employee of the company. Then the *Employee* class would naturally contain an instance variable of type *Employee* that points to the employee's supervisor:

```
/**
 * An object of type Employee holds data about one employee.
 */
public class Employee {

   String name;          // Name of the employee.

   Employee supervisor;  // The employee's supervisor.

      .
      .  // (Other instance variables and methods.)
      .

} // end class Employee
```

If `emp` is a variable of type *Employee*, then `emp.supervisor` is another variable of type *Employee*. If `emp` refers to the boss, then the value of `emp.supervisor` should be `null` to indicate the fact that the boss has no supervisor. If we wanted to print out the name of the employee's supervisor, for example, we could use the following Java statement:

```
if ( emp.supervisor == null) {
   System.out.println( emp.name + " is the boss and has no supervisor!" );
}
else {
   System.out.print( "The supervisor of " + emp.name + " is " );
   System.out.println( emp.supervisor.name );
}
```

Now, suppose that we want to know how many levels of supervisors there are between a given employee and the boss. We just have to follow the chain of command through a series of `supervisor` links, and count how many steps it takes to get to the boss:

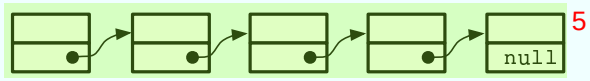
```
if ( emp.supervisor == null ) {
   System.out.println( emp.name + " is the boss!" );
}
else {
```

```
   Employee runner;  // For "running" up the chain of command.
   runner = emp.supervisor;
   if ( runner.supervisor == null) {
      System.out.println( emp.name  + " reports directly to the boss." );
   }
   else {
      int count = 0;
      while ( runner.supervisor != null ) {
         count++;  // Count the supervisor on this level.
         runner = runner.supervisor; // Move up to the next level.
      }
      System.out.println( "There are " + count
                              + " supervisors between " + emp.name
                              + " and the boss." );
   }
}
```

2

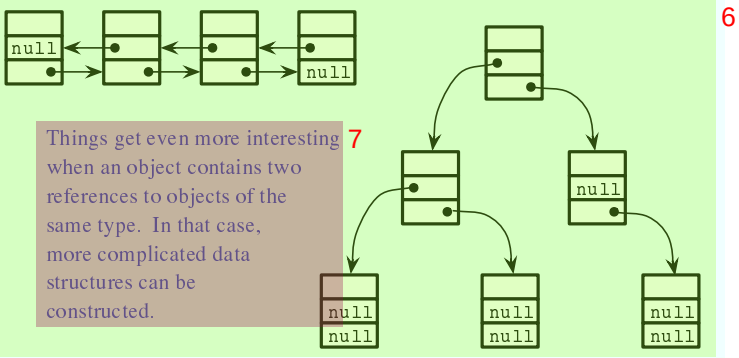
As the `while` loop is executed, `runner` points in turn to the original employee, `emp`, then to `emp's` supervisor, then to the supervisor of `emp's` supervisor, and so on. The `count` variable is incremented each time `runner` "visits" a new employee. The loop ends when `runner.supervisor` is `null`, which indicates that `runner` has reached the boss. At that point, `count` has counted the number of steps between `emp` and the boss. 3

In this example, the `supervisor` variable is quite natural and useful. In fact, data structures that are built by linking objects together are so useful that they are a major topic of study in computer science. We'll be looking at a few typical examples. In this section and the next, we'll be looking at ***linked lists***. A linked list consists of a chain of objects of the same type, linked together by pointers from one object to the next. This is much like the chain of supervisors between `emp` and the boss in the above example. It's also possible to have more complex situations, in which one object can contain links to several other objects. We'll look at an example of this in Section 9.4. 4



5

When an object contains a reference to an object of the same type, then several objects can be linked together into a list. Each object refers to the next object. 1



6

Things get even more interesting when an object contains two references to objects of the same type. In that case, more complicated data structures can be constructed. 7

9.2.2 Linked Lists

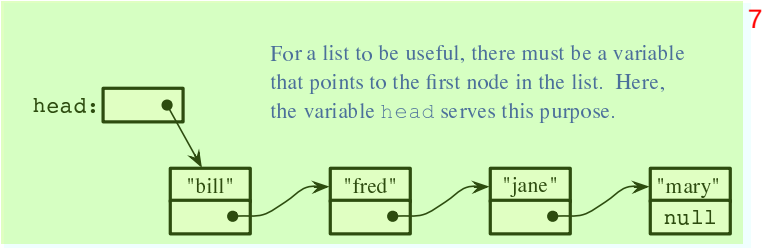
For most of the examples in the rest of this section, linked lists will be constructed out of objects belonging to the class *Node* which is defined as follows:

```
class Node {
   String item;
   Node next;
}
```

The term ***node*** is often used to refer to one of the objects in a linked data structure. Objects of type *Node* can be chained together as shown in the top part of the above picture. Each node holds a *String* and a pointer to the next node in the list (if any). The last node in such a list can always be identified by the fact that the instance variable `next` in the last node holds the value `null` instead of a pointer to another node. The purpose of the chain of nodes is to represent a list of strings. The first string in the list is stored in the first node, the second string is stored in the second node, and so on. The pointers and the node objects are used to build the structure, but the data that we are interested in representing is the list of strings. Of course, we could just as easily represent a list of integers or a list of *JButtons* or a list of any other type of data by changing the type of the `item` that is stored in each node.

Although the *Nodes* in this example are very simple, we can use them to illustrate the common operations on linked lists. Typical operations include deleting nodes from the list, inserting new nodes into the list, and searching for a specified *String* among the `items` in the list. We will look at subroutines to perform all of these operations, among others.

For a linked list to be used in a program, that program needs a variable that refers to the first node in the list. It only needs a pointer to the first node since all the other nodes in the list can be accessed by starting at the first node and following links along the list from one node to the next. In my examples, I will always use a variable named `head`, of type *Node*, that points to the first node in the linked list. When the list is empty, the value of `head` is null.



9.2.3 Basic Linked List Processing

It is very common to want to process all the items in a linked list in some way. The common pattern is to start at the head of the list, then move from each node to the next by by following the pointer in the node, stopping when the `null` that marks the end of the list is reached. If `head` is a variable of type *Node* that points to the first node in the list, then the general form of the code is:

```
Node runner;    // A pointer that will be used to traverse the list.
runner = head;  // Start with runner pointing to the head of the list.
while ( runner != null ) {     // Continue until null is encountered.
   process( runner.item );     // Do something with the item in the current node.
```

```
   runner = runner.next;       // Move on to the next node in the list.
}
```

Our only access to the list is through the variable `head`, so we start by getting a copy of the value in `head` with the assignment statement `runner = head`. We need a **copy** of `head` because we are going to change the value of `runner`. We can't change the value of `head`, or we would lose our only access to the list! The variable `runner` will point to each node of the list in turn. When `runner` points to one of the nodes in the list, `runner.next` is a pointer to the next node in the list, so the assignment statement `runner = runner.next` moves the pointer along the list from each node to the next. We know that we've reached the end of the list when `runner` becomes equal to `null`. Note that our list-processing code works even for an empty list, since for an empty list the value of `head` is `null` and the body of the while loop is not executed at all. As an example, we can print all the strings in a list of *Strings* by saying:

```
Node runner = head;
while ( runner != null ) {
   System.out.println(  runner.item );
   runner = runner.next;
}
```

The `while` loop can, by the way, be rewritten as a `for` loop. Remember that even though the loop control variable in a `for` loop is often numerical, that is not a requirement. Here is a `for` loop that is equivalent to the above `while` loop:

```
for ( Node runner = head; runner != null; runner = runner.next ) {
   System.out.println( runner.item );
}
```

Similarly, we can traverse a list of integers to add up all the numbers in the list. A linked list of integers can be constructed using the class

```
public class IntNode {
   int item;       // One of the integers in the list.
   IntNode next;   // Pointer to the next node in the list.
}
```

If `head` is a variable of type *IntNode* that points to a linked list of integers, we can find the sum of the integers in the list using:

```
int sum = 0;
IntNode runner = head;
while ( runner != null ) {
   sum = sum + runner.item;   // Add current item to the sum.
   runner = runner.next;
}
System.out.println("The sum of the list items is " + sum);
```

It is also possible to use recursion to process a linked list. Recursion is rarely the natural way to process a list, since it's so easy to use a loop to traverse the list. However, understanding how to apply recursion to lists can help with understanding the recursive processing of more complex data structures. A non-empty linked list can be thought of as consisting of two parts: the ***head*** of the list, which is just the first node in the list, and the ***tail*** of the list, which consists of the remainder of the list after the head. Note that the tail is itself a linked list and that it is shorter than the original list (by one node). This is a natural setup for recursion, where the problem of processing a list can be divided into processing the head and recursively

processing the tail. The base case occurs in the case of an empty list (or sometimes in the case of a list of length one). For example, here is a recursive algorithm for adding up the numbers in a linked list of integers:

```
if the list is empty then
   return 0 (since there are no numbers to be added up)
otherwise
   let listsum = the number in the head node
   let tailsum be the sum of the numbers in the tail list (recursively)
   add tailsum to listsum
   return listsum
```

One remaining question is, how do we get the tail of a non-empty linked list? If `head` is a variable that points to the head node of the list, then `head.next` is a variable that points to the second node of the list—and that node is in fact the first node of the tail. So, we can view `head.next` as a pointer to the tail of the list. One special case is when the original list consists of a single node. In that case, the tail of the list is empty, and `head.next` is `null`. Since an empty list is represented by a null pointer, `head.next` represents the tail of the list even in this special case. This allows us to write a recursive list-summing function in Java as

```
/**
 *  Compute the sum of all the integers in a linked list of integers.
 *  @param head a pointer to the first node in the linked list
 */
public static int addItemsInList( IntNode head ) {
   if ( head == null ) {
         // Base case:  The list is empty,  so the sum is zero.
      return 0;
   }
   else {
         // Recursive case:  The list is non-empty.  Find the sum of
         // the tail list, and add that to the item in the head node.
         // (Note that this case could be written simply as
         //     return head.item + addItemsInList( head.next );)
      int listsum = head.item;
      int tailsum = addItemsInList( head.next );
      listsum = listsum + tailsum;
      return listsum;
   }
}
```

I will finish by presenting a list-processing problem that is easy to solve with recursion, but quite tricky to solve without it. The problem is to print out all the strings in a linked list of strings in the **reverse** of the order in which they occur in the list. Note that when we do this, the item in the head of a list is printed out after all the items in the tail of the list. This leads to the following recursive routine. You should convince yourself that it works, and you should think about trying to do the same thing without using recursion:

```
public static void printReversed( Node head ) {
   if ( head == null ) {
         // Base case:  The list is empty, and there is nothing to print.
      return;
   }
   else {
```

```
         // Recursive case:  The list is non-empty.
      printReversed( head.next );  // Print strings in tail, in reverse order.
      System.out.println( head.item );  // Print string in head node.
   }
}
```

* * *

In the rest of this section, we'll look at a few more advanced operations on a linked list of strings. The subroutines that we consider are instance methods in a class, *StringList*. An object of type *StringList* represents a linked list of nodes. The class has a private instance variable named `head` of type *Node* that points to the first node in the list, or is null if the list is empty. Instance methods in class *StringList* access `head` as a global variable. The source code for *StringList* is in the file *StringList.java*, and it is used in the sample program *ListDemo.java*.

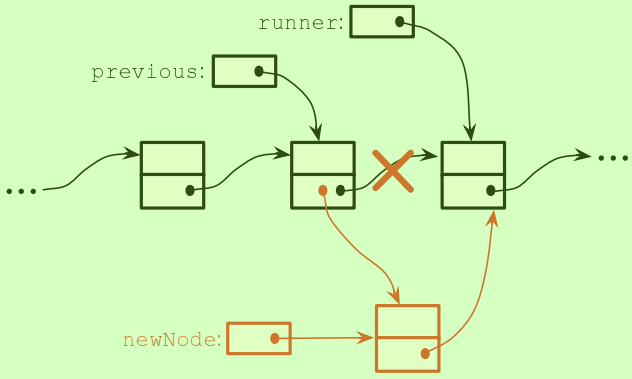
Suppose we want to know whether a specified string, `searchItem`, occurs somewhere in a list of strings. We have to compare `searchItem` to each `item` in the list. This is an example of basic list traversal and processing. However, in this case, we can stop processing if we find the item that we are looking for.

```
/**
 * Searches the list for a specified item.
 * @param searchItem the item that is to be searched for
 * @return true if searchItem is one of the items in the list or false if
 *    searchItem does not occur in the list.
 */
public boolean find(String searchItem) {

   Node runner;    // A pointer for traversing the list.

   runner = head;  // Start by looking at the head of the list.
                   //   (head is an instance variable! )

   while ( runner != null ) {
         // Go through the list looking at the string in each
         // node.  If the string is the one we are looking for,
         // return true, since the string has been found in the list.
      if ( runner.item.equals(searchItem) )
         return true;
      runner = runner.next;  // Move on to the next node.
   }

   // At this point, we have looked at all the items in the list
   // without finding searchItem.  Return false to indicate that
   // the item does not exist in the list.

   return false;

} // end find()
```

It is possible that the list is empty, that is, that the value of `head` is `null`. We should be careful that this case is handled properly. In the above code, if `head` is `null`, then the body of the `while` loop is never executed at all, so no nodes are processed and the return value is `false`. This is exactly what we want when the list is empty, since the `searchItem` can't occur in an empty list.

9.2.4 Inserting into a Linked List

The problem of inserting a new item into a linked list is more difficult, at least in the case where the item is inserted into the middle of the list. (In fact, it's probably the most difficult operation on linked data structures that you'll encounter in this chapter.) In the *StringList* class, the `items` in the nodes of the linked list are kept in increasing order. When a new item is inserted into the list, it must be inserted at the correct position according to this ordering. This means that, usually, we will have to insert the new item somewhere in the middle of the list, between two existing nodes. To do this, it's convenient to have two variables of type *Node*, which refer to the existing nodes that will lie on either side of the new node. In the following illustration, these variables are `previous` and `runner`. Another variable, `newNode`, refers to the new node. In order to do the insertion, the link from `previous` to `runner` must be "broken," and new links from `previous` to `newNode` and from `newNode` to `runner` must be added:



Inserting a new node
into the middle of a list

Once we have `previous` and `runner` pointing to the right nodes, the command "`previous.next = newNode;`" can be used to make `previous.next` point to the new node, instead of to the node indicated by `runner`. And the command "`newNode.next = runner`" will set `newNode.next` to point to the correct place. However, before we can use these commands, we need to set up `runner` and `previous` as shown in the illustration. The idea is to start at the first node of the list, and then move along the list past all the items that are less than the new item. While doing this, we have to be aware of the danger of "falling off the end of the list." That is, we can't continue if `runner` reaches the end of the list and becomes `null`. If `insertItem` is the item that is to be inserted, and if we assume that it does, in fact, belong somewhere in the middle of the list, then the following code would correctly position `previous` and `runner`:

```
Node runner, previous;
previous = head;     // Start at the beginning of the list.
runner = head.next;
while ( runner != null && runner.item.compareTo(insertItem) < 0 ) {
   previous = runner;  // "previous = previous.next" would also work
   runner = runner.next;
}
```

(This uses the `compareTo()` instance method from the *String* class to test whether the item in the node is less than the item that is being inserted. See Subsection 2.3.2.)

This is fine, except that the assumption that the new node is inserted into the middle of the list is not always valid. It might be that `insertItem` is less than the first item of the list. In that case, the new node must be inserted at the head of the list. This can be done with the instructions

```
newNode.next = head;   // Make newNode.next point to the old head.
head = newNode;        // Make newNode the new head of the list.
```

It is also possible that the list is empty. In that case, `newNode` will become the first and only node in the list. This can be accomplished simply by setting `head = newNode`. The following `insert()` method from the *StringList* class covers all of these possibilities:

```
/**
 * Insert a specified item to the list, keeping the list in order.
 * @param insertItem the item that is to be inserted.
 */
public void insert(String insertItem) {

   Node newNode;          // A Node to contain the new item.
   newNode = new Node();
   newNode.item = insertItem;  // (N.B.  newNode.next is null.)

   if ( head == null ) {
          // The new item is the first (and only) one in the list.
          // Set head to point to it.
      head = newNode;
   }
   else if ( head.item.compareTo(insertItem) >= 0 ) {
          // The new item is less than the first item in the list,
          // so it has to be inserted at the head of the list.
      newNode.next = head;
      head = newNode;
   }
   else {
          // The new item belongs somewhere after the first item
          // in the list.  Search for its proper position and insert it.
      Node runner;     // A node for traversing the list.
      Node previous;   // Always points to the node preceding runner.
      runner = head.next;   // Start by looking at the SECOND position.
      previous = head;
      while ( runner != null && runner.item.compareTo(insertItem) < 0 ) {
             // Move previous and runner along the list until runner
             // falls off the end or hits a list element that is
             // greater than or equal to insertItem.  When this
             // loop ends, runner indicates the position where
             // insertItem must be inserted.
         previous = runner;
         runner = runner.next;
      }
      newNode.next = runner;     // Insert newNode after previous.
      previous.next = newNode;
   }

}  // end insert()
```

If you were paying close attention to the above discussion, you might have noticed that there is one special case which is not mentioned. What happens if the new node has to be inserted at the **end** of the list? This will happen if all the items in the list are less than the new item. In fact, this case is already handled correctly by the subroutine, in the last part of the `if` statement. If `insertItem` is greater than all the items in the list, then the `while` loop will end when `runner` has traversed the entire list and become `null`. However, when that happens, `previous` will be left pointing to the last node in the list. Setting `previous.next = newNode` adds `newNode` onto the end of the list. Since `runner` is `null`, the command `newNode.next = runner` sets `newNode.next` to `null`, which is the correct value that is needed to mark the end of the list.

9.2.5 Deleting from a Linked List

The delete operation is similar to insert, although a little simpler. There are still special cases to consider. When the first node in the list is to be deleted, then the value of `head` has to be changed to point to what was previously the second node in the list. Since `head.next` refers to the second node in the list, this can be done by setting `head = head.next`. (Once again, you should check that this works when `head.next` is `null`, that is, when there is no second node in the list. In that case, the list becomes empty.)

If the node that is being deleted is in the middle of the list, then we can set up `previous` and `runner` with `runner` pointing to the node that is to be deleted and with `previous` pointing to the node that precedes that node in the list. Once that is done, the command "`previous.next = runner.next;`" will delete the node. The deleted node will be garbage collected. I encourage you to draw a picture for yourself to illustrate this operation. Here is the complete code for the `delete()` method:

```
/**
 * Delete a specfied item from the list, if that item is present.
 * If multiple copies of the item are present in the list, only
 * the one that comes first in the list is deleted.
 * @param deleteItem the item to be deleted
 * @return true if the item was found and deleted, or false if the item
 *    was not in the list.
 */
public boolean delete(String deleteItem) {

   if ( head == null ) {
          // The list is empty, so it certainly doesn't contain deleteString.
      return false;
   }
   else if ( head.item.equals(deleteItem) ) {
           // The string is the first item of the list.  Remove it.
      head = head.next;
      return true;
   }
   else {
          // The string, if it occurs at all, is somewhere beyond the
          // first element of the list.  Search the list.
      Node runner;     // A node for traversing the list.
      Node previous;   // Always points to the node preceding runner.
      runner = head.next;   // Start by looking at the SECOND list node.
      previous = head;
```

```
      while ( runner != null && runner.item.compareTo(deleteItem) < 0 ) {
             // Move previous and runner along the list until runner
             // falls off the end or hits a list element that is
             // greater than or equal to deleteItem.  When this
             // loop ends, runner indicates the position where
             // deleteItem must be, if it is in the list.
         previous = runner;
         runner = runner.next;
      }
      if ( runner != null && runner.item.equals(deleteItem) ) {
             // Runner points to the node that is to be deleted.
             // Remove it by changing the pointer in the previous node.
         previous.next = runner.next;
         return true;
      }
      else {
             // The item does not exist in the list.
         return false;
      }
   }

} // end delete()
```

9.3 Stacks, Queues, and ADTs

A LINKED LIST is a particular type of data structure, made up of objects linked together by pointers. In the previous section, we used a linked list to store an ordered list of *Strings*, and we implemented `insert`, `delete`, and `find` operations on that list. However, we could easily have stored the list of *Strings* in an array or *ArrayList*, instead of in a linked list. We could still have implemented the same operations on the list. The implementations of these operations would have been different, but their interfaces and logical behavior would still be the same.

The term ***abstract data type***, or ***ADT***, refers to a set of possible values and a set of operations on those values, without any specification of how the values are to be represented or how the operations are to be implemented. An "ordered list of strings" can be defined as an abstract data type. Any sequence of *Strings* that is arranged in increasing order is a possible value of this data type. The operations on the data type include inserting a new string, deleting a string, and finding a string in the list. There are often several different ways to implement the same abstract data type. For example, the "ordered list of strings" ADT can be implemented as a linked list or as an array. A program that only depends on the abstract definition of the ADT can use either implementation, interchangeably. In particular, the implementation of the ADT can be changed without affecting the program as a whole. This can make the program easier to debug and maintain, so ADTs are an important tool in software engineering.

In this section, we'll look at two common abstract data types, ***stacks*** and ***queues***. Both stacks and queues are often implemented as linked lists, but that is not the only possible implementation. You should think of the rest of this section partly as a discussion of stacks and queues and partly as a case study in ADTs.

9.3.1 Stacks 1

A stack consists of a sequence of items, which should be thought of as piled one on top of the other like a physical stack of boxes or cafeteria trays. Only the top item on the stack is accessible at any given time. It can be removed from the stack with an operation called ***pop***. An item lower down on the stack can only be removed after all the items on top of it have been popped off the stack. A new item can be added to the top of the stack with an operation called ***push***. We can make a stack of any type of items. If, for example, the items are values of type **int**, then the push and pop operations can be implemented as instance methods 2

- `void push (int newItem)` — Add newItem to top of stack. 3
- `int pop()` — Remove the top int from the stack and return it. 4

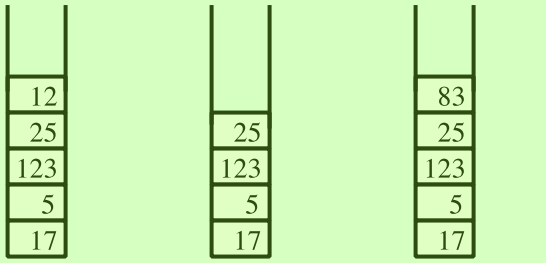
It is an error to try to pop an item from an empty stack, so it is important to be able to tell whether a stack is empty. We need another stack operation to do the test, implemented as an instance method 5

- `boolean isEmpty()` — Returns true if the stack is empty. 6

This defines a "stack of ints" as an abstract data type. This ADT can be implemented in several ways, but however it is implemented, its behavior must correspond to the abstract mental image of a stack. 7

9

In a stack, all operations take place at the "top" of the stack. The "push" operation adds an item to the top of the stack. The "pop" operation removes the item on the top of the stack and returns it. 8



In the linked list implementation of a stack, the top of the stack is actually the node at the head of the list. It is easy to add and remove nodes at the front of a linked list—much easier than inserting and deleting nodes in the middle of the list. Here is a class that implements the "stack of ints" ADT using a linked list. (It uses a static nested class to represent the nodes of the linked list. If the nesting bothers you, you could replace it with a separate *Node* class.) 10

```
public class StackOfInts { 11

   /** 12
    * An object of type Node holds one of the items in the linked list 13
    * that represents the stack. 14
    */
```

```
   private static class Node {
      int item;
      Node next;
   }

   private Node top;  // Pointer to the Node that is at the top of
                      //   of the stack.  If top == null, then the
                      //   stack is empty.

   /**
    * Add N to the top of the stack.
    */
   public void push( int N ) {
      Node newTop;         // A Node to hold the new item.
      newTop = new Node();
      newTop.item = N;     // Store N in the new Node.
      newTop.next = top;   // The new Node points to the old top.
      top = newTop;        // The new item is now on top.
   }

   /**
    * Remove the top item from the stack, and return it.
    * Throws an IllegalStateException if the stack is empty when
    * this method is called.
    */
   public int pop() {
      if ( top == null )
         throw new IllegalStateException("Can't pop from an empty stack.");
      int topItem = top.item;  // The item that is being popped.
      top = top.next;          // The previous second item is now on top.
      return topItem;
   }

   /**
    * Returns true if the stack is empty.  Returns false
    * if there are one or more items on the stack.
    */
   public boolean isEmpty() {
      return (top == null);
   }

} // end class StackOfInts
```

1

You should make sure that you understand how the `push` and `pop` operations operate on the linked list. Drawing some pictures might help. Note that the linked list is part of the `private` implementation of the *StackOfInts* class. A program that uses this class doesn't even need to know that a linked list is being used.

2

Now, it's pretty easy to implement a stack as an array instead of as a linked list. Since the number of items on the stack varies with time, a counter is needed to keep track of how many spaces in the array are actually in use. If this counter is called `top`, then the items on the stack are stored in positions `0`, `1`, ..., `top-1` in the array. The item in position `0` is on the bottom of the stack, and the item in position `top-1` is on the top of the stack. Pushing an item onto the stack is easy: Put the item in position `top` and add `1` to the value of `top`. If we don't want to put a limit on the number of items that the stack can hold, we can use the dynamic array techniques from Subsection 7.3.2. Note that the typical picture of the array would show the

3

stack "upside down", with the top of the stack at the bottom of the array. This doesn't matter. The array is just an implementation of the abstract idea of a stack, and as long as the stack operations work the way they are supposed to, we are OK. Here is a second implementation of the *StackOfInts* class, using a dynamic array:

```
public class StackOfInts {  // (alternate version, using an array)

   private int[] items = new int[10];  // Holds the items on the stack.

   private int top = 0;  // The number of items currently on the stack.

   /**
    * Add N to the top of the stack.
    */
   public void push( int N ) {
       if (top == items.length) {
              // The array is full, so make a new, larger array and
              // copy the current stack items into it.
           int[] newArray = new int[ 2*items.length ];
           System.arraycopy(items, 0, newArray, 0, items.length);
           items = newArray;
       }
       items[top] = N;  // Put N in next available spot.
       top++;           // Number of items goes up by one.
   }

   /**
    * Remove the top item from the stack, and return it.
    * Throws an IllegalStateException if the stack is empty when
    * this method is called.
    */
   public int pop() {
       if ( top == 0 )
          throw new IllegalStateException("Can't pop from an empty stack.");
       int topItem = items[top - 1]  // Top item in the stack.
       top--;    // Number of items on the stack goes down by one.
       return topItem;
   }

   /**
    * Returns true if the stack is empty.  Returns false
    * if there are one or more items on the stack.
    */
   public boolean isEmpty() {
      return (top == 0);
   }

} // end class StackOfInts
```

Once again, the implentation of the stack (as an array) is private to the class. The two versions of the *StackOfInts* class can be used interchangeably, since their public interfaces are identical.

* * *

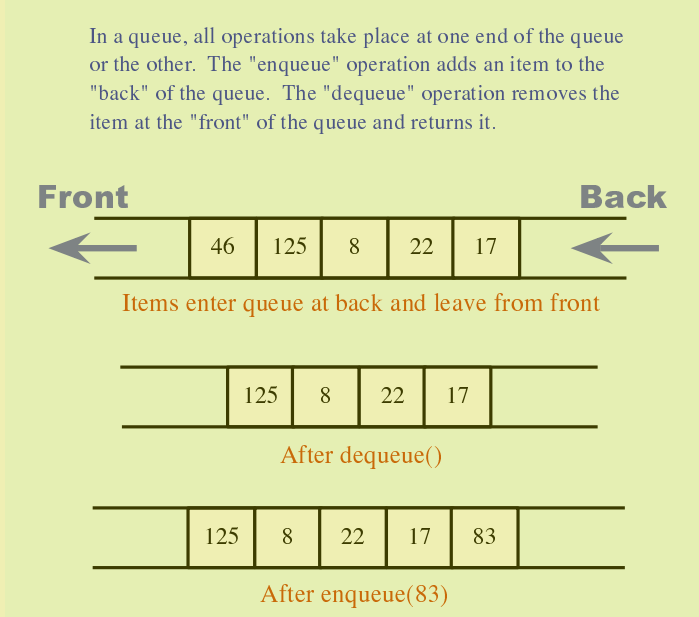
It's interesting to look at the run time analysis of stack operations. (See Section 8.6). We can measure the size of the problem by the number of items that are on the stack. For the linked list implementation of a stack, the worst case run time both for the `push` and for the `pop`

operation is $\Theta(1)$. This just means that the run time is less than some constant, independent of the number of items on the stack. This is easy to see if you look at the code. The operations are implemented with a few simple assignment statements, and the number of items on the stack has no effect.

For the array implementation, on the other hand, a special case occurs in the `push` operation when the array is full. In that case, a new array is created and all the stack items are copied into the new array. This takes an amount of time that is proportional to the number of items on the stack. So, although the run time for `push` is usually $\Theta(1)$, the worst case run time is $\Theta(n)$, where n is the number of items on the stack.

9.3.2 Queues

Queues are similar to stacks in that a queue consists of a sequence of items, and there are restrictions about how items can be added to and removed from the list. However, a queue has two ends, called the front and the back of the queue. Items are always added to the queue at the back and removed from the queue at the front. The operations of adding and removing items are called ***enqueue*** and ***dequeue***. An item that is added to the back of the queue will remain on the queue until all the items in front of it have been removed. This should sound familiar. A queue is like a "line" or "queue" of customers waiting for service. Customers are serviced in the order in which they arrive on the queue.



A queue can hold items of any type. For a queue of `ints`, the enqueue and dequeue operations can be implemented as instance methods in a "*QueueOfInts*" class. We also need an instance method for checking whether the queue is empty:

- `void enqueue(int N)` — Add N to the back of the queue.
- `int dequeue()` — Remove the item at the front and return it.
- `boolean isEmpty()` — Return true if the queue is empty.

A queue can be implemented as a linked list or as an array. An efficient array implementation is a little trickier than the array implementation of a stack, so I won't give it here. In the linked

list implementation, the first item of the list is at the front of the queue. Dequeueing an item from the front of the queue is just like popping an item off a stack. The back of the queue is at the end of the list. Enqueueing an item involves setting a pointer in the last node of the current list to point to a new node that contains the item. To do this, we'll need a command like "`tail.next = newNode;`", where `tail` is a pointer to the last node in the list. If `head` is a pointer to the first node of the list, it would always be possible to get a pointer to the last node of the list by saying:

```
Node tail;    // This will point to the last node in the list.
tail = head;  // Start at the first node.
while (tail.next != null) {
   tail = tail.next;  // Move to next node.
}
// At this point, tail.next is null, so tail points to
// the last node in the list.
```

However, it would be very inefficient to do this over and over every time an item is enqueued. For the sake of efficiency, we'll keep a pointer to the last node in an instance variable. This complicates the class somewhat; we have to be careful to update the value of this variable whenever a new node is added to the end of the list. Given all this, writing the *QueueOfInts* class is not all that difficult:

```
public class QueueOfInts {

   /**
    * An object of type Node holds one of the items
    * in the linked list that represents the queue.
    */
   private static class Node {
      int item;
      Node next;
   }

   private Node head = null;  // Points to first Node in the queue.
                              // The queue is empty when head is null.

   private Node tail = null;  // Points to last Node in the queue.

   /**
    * Add N to the back of the queue.
    */
   public void enqueue( int N ) {
      Node newTail = new Node();  // A Node to hold the new item.
      newTail.item = N;
      if (head == null) {
            // The queue was empty.  The new Node becomes
            // the only node in the list.  Since it is both
            // the first and last node, both head and tail
            // point to it.
         head = newTail;
         tail = newTail;
      }
      else {
            // The new node becomes the new tail of the list.
            // (The head of the list is unaffected.)
```

```
            tail.next = newTail;
            tail = newTail;
        }
    }

    /**
     * Remove and return the front item in the queue.
     * Throws an IllegalStateException if the queue is empty.
     */
    public int dequeue() {
        if ( head == null)
            throw new IllegalStateException("Can't dequeue from an empty queue.");
        int firstItem = head.item;
        head = head.next;  // The previous second item is now first.
        if (head == null) {
              // The queue has become empty.  The Node that was
              // deleted was the tail as well as the head of the
              // list, so now there is no tail.  (Actually, the
              // class would work fine without this step.)
            tail = null;
        }
        return firstItem;
    }

    /**
     * Return true if the queue is empty.
     */
    boolean isEmpty() {
        return (head == null);
    }

} // end class QueueOfInts
```

Queues are typically used in a computer (as in real life) when only one item can be processed at a time, but several items can be waiting for processing. For example:

- In a Java program that has multiple threads, the threads that want processing time on the CPU are kept in a queue. When a new thread is started, it is added to the back of the queue. A thread is removed from the front of the queue, given some processing time, and then—if it has not terminated—is sent to the back of the queue to wait for another turn.
- Events such as keystrokes and mouse clicks are stored in a queue called the "event queue". A program removes events from the event queue and processes them. It's possible for several more events to occur while one event is being processed, but since the events are stored in a queue, they will always be processed in the order in which they occurred.
- A web server is a progam that receives requests from web browsers for "pages." It is easy for new requests to arrive while the web server is still fulfilling a previous request. Requests that arrive while the web server is busy are placed into a queue to await processing. Using a queue ensures that requests will be processed in the order in which they were received.

Queues are said to implement a ***FIFO*** policy: First In, First Out. Or, as it is more commonly expressed, first come, first served. Stacks, on the other hand implement a ***LIFO*** policy: Last In, First Out. The item that comes out of the stack is the last one that was put in. Just like queues, stacks can be used to hold items that are waiting for processing (although in applications where queues are typically used, a stack would be considered "unfair").

* * *

To get a better handle on the difference between stacks and queues, consider the sample program *DepthBreadth.java*. I suggest that you run the program or try the applet version that can be found in the on-line version of this section. The program shows a grid of squares. Initially, all the squares are white. When you click on a white square, the program will gradually mark all the squares in the grid, starting from the one where you click. To understand how the program does this, think of yourself in the place of the program. When the user clicks a square, you are handed an index card. The location of the square—its row and column—is written on the card. You put the card in a pile, which then contains just that one card. Then, you repeat the following: If the pile is empty, you are done. Otherwise, take an index card from the pile. The index card specifies a square. Look at each horizontal and vertical neighbor of that square. If the neighbor has not already been encountered, write its location on a new index card and put the card in the pile.

While a square is in the pile, waiting to be processed, it is colored red; that is, red squares have been encountered but not yet processed. When a square is taken from the pile and processed, its color changes to gray. Once a square has been colored gray, its color won't change again. Eventually, all the squares have been processed, and the procedure ends. In the index card analogy, the pile of cards has been emptied.

The program can use your choice of three methods: Stack, Queue, and Random. In each case, the same general procedure is used. The only difference is how the "pile of index cards" is managed. For a stack, cards are added and removed at the top of the pile. For a queue, cards are added to the bottom of the pile and removed from the top. In the random case, the card to be processed is picked at random from among all the cards in the pile. The order of processing is very different in these three cases.

You should experiment with the program to see how it all works. Try to understand how stacks and queues are being used. Try starting from one of the corner squares. While the process is going on, you can click on other white squares, and they will be added to the pile. When you do this with a stack, you should notice that the square you click is processed immediately, and all the red squares that were already waiting for processing have to wait. On the other hand, if you do this with a queue, the square that you click will wait its turn until all the squares that were already in the pile have been processed.

* * *

Queues seem very natural because they occur so often in real life, but there are times when stacks are appropriate and even essential. For example, consider what happens when a routine calls a subroutine. The first routine is suspended while the subroutine is executed, and it will continue only when the subroutine returns. Now, suppose that the subroutine calls a second subroutine, and the second subroutine calls a third, and so on. Each subroutine is suspended while the subsequent subroutines are executed. The computer has to keep track of all the subroutines that are suspended. It does this with a stack.

When a subroutine is called, an ***activation record*** is created for that subroutine. The activation record contains information relevant to the execution of the subroutine, such as its local variables and parameters. The activation record for the subroutine is placed on a stack. It will be removed from the stack and destroyed when the subroutine returns. If the subroutine calls another subroutine, the activation record of the second subroutine is pushed onto the stack, on top of the activation record of the first subroutine. The stack can continue to grow as more subroutines are called, and it shrinks as those subroutines return.

9.3.3 Postfix Expressions

As another example, stacks can be used to evaluate ***postfix expressions***. An ordinary mathematical expression such as `2+(15-12)*17` is called an ***infix expression***. In an infix expression, an operator comes in between its two operands, as in "`2 + 2`". In a postfix expression, an operator comes after its two operands, as in "`2 2 +`". The infix expression "`2+(15-12)*17`" would be written in postfix form as "`2 15 12 - 17 * +`". The "`-`" operator in this expression applies to the two operands that precede it, namely "`15`" and "`12`". The "`*`" operator applies to the two operands that precede it, namely "`15 12 -`" and "`17`". And the "`+`" operator applies to "`2`" and "`15 12 - 17 *`". These are the same computations that are done in the original infix expression.

Now, suppose that we want to process the expression "`2 15 12 - 17 * +`", from left to right and find its value. The first item we encounter is the `2`, but what can we do with it? At this point, we don't know what operator, if any, will be applied to the `2` or what the other operand might be. We have to remember the `2` for later processing. We do this by pushing it onto a stack. Moving on to the next item, we see a `15`, which is pushed onto the stack on top of the `2`. Then the `12` is added to the stack. Now, we come to the operator, "`-`". This operation applies to the two operands that preceded it in the expression. We have saved those two operands on the stack. So, to process the "`-`" operator, we pop two numbers from the stack, `12` and `15`, and compute `15 - 12` to get the answer `3`. This `3` must be remembered to be used in later processing, so we push it onto the stack, on top of the `2` that is still waiting there. The next item in the expression is a `17`, which is processed by pushing it onto the stack, on top of the `3`. To process the next item, "`*`", we pop two numbers from the stack. The numbers are `17` and the `3` that represents the value of "`15 12 -`". These numbers are multiplied, and the result, `51` is pushed onto the stack. The next item in the expression is a "`+`" operator, which is processed by popping `51` and `2` from the stack, adding them, and pushing the result, `53`, onto the stack. Finally, we've come to the end of the expression. The number on the stack is the value of the entire expression, so all we have to do is pop the answer from the stack, and we are done! The value of the expression is `53`.

Although it's easier for people to work with infix expressions, postfix expressions have some advantages. For one thing, postfix expressions don't require parentheses or precedence rules. The order in which operators are applied is determined entirely by the order in which they occur in the expression. This allows the algorithm for evaluating postfix expressions to be fairly straightforward:

```
Start with an empty stack
for each item in the expression:
    if the item is a number:
       Push the number onto the stack
    else if the item is an operator:
       Pop the operands from the stack  // Can generate an error
       Apply the operator to the operands
       Push the result onto the stack
    else
       There is an error in the expression
Pop a number from the stack  // Can generate an error
if the stack is not empty:
   There is an error in the expression
else:
   The last number that was popped is the value of the expression
```

Errors in an expression can be detected easily. For example, in the expression "2 3 + *", there are not enough operands for the "*" operation. This will be detected in the algorithm when an attempt is made to pop the second operand for "*" from the stack, since the stack will be empty. The opposite problem occurs in "2 3 4 +". There are not enough operators for all the numbers. This will be detected when the 2 is left still sitting in the stack at the end of the algorithm. 1

This algorithm is demonstrated in the sample program *PostfixEval.java*. This program lets you type in postfix expressions made up of non-negative real numbers and the operators "+", "-", "*", "/", and "^". The "^" represents exponentiation. That is, "2 3 ^" is evaluated as 2^3. The program prints out a message as it processes each item in the expression. The stack class that is used in the program is defined in the file *StackOfDouble.java*. The *StackOfDouble* class is identical to the first *StackOfInts* class, given above, except that it has been modified to store values of type **double** instead of values of type **int**. 2

The only interesting aspect of this program is the method that implements the postfix evaluation algorithm. It is a direct implementation of the pseudocode algorithm given above: 3

```
/**
 *  Read one line of input and process it as a postfix expression.
 *  If the input is not a legal postfix expression, then an error
 *  message is displayed.  Otherwise, the value of the expression
 *  is displayed.  It is assumed that the first character on
 *  the input line is a non-blank.
 */
private static void readAndEvaluate() {

   StackOfDouble stack;  // For evaluating the expression.

   stack = new StackOfDouble();  // Make a new, empty stack.

   TextIO.putln();

   while (TextIO.peek() != '\n') {

      if ( Character.isDigit(TextIO.peek()) ) {
             // The next item in input is a number.  Read it and
             // save it on the stack.
         double num = TextIO.getDouble();
         stack.push(num);
         TextIO.putln("   Pushed constant " + num);
      }
      else {
             // Since the next item is not a number, the only thing
             // it can legally be is an operator.  Get the operator
             // and perform the operation.
         char op;  // The operator, which must be +, -, *, /, or ^.
         double x,y;     // The operands, from the stack, for the operation.
         double answer;  // The result, to be pushed onto the stack.
         op = TextIO.getChar();
         if (op != '+' && op != '-' && op != '*' && op != '/' && op != '^') {
                // The character is not one of the acceptable operations.
            TextIO.putln("\nIllegal operator found in input: " + op);
            return;
         }
         if (stack.isEmpty()) {
```
4

```
            TextIO.putln("   Stack is empty while trying to evaluate " + op);
            TextIO.putln("\nNot enough numbers in expression!");
            return;
         }
         y = stack.pop();
         if (stack.isEmpty()) {
            TextIO.putln("   Stack is empty while trying to evaluate " + op);
            TextIO.putln("\nNot enough numbers in expression!");
            return;
         }
         x = stack.pop();
         switch (op) {
         case '+':
            answer = x + y;
            break;
         case '-':
            answer = x - y;
            break;
         case '*':
            answer = x * y;
            break;
         case '/':
            answer = x / y;
            break;
         default:
            answer = Math.pow(x,y);  // (op must be '^'.)
         }
         stack.push(answer);
         TextIO.putln("   Evaluated " + op + " and pushed " + answer);
      }

      TextIO.skipBlanks();

   }  // end while

   // If we get to this point, the input has been read successfully.
   // If the expression was legal, then the value of the expression is
   // on the stack, and it is the only thing on the stack.

   if (stack.isEmpty()) {  // Impossible if the input is really non-empty.
      TextIO.putln("No expression provided.");
      return;
   }

   double value = stack.pop();  // Value of the expression.
   TextIO.putln("   Popped " + value + " at end of expression.");

   if (stack.isEmpty() == false) {
      TextIO.putln("   Stack is not empty.");
      TextIO.putln("\nNot enough operators for all the numbers!");
      return;
   }

   TextIO.putln("\nValue = " + value);


} // end readAndEvaluate()
```

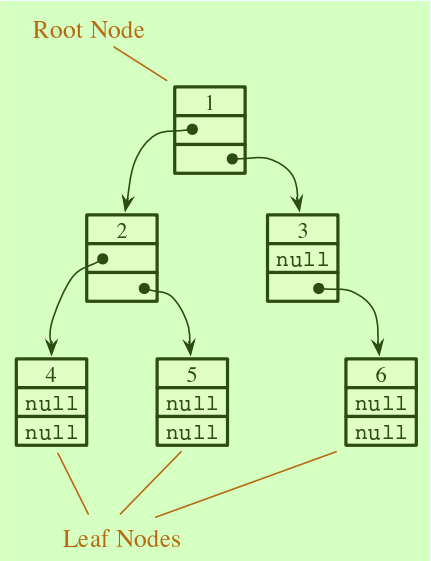
Postfix expressions are often used internally by computers. In fact, the Java virtual machine is a "stack machine" which uses the stack-based approach to expression evaluation that we have been discussing. The algorithm can easily be extended to handle variables, as well as constants. When a variable is encountered in the expression, the value of the variable is pushed onto the stack. It also works for operators with more or fewer than two operands. As many operands as are needed are popped from the stack and the result is pushed back on to the stack. For example, the ***unary minus*** operator, which is used in the expression "`-x`", has a single operand. We will continue to look at expressions and expression evaluation in the next two sections.

9.4 Binary Trees

We have seen in the two previous sections how objects can be linked into lists. When an object contains two pointers to objects of the same type, structures can be created that are much more complicated than linked lists. In this section, we'll look at one of the most basic and useful structures of this type: ***binary trees***. Each of the objects in a binary tree contains two pointers, typically called `left` and `right`. In addition to these pointers, of course, the nodes can contain other types of data. For example, a binary tree of integers could be made up of objects of the following type:

```
class TreeNode {
   int item;        // The data in this node.
   TreeNode left;   // Pointer to the left subtree.
   TreeNode right;  // Pointer to the right subtree.
}
```

The `left` and `right` pointers in a `TreeNode` can be `null` or can point to other objects of type `TreeNode`. A node that points to another node is said to be the ***parent*** of that node, and the node it points to is called a ***child***. In the picture below, for example, node 3 is the parent of node 6, and nodes 4 and 5 are children of node 2. Not every linked structure made up of tree nodes is a binary tree. A binary tree must have the following properties: There is exactly one node in the tree which has no parent. This node is called the ***root*** of the tree. Every other node in the tree has exactly one parent. Finally, there can be no loops in a binary tree. That is, it is not possible to follow a chain of pointers starting at some node and arriving back at the same node.



A node that has no children is called a ***leaf***. A leaf node can be recognized by the fact that both the left and right pointers in the node are `null`. In the standard picture of a binary tree, the root node is shown at the top and the leaf nodes at the bottom—which doesn't show much respect for the analogy to real trees. But at least you can see the branching, tree-like structure that gives a binary tree its name.

9.4.1 Tree Traversal

Consider any node in a binary tree. Look at that node together with all its descendents (that is, its children, the children of its children, and so on). This set of nodes forms a binary tree, which is called a ***subtree*** of the original tree. For example, in the picture, nodes 2, 4, and 5 form a subtree. This subtree is called the ***left subtree*** of the root. Similarly, nodes 3 and 6 make up the ***right subtree*** of the root. We can consider any non-empty binary tree to be made up of a root node, a left subtree, and a right subtree. Either or both of the subtrees can be empty. This is a recursive definition, matching the recursive definition of the *TreeNode* class. So it should not be a surprise that recursive subroutines are often used to process trees.

Consider the problem of counting the nodes in a binary tree. (As an exercise, you might try to come up with a non-recursive algorithm to do the counting, but you shouldn't expect to find one.) The heart of problem is keeping track of which nodes remain to be counted. It's not so easy to do this, and in fact it's not even possible without an auxiliary data structure such as a stack or queue. With recursion, however, the algorithm is almost trivial. Either the tree is empty or it consists of a root and two subtrees. If the tree is empty, the number of nodes is zero. (This is the base case of the recursion.) Otherwise, use recursion to count the nodes in each subtree. Add the results from the subtrees together, and add one to count the root. This gives the total number of nodes in the tree. Written out in Java:

```
/**
 * Count the nodes in the binary tree to which root points, and
 * return the answer.  If root is null, the answer is zero.
 */
static int countNodes( TreeNode root ) {
   if ( root == null )
```

```
      return 0;  // The tree is empty.  It contains no nodes.
   else {
      int count = 1;   // Start by counting the root.
      count += countNodes(root.left);  // Add the number of nodes
                                       //     in the left subtree.
      count += countNodes(root.right); // Add the number of nodes
                                       //    in the right subtree.
      return count;  // Return the total.
   }
} // end countNodes()
```

Or, consider the problem of printing the items in a binary tree. If the tree is empty, there is nothing to do. If the tree is non-empty, then it consists of a root and two subtrees. Print the item in the root and use recursion to print the items in the subtrees. Here is a subroutine that prints all the items on one line of output:

```
/**
 * Print all the items in the tree to which root points.
 * The item in the root is printed first, followed by the
 * items in the left subtree and then the items in the
 * right subtree.
 */
static void preorderPrint( TreeNode root ) {
   if ( root != null ) {  // (Otherwise, there's nothing to print.)
      System.out.print( root.item + " " );  // Print the root item.
      preorderPrint( root.left );   // Print items in left subtree.
      preorderPrint( root.right );  // Print items in right subtree.
   }
} // end preorderPrint()
```

This routine is called "preorderPrint" because it uses a ***preorder traversal*** of the tree. In a preorder traversal, the root node of the tree is processed first, then the left subtree is traversed, then the right subtree. In a ***postorder traversal***, the left subtree is traversed, then the right subtree, and then the root node is processed. And in an ***inorder traversal***, the left subtree is traversed first, then the root node is processed, then the right subtree is traversed. Printing subroutines that use postorder and inorder traversal differ from `preorderPrint` only in the placement of the statement that outputs the root item:

```
/**
 * Print all the items in the tree to which root points.
 * The item in the left subtree printed first, followed
 * by the items in the right subtree and then the item
 * in the root node.
 */
static void postorderPrint( TreeNode root ) {
   if ( root != null ) {  // (Otherwise, there's nothing to print.)
      postorderPrint( root.left );   // Print items in left subtree.
      postorderPrint( root.right );  // Print items in right subtree.
      System.out.print( root.item + " " );  // Print the root item.
   }
} // end postorderPrint()

/**
 * Print all the items in the tree to which root points.
```

```
 * The item in the left subtree printed first, followed
 * by the item in the root node and then the items
 * in the right subtree.
 */
static void inorderPrint( TreeNode root ) {
   if ( root != null ) {  // (Otherwise, there's nothing to print.)
      inorderPrint( root.left );   // Print items in left subtree.
      System.out.print( root.item + " " );  // Print the root item.
      inorderPrint( root.right );  // Print items in right subtree.
   }
} // end inorderPrint()
```

Each of these subroutines can be applied to the binary tree shown in the illustration at the beginning of this section. The order in which the items are printed differs in each case:

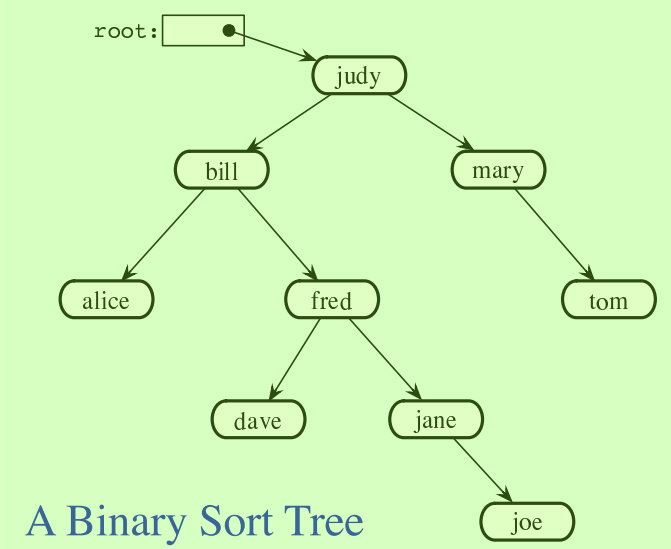
```
preorderPrint outputs:   1  2  4  5  3  6

postorderPrint outputs:  4  5  2  6  3  1

inorderPrint outputs:    4  2  5  1  3  6
```

In `preorderPrint`, for example, the item at the root of the tree, `1`, is output before anything else. But the preorder printing also applies to each of the subtrees of the root. The root item of the left subtree, `2`, is printed before the other items in that subtree, `4` and `5`. As for the right subtree of the root, `3` is output before `6`. A preorder traversal applies at all levels in the tree. The other two traversal orders can be analyzed similarly.

9.4.2 Binary Sort Trees

One of the examples in Section 9.2 was a linked list of strings, in which the strings were kept in increasing order. While a linked list works well for a small number of strings, it becomes inefficient for a large number of items. When inserting an item into the list, searching for that item's position requires looking at, on average, half the items in the list. Finding an item in the list requires a similar amount of time. If the strings are stored in a sorted array instead of in a linked list, then searching becomes more efficient because binary search can be used. However, inserting a new item into the array is still inefficient since it means moving, on average, half of the items in the array to make a space for the new item. A binary tree can be used to store an ordered list of strings, or other items, in a way that makes both searching and insertion efficient. A binary tree used in this way is called a ***binary sort tree.***

A binary sort tree is a binary tree with the following property: For every node in the tree, the item in that node is greater than every item in the left subtree of that node, and it is less than or equal to all the items in the right subtree of that node. Here for example is a binary sort tree containing items of type *String*. (In this picture, I haven't bothered to draw all the pointer variables. Non-null pointers are shown as arrows.)



Binary sort trees have this useful property: An inorder traversal of the tree will process the items in increasing order. In fact, this is really just another way of expressing the definition. For example, if an inorder traversal is used to print the items in the tree shown above, then the items will be in alphabetical order. The definition of an inorder traversal guarantees that all the items in the left subtree of "judy" are printed before "judy", and all the items in the right subtree of "judy" are printed after "judy". But the binary sort tree property guarantees that the items in the left subtree of "judy" are precisely those that precede "judy" in alphabetical order, and all the items in the right subtree follow "judy" in alphabetical order. So, we know that "judy" is output in its proper alphabetical position. But the same argument applies to the subtrees. "Bill" will be output after "alice" and before "fred" and its descendents. "Fred" will be output after "dave" and before "jane" and "joe". And so on.

Suppose that we want to search for a given item in a binary search tree. Compare that item to the root item of the tree. If they are equal, we're done. If the item we are looking for is less than the root item, then we need to search the left subtree of the root—the right subtree can be eliminated because it only contains items that are greater than or equal to the root. Similarly, if the item we are looking for is greater than the item in the root, then we only need to look in the right subtree. In either case, the same procedure can then be applied to search the subtree. Inserting a new item is similar: Start by searching the tree for the position where the new item belongs. When that position is found, create a new node and attach it to the tree at that position.

Searching and inserting are efficient operations on a binary search tree, provided that the tree is close to being ***balanced***. A binary tree is balanced if for each node, the left subtree of that node contains approximately the same number of nodes as the right subtree. In a perfectly balanced tree, the two numbers differ by at most one. Not all binary trees are balanced, but if the tree is created by inserting items in a random order, there is a high probability that the tree is approximately balanced. (If the order of insertion is not random, however, it's quite possible for the tree to be very unbalanced.) During a search of any binary sort tree, every comparison eliminates one of two subtrees from further consideration. If the tree is balanced, that means cutting the number of items still under consideration in half. This is exactly the same as the binary search algorithm, and the result, is a similarly efficient algorithm.

In terms of asymptotic analysis (Section 8.6), searching, inserting, and deleting in a binary

search tree have average case run time $\Theta(\log(n))$. The problem size, n, is the number of items in the tree, and the average is taken over all the different orders in which the items could have been inserted into the tree. As long the actual insertion order is random, the actual run time can be expected to be close to the average. However, the worst case run time for binary search tree operations is $\Theta(n)$, which is much worse than $\Theta(\log(n))$. The worst case occurs for certain particular insertion orders. For example, if the items are inserted into the tree in order of increasing size, then every item that is inserted moves always to the right as it moves down the tree. The result is a "tree" that looks more like a linked list, since it consists of a linear string of nodes strung together by their `right` child pointers. Operations on such a tree have the same performance as operations on a linked list. Now, there are data structures that are similar to simple binary sort trees, except that insertion and deletion of nodes are implemented in a way that will always keep the tree balanced, or almost balanced. For these data structures, searching, inserting, and deleting have both average case and worst case run times that are $\Theta(\log(n))$. Here, however, we will look at only the simple versions of inserting and searching.

The sample program *SortTreeDemo.java* is a demonstration of binary sort trees. The program includes subroutines that implement inorder traversal, searching, and insertion. We'll look at the latter two subroutines below. The `main()` routine tests the subroutines by letting you type in strings to be inserted into the tree.

In this program, nodes in the binary tree are represented using the following static nested class, including a simple constructor that makes creating nodes easier:

```
/**
 * An object of type TreeNode represents one node in a binary tree of strings.
 */
private static class TreeNode {
   String item;      // The data in this node.
   TreeNode left;    // Pointer to left subtree.
   TreeNode right;   // Pointer to right subtree.
   TreeNode(String str) {
          // Constructor.  Make a node containing str.
      item = str;
   }
}  // end class TreeNode
```

A static member variable of type *TreeNode* points to the binary sort tree that is used by the program:

```
private static TreeNode root;  // Pointer to the root node in the tree.
                               // When the tree is empty, root is null.
```

A recursive subroutine named `treeContains` is used to search for a given item in the tree. This routine implements the search algorithm for binary trees that was outlined above:

```
/**
 * Return true if item is one of the items in the binary
 * sort tree to which root points.   Return false if not.
 */
static boolean treeContains( TreeNode root, String item ) {
   if ( root == null ) {
          // Tree is empty, so it certainly doesn't contain item.
      return false;
   }
   else if ( item.equals(root.item) ) {
```

```
            // Yes, the item has been found in the root node.
         return true;
      }
      else if ( item.compareTo(root.item) < 0 ) {
            // If the item occurs, it must be in the left subtree.
         return treeContains( root.left, item );
      }
      else {
            // If the item occurs, it must be in the right subtree.
         return treeContains( root.right, item );
      }
   }  // end treeContains()
```

When this routine is called in the `main()` routine, the first parameter is the static member variable `root`, which points to the root of the entire binary sort tree.

It's worth noting that recursion is not really essential in this case. A simple, non-recursive algorithm for searching a binary sort tree follows the rule: Start at the root and move down the tree until you find the item or reach a null pointer. Since the search follows a single path down the tree, it can be implemented as a `while` loop. Here is non-recursive version of the search routine:

```
private static boolean treeContainsNR( TreeNode root, String item ) {
   TreeNode runner;  // For "running" down the tree.
   runner = root;    // Start at the root node.
   while (true) {
      if (runner == null) {
            // We've fallen off the tree without finding item.
         return false;
      }
      else if ( item.equals(node.item) ) {
            // We've found the item.
         return true;
      }
      else if ( item.compareTo(node.item) < 0 ) {
            // If the item occurs, it must be in the left subtree,
            // So, advance the runner down one level to the left.
         runner = runner.left;
      }
      else {
            // If the item occurs, it must be in the right subtree.
            // So, advance the runner down one level to the right.
         runner = runner.right;
      }
   }  // end while
} // end treeContainsNR();
```

The subroutine for inserting a new item into the tree turns out to be more similar to the non-recursive search routine than to the recursive. The insertion routine has to handle the case where the tree is empty. In that case, the value of `root` must be changed to point to a node that contains the new item:

```
root = new TreeNode( newItem );
```

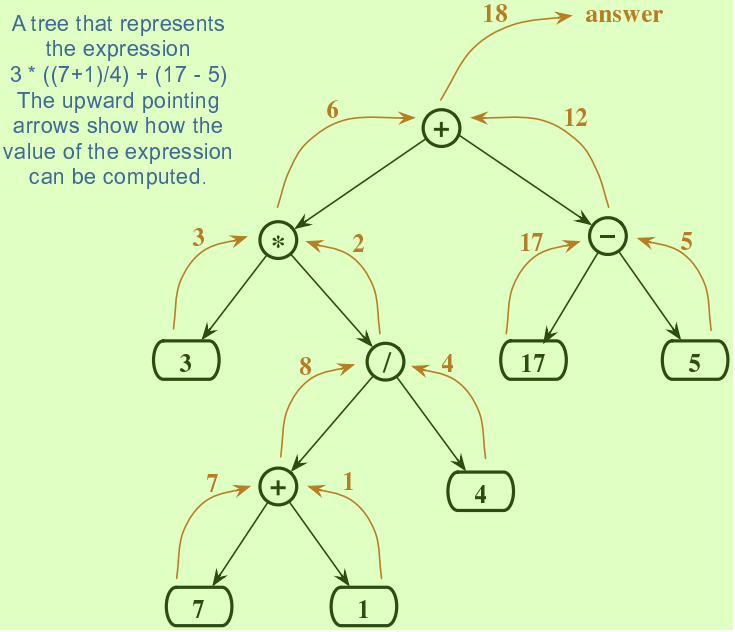
But this means, effectively, that the root can't be passed as a parameter to the subroutine, because it is impossible for a subroutine to change the value stored in an actual parameter. (I should note that this is something that **is** possible in other languages.) Recursion uses parameters in an essential way. There are ways to work around the problem, but the easiest thing is just to use a non-recursive insertion routine that accesses the static member variable `root` directly. One difference between inserting an item and searching for an item is that we have to be careful not to fall off the tree. That is, we have to stop searching just **before** `runner` becomes `null`. When we get to an empty spot in the tree, that's where we have to insert the new node:

```
/**
 * Add the item to the binary sort tree to which the global variable
 * "root" refers.  (Note that root can't be passed as  a parameter to
 * this routine because the value of root might change, and a change
 * in the value of a formal parameter does not change the actual parameter.)
 */
private static void treeInsert(String newItem) {
   if ( root == null ) {
          // The tree is empty.  Set root to point to a new node containing
          // the new item.  This becomes the only node in the tree.
      root = new TreeNode( newItem );
      return;
   }
   TreeNode runner;  // Runs down the tree to find a place for newItem.
   runner = root;   // Start at the root.
   while (true) {
      if ( newItem.compareTo(runner.item) < 0 ) {
             // Since the new item is less than the item in runner,
             // it belongs in the left subtree of runner.  If there
             // is an open space at runner.left, add a new node there.
             // Otherwise, advance runner down one level to the left.
         if ( runner.left == null ) {
            runner.left = new TreeNode( newItem );
            return;  // New item has been added to the tree.
         }
         else
            runner = runner.left;
      }
      else {
             // Since the new item is greater than or equal to the item in
             // runner, it belongs in the right subtree of runner.  If there
             // is an open space at runner.right, add a new node there.
             // Otherwise, advance runner down one level to the right.
         if ( runner.right == null ) {
            runner.right = new TreeNode( newItem );
            return;  // New item has been added to the tree.
         }
         else
            runner = runner.right;
      }
   } // end while
}  // end treeInsert()
```

9.4.3 Expression Trees

Another application of trees is to store mathematical expressions such as `15*(x+y)` or `sqrt(42)+7` in a convenient form. Let's stick for the moment to expressions made up of numbers and the operators `+`, `-`, `*`, and `/`. Consider the expression `3*((7+1)/4)+(17-5)`. This expression is made up of two subexpressions, `3*((7+1)/4)` and `(17-5)`, combined with the operator "+". When the expression is represented as a binary tree, the root node holds the operator `+`, while the subtrees of the root node represent the subexpressions `3*((7+1)/4)` and `(17-5)`. Every node in the tree holds either a number or an operator. A node that holds a number is a leaf node of the tree. A node that holds an operator has two subtrees representing the operands to which the operator applies. The tree is shown in the illustration below. I will refer to a tree of this type as an ***expression tree***.

Given an expression tree, it's easy to find the value of the expression that it represents. Each node in the tree has an associated value. If the node is a leaf node, then its value is simply the number that the node contains. If the node contains an operator, then the associated value is computed by first finding the values of its child nodes and then applying the operator to those values. The process is shown by the upward-directed arrows in the illustration. The value computed for the root node is the value of the expression as a whole. There are other uses for expression trees. For example, a postorder traversal of the tree will output the postfix form of the expression.



An expression tree contains two types of nodes: nodes that contain numbers and nodes that contain operators. Furthermore, we might want to add other types of nodes to make the trees more useful, such as nodes that contain variables. If we want to work with expression trees in Java, how can we deal with this variety of nodes? One way—which will be frowned upon by object-oriented purists—is to include an instance variable in each node object to record which type of node it is:

```
enum NodeType { NUMBER, OPERATOR }   // Possible kinds of node.
```

```
class ExpNode {  // A node in an expression tree.

   NoteType kind;  // Which type of node is this?
   double number;  // The value in a node of type NUMBER.
   char op;        // The operator in a node of type OPERATOR.
   ExpNode left;   // Pointers to subtrees,
   ExpNode right;  //     in a node of type OPERATOR.

   ExpNode( double val ) {
         // Constructor for making a node of type NUMBER.
      kind = NodeType.NUMBER;
      number = val;
   }

   ExpNode( char op, ExpNode left, ExpNode right ) {
         // Constructor for making a node of type OPERATOR.
      kind = NodeType.OPERATOR;
      this.op = op;
      this.left = left;
      this.right = right;
   }

} // end class ExpNode
```

Given this definition, the following recursive subroutine will find the value of an expression tree:

```
static double getValue( ExpNode node ) {
       // Return the value of the expression represented by
       // the tree to which node refers.  Node must be non-null.
   if ( node.kind == NodeType.NUMBER ) {
         // The value of a NUMBER node is the number it holds.
      return node.number;
   }
   else {  // The kind must be OPERATOR.
           // Get the values of the operands and combine them
           //    using the operator.
      double leftVal = getValue( node.left );
      double rightVal = getValue( node.right );
      switch ( node.op ) {
         case '+':  return leftVal + rightVal;
         case '-':  return leftVal - rightVal;
         case '*':  return leftVal * rightVal;
         case '/':  return leftVal / rightVal;
         default:   return Double.NaN;  // Bad operator.
      }
   }
} // end getValue()
```

Although this approach works, a more object-oriented approach is to note that since there are two types of nodes, there should be two classes to represent them, *ConstNode* and *BinOpNode*. To represent the general idea of a node in an expression tree, we need another class, *ExpNode*. Both *ConstNode* and *BinOpNode* will be subclasses of *ExpNode*. Since any actual node will be either a `ConstNode` or a *BinOpNode*, *ExpNode* should be an abstract class. (See Subsection 5.5.5.) Since one of the things we want to do with nodes is find their values, each class should have an instance method for finding the value:

```
abstract class ExpNode {
       // Represents a node of any type in an expression tree.

    abstract double value();  // Return the value of this node.

} // end class ExpNode


class ConstNode extends ExpNode {
       // Represents a node that holds a number.

    double number;  // The number in the node.

    ConstNode( double val ) {
          // Constructor.  Create a node to hold val.
       number = val;
    }

    double value() {
          // The value is just the number that the node holds.
       return number;
    }

 } // end class ConstNode


 class BinOpNode extends ExpNode {
       // Represents a node that holds an operator.

    char op;        // The operator.
    ExpNode left;   // The left operand.
    ExpNode right;  // The right operand.

    BinOpNode( char op, ExpNode left, ExpNode right ) {
          // Constructor.  Create a node to hold the given data.
       this.op = op;
       this.left = left;
       this.right = right;
    }

    double value() {
          // To get the value, compute the value of the left and
          // right operands, and combine them with the operator.
        double leftVal = left.value();
        double rightVal = right.value();
        switch ( op ) {
            case '+':  return leftVal + rightVal;
            case '-':  return leftVal - rightVal;
            case '*':  return leftVal * rightVal;
            case '/':  return leftVal / rightVal;
            default:   return Double.NaN;  // Bad operator.
         }
    }

 } // end class BinOpNode
```

Note that the left and right operands of a *BinOpNode* are of type *ExpNode*, not *BinOpNode*. This allows the operand to be either a *ConstNode* or another *BinOpNode*—or any other type of *ExpNode* that we might eventually create. Since every *ExpNode* has a `value()` method, we can

call `left.value()` to compute the value of the left operand. If `left` is in fact a *ConstNode*, this will call the `value()` method in the *ConstNode* class. If it is in fact a *BinOpNode*, then `left.value()` will call the `value()` method in the *BinOpNode* class. Each node knows how to compute its own value.

Although it might seem more complicated at first, the object-oriented approach has some advantages. For one thing, it doesn't waste memory. In the original *ExpNode* class, only some of the instance variables in each node were actually used, and we needed an extra instance variable to keep track of the type of node. More important, though, is the fact that new types of nodes can be added more cleanly, since it can be done by creating a new subclass of *ExpNode* rather than by modifying an existing class.

We'll return to the topic of expression trees in the next section, where we'll see how to create an expression tree to represent a given expression.

9.5 A Simple Recursive Descent Parser

I HAVE ALWAYS been fascinated by language—both natural languages like English and the artificial languages that are used by computers. There are many difficult questions about how languages can convey information, how they are structured, and how they can be processed. Natural and artificial languages are similar enough that the study of programming languages, which are pretty well understood, can give some insight into the much more complex and difficult natural languages. And programming languages raise more than enough interesting issues to make them worth studying in their own right. How can it be, after all, that computers can be made to "understand" even the relatively simple languages that are used to write programs? Computers, after all, can only directly use instructions expressed in very simple machine language. Higher level languages must be translated into machine language. But the translation is done by a compiler, which is just a program. How could such a translation program be written?

9.5.1 Backus-Naur Form

Natural and artificial languages are similar in that they have a structure known as grammar or syntax. Syntax can be expressed by a set of rules that describe what it means to be a legal sentence or program. For programming languages, syntax rules are often expressed in ***BNF*** (Backus-Naur Form), a system that was developed by computer scientists John Backus and Peter Naur in the late 1950s. Interestingly, an equivalent system was developed independently at about the same time by linguist Noam Chomsky to describe the grammar of natural language. BNF cannot express all possible syntax rules. For example, it can't express the fact that a variable must be defined before it is used. Furthermore, it says nothing about the meaning or semantics of the langauge. The problem of specifying the semantics of a language—even of an artificial programming langauge—is one that is still far from being completely solved. However, BNF does express the basic structure of the language, and it plays a central role in the design of translation programs.

In English, terms such as "noun", "transitive verb," and "prepositional phrase" are ***syntactic categories*** that describe building blocks of sentences. Similarly, "statement", "number," and "while loop" are syntactic categories that describe building blocks of Java programs. In BNF, a syntactic category is written as a word enclosed between "<" and ">". For example: `<noun>`, `<verb-phrase>`, or `<while-loop>`. A ***rule*** in BNF specifies the structure of an item

in a given syntactic category, in terms of other syntactic categories and/or basic symbols of the language. For example, one BNF rule for the English language might be

```
<sentence>  ::=  <noun-phrase> <verb-phrase>
```

The symbol "`::=`" is read "can be", so this rule says that a `<sentence>` can be a `<noun-phrase>` followed by a `<verb-phrase>`. (The term is "can be" rather than "is" because there might be other rules that specify other possible forms for a sentence.) This rule can be thought of as a recipe for a sentence: If you want to make a sentence, make a noun-phrase and follow it by a verb-phrase. Noun-phrase and verb-phrase must, in turn, be defined by other BNF rules.

In BNF, a choice between alternatives is represented by the symbol "`|`", which is read "or". For example, the rule

```
<verb-phrase>  ::=  <intransitive-verb>  |
                    ( <transitive-verb> <noun-phrase> )
```

says that a `<verb-phrase>` can be an `<intransitive-verb>`, or a `<transitive-verb>` followed by a `<noun-phrase>`. Note also that parentheses can be used for grouping. To express the fact that an item is optional, it can be enclosed between "`[`" and "`]`". An optional item that can be repeated one or more times is enclosed between "`[`" and "`]...`". And a symbol that is an actual part of the language that is being described is enclosed in quotes. For example,

```
<noun-phrase>  ::=  <common-noun> [ "that" <verb-phrase> ]  |
                    <common-noun> [ <prepositional-phrase> ]...
```

says that a `<noun-phrase>` can be a `<common-noun>`, optionally followed by the literal word "`that`" and a `<verb-phrase>`, or it can be a `<common-noun>` followed by zero or more `<prepositional-phrase>`'s. Obviously, we can describe very complex structures in this way. The real power comes from the fact that BNF rules can be **recursive**. In fact, the two preceding rules, taken together, are recursive. A `<noun-phrase>` is defined partly in terms of `<verb-phrase>`, while `<verb-phrase>` is defined partly in terms of `<noun-phrase>`. For example, a `<noun-phrase>` might be "the rat that ate the cheese", since "ate the cheese" is a `<verb-phrase>`. But then we can, recursively, make the more complex `<noun-phrase>` "the cat that caught the rat that ate the cheese" out of the <common-noun> "the cat", the word "that" and the `<verb-phrase>` "caught the rat that ate the cheese". Building from there, we can make the `<noun-phrase>` "the dog that chased the cat that caught the rat that ate the cheese". The recursive structure of language is one of the most fundamental properties of language, and the ability of BNF to express this recursive structure is what makes it so useful.

BNF can be used to describe the syntax of a programming language such as Java in a formal and precise way. For example, a `<while-loop>` can be defined as

```
<while-loop>  ::=  "while" "(" <condition> ")" <statement>
```

This says that a `<while-loop>` consists of the word "while", followed by a left parenthesis, followed by a `<condition>`, followed by a right parenthesis, followed by a `<statement>`. Of course, it still remains to define what is meant by a condition and by a statement. Since a statement can be, among other things, a `while` loop, we can already see the recursive structure of the Java language. The exact specification of an `if` statement, which is hard to express clearly in words, can be given as

```
<if-statement>  ::=
            "if" "(" <condition> ")" <statement>
            [ "else" "if" "(" <condition> ")" <statement> ]...
            [ "else" <statement> ]
```

This rule makes it clear that the "`else`" part is optional and that there can be, optionally, one or more "`else if`" parts.

9.5.2 Recursive Descent Parsing

In the rest of this section, I will show how a BNF grammar for a language can be used as a guide for constructing a parser. A parser is a program that determines the grammatical structure of a phrase in the language. This is the first step to determining the meaning of the phrase—which for a programming language means translating it into machine language. Although we will look at only a simple example, I hope it will be enough to convince you that compilers can in fact be written and understood by mortals and to give you some idea of how that can be done.

The parsing method that we will use is called ***recursive descent parsing***. It is not the only possible parsing method, or the most efficient, but it is the one most suited for writing compilers by hand (rather than with the help of so called "parser generator" programs). In a recursive descent parser, every rule of the BNF grammar is the model for a subroutine. Not every BNF grammar is suitable for recursive descent parsing. The grammar must satisfy a certain property. Essentially, while parsing a phrase, it must be possible to tell what syntactic category is coming up next just by looking at the next item in the input. Many grammars are designed with this property in mind.

I should also mention that many variations of BNF are in use. The one that I've described here is one that is well-suited for recursive descent parsing.

* * *

When we try to parse a phrase that contains a syntax error, we need some way to respond to the error. A convenient way of doing this is to throw an exception. I'll use an exception class called *ParseError*, defined as follows:

```
/**
 * An object of type ParseError represents a syntax error found in
 * the user's input.
 */
private static class ParseError extends Exception {
   ParseError(String message) {
      super(message);
   }
} // end nested class ParseError
```

Another general point is that our BNF rules don't say anything about spaces between items, but in reality we want to be able to insert spaces between items at will. To allow for this, I'll always call the routine `TextIO.skipBlanks()` before trying to look ahead to see what's coming up next in input. `TextIO.skipBlanks()` skips past any whitespace, such as spaces and tabs, in the input, and stops when the next character in the input is either a non-blank character or the end-of-line character.

Let's start with a very simple example. A "fully parenthesized expression" can be specified in BNF by the rules

```
<expression>  ::=  <number>  |
                   "(" <expression> <operator> <expression> ")"

<operator>  ::=  "+" | "-" | "*" | "/"
```

where <number> refers to any non-negative real number. An example of a fully parenthesized expression is "(((34-17)*8)+(2*7))". Since every operator corresponds to a pair of parentheses, there is no ambiguity about the order in which the operators are to be applied. Suppose we want a program that will read and evaluate such expressions. We'll read the expressions from standard input, using *TextIO*. To apply recursive descent parsing, we need a subroutine for each rule in the grammar. Corresponding to the rule for <operator>, we get a subroutine that reads an operator. The operator can be a choice of any of four things. Any other input will be an error.

```
/**
 * If the next character in input is one of the legal operators,
 * read it and return it.  Otherwise, throw a ParseError.
 */
static char getOperator() throws ParseError {
   TextIO.skipBlanks();
   char op = TextIO.peek();
   if ( op == '+' || op == '-' || op == '*' || op == '/' ) {
      TextIO.getAnyChar();
      return op;
   }
   else if (op == '\n')
      throw new ParseError("Missing operator at end of line.");
   else
      throw new ParseError("Missing operator.  Found \"" +
            op + "\" instead of +, -, *, or /.");
} // end getOperator()
```

I've tried to give a reasonable error message, depending on whether the next character is an end-of-line or something else. I use TextIO.peek() to look ahead at the next character before I read it, and I call TextIO.skipBlanks() before testing TextIO.peek() in order to ignore any blanks that separate items. I will follow this same pattern in every case.

When we come to the subroutine for <expression>, things are a little more interesting. The rule says that an expression can be either a number or an expression enclosed in parentheses. We can tell which it is by looking ahead at the next character. If the character is a digit, we have to read a number. If the character is a "(", we have to read the "(", followed by an expression, followed by an operator, followed by another expression, followed by a ")". If the next character is anything else, there is an error. Note that we need recursion to read the nested expressions. The routine doesn't just read the expression. It also computes and returns its value. This requires semantical information that is not specified in the BNF rule.

```
/**
 * Read an expression from the current line of input and return its value.
 * @throws ParseError if the input contains a syntax error
 */
private static double expressionValue() throws ParseError {
   TextIO.skipBlanks();
   if ( Character.isDigit(TextIO.peek()) ) {
          // The next item in input is a number, so the expression
          // must consist of just that number.  Read and return
          // the number.
      return TextIO.getDouble();
   }
   else if ( TextIO.peek() == '(' ) {
```

```
            // The expression must be of the form
            //         "(" <expression> <operator> <expression> ")"
            // Read all these items, perform the operation, and
            // return the result.
         TextIO.getAnyChar();  // Read the "("
         double leftVal = expressionValue();  // Read and evaluate first operand.
         char op = getOperator();             // Read the operator.
         double rightVal = expressionValue(); // Read and evaluate second operand.
         TextIO.skipBlanks();
         if ( TextIO.peek() != ')' ) {
                // According to the rule, there must be a ")" here.
                // Since it's missing, throw a ParseError.
            throw new ParseError("Missing right parenthesis.");
         }
         TextIO.getAnyChar();  // Read the ")"
         switch (op) {   //  Apply the operator and return the result.
         case '+':  return leftVal + rightVal;
         case '-':  return leftVal - rightVal;
         case '*':  return leftVal * rightVal;
         case '/':  return leftVal / rightVal;
         default:   return 0;  // Can't occur since op is one of the above.
                               // (But Java syntax requires a return value.)
         }
      }
      else {  // No other character can legally start an expression.
         throw new ParseError("Encountered unexpected character, \"" +
               TextIO.peek() + "\" in input.");
      }
   } // end expressionValue()
```

1

I hope that you can see how this routine corresponds to the BNF rule. Where the rule uses "`|`" to give a choice between alternatives, there is an `if` statement in the routine to determine which choice to take. Where the rule contains a sequence of items, "(" `<expression>` `<operator>` `<expression>` ")", there is a sequence of statements in the subroutine to read each item in turn. 2

When `expressionValue()` is called to evaluate the expression `(((34-17)*8)+(2*7))`, it sees the "(" at the beginning of the input, so the `else` part of the `if` statement is executed. The "(" is read. Then the first recursive call to `expressionValue()` reads and evaluates the subexpression `((34-17)*8)`, the call to `getOperator()` reads the "+" operator, and the second recursive call to `expressionValue()` reads and evaluates the second subexpression `(2*7)`. Finally, the ")" at the end of the expression is read. Of course, reading the first subexpression, `((34-17)*8)`, involves further recursive calls to the `expressionValue()` routine, but it's better not to think too deeply about that! Rely on the recursion to handle the details. 3

You'll find a complete program that uses these routines in the file *SimpleParser1.java*. 4

* * * 5

Fully parenthesized expressions aren't very natural for people to use. But with ordinary expressions, we have to worry about the question of operator precedence, which tells us, for example, that the "`*`" in the expression "`5+3*7`" is applied before the "`+`". The complex expression "`3*6+8*(7+1)/4-24`" should be seen as made up of three "terms", `3*6`, `8*(7+1)/4`, and `24`, combined with "`+`" and "`-`" operators. A term, on the other hand, can be made up of several factors combined with "`*`" and "`/`" operators. For example, `8*(7+1)/4` contains the 6

factors `8`, `(7+1)` and `4`. This example also shows that a factor can be either a number or an expression in parentheses. To complicate things a bit more, we allow for leading minus signs in expressions, as in "`-(3+4)`" or "`-7`". (Since a `<number>` is a positive number, this is the only way we can get negative numbers. It's done this way to avoid "`3 * -7`", for example.) This structure can be expressed by the BNF rules

```
<expression>  ::=  [ "-" ] <term> [ ( "+" | "-" ) <term> ]...
<term>  ::=  <factor> [ ( "*" | "/" ) <factor> ]...
<factor>  ::=  <number>  |  "(" <expression> ")"
```

The first rule uses the "`[ ]...`" notation, which says that the items that it encloses can occur zero, one, two, or more times. This means that an `<expression>` can begin, optionally, with a "-". Then there must be a `<term>` which can optionally be followed by one of the operators "+" or "−" and another `<term>`, optionally followed by another operator and `<term>`, and so on. In a subroutine that reads and evaluates expressions, this repetition is handled by a `while` loop. An `if` statement is used at the beginning of the loop to test whether a leading minus sign is present:

```
/**
 * Read an expression from the current line of input and return its value.
 * @throws ParseError if the input contains a syntax error
 */
private static double expressionValue() throws ParseError {
   TextIO.skipBlanks();
   boolean negative;  // True if there is a leading minus sign.
   negative = false;
   if (TextIO.peek() == '-') {
      TextIO.getAnyChar();  // Read the minus sign.
      negative = true;
   }
   double val;  // Value of the expression.
   val = termValue();
   if (negative)
      val = -val;
   TextIO.skipBlanks();
   while ( TextIO.peek() == '+' || TextIO.peek() == '-' ) {
          // Read the next term and add it to or subtract it from
          // the value of previous terms in the expression.
      char op = TextIO.getAnyChar();  // Read the operator.
      double nextVal = termValue();
      if (op == '+')
         val += nextVal;
      else
         val -= nextVal;
      TextIO.skipBlanks();
   }
   return val;
} // end expressionValue()
```

The subroutine for `<term>` is very similar to this, and the subroutine for `<factor>` is similar to the example given above for fully parenthesized expressions. A complete program that reads and evaluates expressions based on the above BNF rules can be found in the file *SimpleParser2.java*.

9.5.3 Building an Expression Tree

Now, so far, we've only evaluated expressions. What does that have to do with translating programs into machine language? Well, instead of actually evaluating the expression, it would be almost as easy to generate the machine language instructions that are needed to evaluate the expression. If we are working with a "stack machine", these instructions would be stack operations such as "push a number" or "apply a + operation". The program *SimpleParser3.java* can both evaluate the expression and print a list of stack machine operations for evaluating the expression.

It's quite a jump from this program to a recursive descent parser that can read a program written in Java and generate the equivalent machine language code—but the conceptual leap is not huge.

The `SimpleParser3` program doesn't actually generate the stack operations directly as it parses an expression. Instead, it builds an expression tree, as discussed in the Section 9.4, to represent the expression. The expression tree is then used to find the value and to generate the stack operations. The tree is made up of nodes belonging to classes *ConstNode* and *BinOpNode* that are similar to those given in the Section 9.4. Another class, *UnaryMinusNode*, has been introduced to represent the unary minus operation. I've added a method, `printStackCommands()`, to each class. This method is responsible for printing out the stack operations that are necessary to evaluate an expression. Here for example is the new `BinOpNode` class from *SimpleParser3.java*:

```
private static class BinOpNode extends ExpNode {
   char op;        // The operator.
   ExpNode left;   // The expression for its left operand.
   ExpNode right;  // The expression for its right operand.
   BinOpNode(char op, ExpNode left, ExpNode right) {
          // Construct a BinOpNode containing the specified data.
      assert op == '+' || op == '-' || op == '*' || op == '/';
      assert left != null && right != null;
      this.op = op;
      this.left = left;
      this.right = right;
   }
   double value() {
          // The value is obtained by evaluating the left and right
          // operands and combining the values with the operator.
      double x = left.value();
      double y = right.value();
      switch (op) {
      case '+':
         return x + y;
      case '-':
         return x - y;
      case '*':
         return x * y;
      case '/':
         return x / y;
      default:
         return Double.NaN;  // Bad operator!
      }
   }
```

```
   void  printStackCommands() {
          // To evalute the expression on a stack machine, first do
          // whatever is necessary to evaluate the left operand, leaving
          // the answer on the stack.  Then do the same thing for the
          // second operand.  Then apply the operator (which means popping
          // the operands, applying the operator, and pushing the result).
      left.printStackCommands();
      right.printStackCommands();
      TextIO.putln("  Operator " + op);
   }
}
```

1

It's also interesting to look at the new parsing subroutines. Instead of computing a value, each subroutine builds an expression tree. For example, the subroutine corresponding to the rule for `<expression>` becomes

2

```
static ExpNode expressionTree() throws ParseError {
       // Read an expression from the current line of input and
       // return an expression tree representing the expression.
   TextIO.skipBlanks();
   boolean negative;  // True if there is a leading minus sign.
   negative = false;
   if (TextIO.peek() == '-') {
      TextIO.getAnyChar();
      negative = true;
   }
   ExpNode exp;   // The expression tree for the expression.
   exp = termTree();  // Start with a tree for first term.
   if (negative) {
          // Build the tree that corresponds to applying a
          // unary minus operator to the term we've
          // just read.
      exp = new UnaryMinusNode(exp);
   }
   TextIO.skipBlanks();
   while ( TextIO.peek() == '+' || TextIO.peek() == '-' ) {
            // Read the next term and combine it with the
            // previous terms into a bigger expression tree.
       char op = TextIO.getAnyChar();
       ExpNode nextTerm = termTree();
            // Create a tree that applies the binary operator
            // to the previous tree and the term we just read.
       exp = new BinOpNode(op, exp, nextTerm);
       TextIO.skipBlanks();
   }
   return exp;
} // end expressionTree()
```

3

In some real compilers, the parser creates a tree to represent the program that is being parsed. This tree is called a ***parse tree***. Parse trees are somewhat different in form from expression trees, but the purpose is the same. Once you have the tree, there are a number of things you can do with it. For one thing, it can be used to generate machine language code. But

4

there are also techniques for examining the tree and detecting certain types of programming errors, such as an attempt to reference a local variable before it has been assigned a value. (The Java compiler, of course, will reject the program if it contains such an error.) It's also possible to manipulate the tree to ***optimize*** the program. In optimization, the tree is transformed to make the program more efficient before the code is generated.

And so we are back where we started in Chapter 1, looking at programming languages, compilers, and machine language. But looking at them, I hope, with a lot more understanding and a much wider perspective.

Exercises for Chapter 9

1. In many textbooks, the first examples of recursion are the mathematical functions *factorial* and *fibonacci*. These functions are defined for non-negative integers using the following recursive formulas:

```
factorial(0)  =  1
factorial(N)  =  N*factorial(N-1)   for N > 0

fibonacci(0)  =  1
fibonacci(1)  =  1
fibonacci(N)  =  fibonacci(N-1) + fibonacci(N-2)   for N > 1
```

Write recursive functions to compute `factorial(N)` and `fibonacci(N)` for a given non-negative integer `N`, and write a `main()` routine to test your functions.

(In fact, *factorial* and *fibonacci* are really not very good examples of recursion, since the most natural way to compute them is to use simple `for` loops. Furthermore, *fibonacci* is a particularly bad example, since the natural recursive approach to computing this function is extremely inefficient.)

2. Exercise 7.6 asked you to read a file, make an alphabetical list of all the words that occur in the file, and write the list to another file. In that exercise, you were asked to use an ArrayList<String> to store the words. Write a new version of the same program that stores the words in a binary sort tree instead of in an arraylist. You can use the binary sort tree routines from *SortTreeDemo.java*, which was discussed in Subsection 9.4.2.

3. Suppose that linked lists of integers are made from objects belonging to the class

```
class ListNode {
   int item;       // An item in the list.
   ListNode next;  // Pointer to the next node in the list.
}
```

Write a subroutine that will make a copy of a list, with the order of the items of the list reversed. The subroutine should have a parameter of type *ListNode*, and it should return a value of type *ListNode*. The original list should not be modified.

You should also write a `main()` routine to test your subroutine.

4. Subsection 9.4.1 explains how to use recursion to print out the items in a binary tree in various orders. That section also notes that a non-recursive subroutine can be used to print the items, provided that a stack or queue is used as an auxiliary data structure. Assuming that a queue is used, here is an algorithm for such a subroutine:

```
Add the root node to an empty queue
while the queue is not empty:
   Get a node from the queue
   Print the item in the node
   if node.left is not null:
      add it to the queue
   if node.right is not null:
      add it to the queue
```

Write a subroutine that implements this algorithm, and write a program to test the subroutine. Note that you will need a queue of *TreeNodes*, so you will need to write a class to represent such queues.

(Note that the order in which items are printed by this algorithm is different from all three of the orders considered in Subsection 9.4.1.)

5. In Subsection 9.4.2, I say that "if the [binary sort] tree is created by inserting items in a random order, there is a high probability that the tree is approximately balanced." For this exercise, you will do an experiment to test whether that is true.

The ***depth*** of a node in a binary tree is the length of the path from the root of the tree to that node. That is, the root has depth 0, its children have depth 1, its grandchildren have depth 2, and so on. In a balanced tree, all the leaves in the tree are about the same depth. For example, in a perfectly balanced tree with 1023 nodes, all the leaves are at depth 9. In an approximately balanced tree with 1023 nodes, the average depth of all the leaves should be not too much bigger than 9.

On the other hand, even if the tree is approximately balanced, there might be a few leaves that have much larger depth than the average, so we might also want to look at the maximum depth among all the leaves in a tree.

For this exercise, you should create a random binary sort tree with 1023 nodes. The items in the tree can be real numbers, and you can create the tree by generating 1023 random real numbers and inserting them into the tree, using the usual `treeInsert()` method for binary sort trees. Once you have the tree, you should compute and output the average depth of all the leaves in the tree and the maximum depth of all the leaves. To do this, you will need three recursive subroutines: one to count the leaves, one to find the sum of the depths of all the leaves, and one to find the maximum depth. The latter two subroutines should have an **int**-valued parameter, `depth`, that tells how deep in the tree you've gone. When you call this routine from the main program, the `depth` parameter is 0; when you call the routine recursively, the parameter increases by 1.

6. The parsing programs in Section 9.5 work with expressions made up of numbers and operators. We can make things a little more interesting by allowing the variable "x" to occur. This would allow expression such as "`3*(x-1)*(x+1)`", for example. Make a new version of the sample program *SimpleParser3.java* that can work with such expressions. In your program, the `main()` routine can't simply print the value of the expression, since the value of the expression now depends on the value of `x`. Instead, it should print the value of the expression for `x=0`, `x=1`, `x=2`, and `x=3`.

The original program will have to be modified in several other ways. Currently, the program uses classes *ConstNode*, *BinOpNode*, and *UnaryMinusNode* to represent nodes in an expression tree. Since expressions can now include `x`, you will need a new class, *VariableNode*, to represent an occurrence of `x` in the expression.

In the original program, each of the node classes has an instance method, "`double value()`", which returns the value of the node. But in your program, the value can depend on `x`, so you should replace this method with one of the form "`double value(double xValue)`", where the parameter `xValue` is the value of `x`.

Finally, the parsing subroutines in your program will have to take into account the fact that expressions can contain `x`. There is just one small change in the BNF rules for the expressions: A `<factor>` is allowed to be the variable `x`:

```
<factor>  ::=  <number>  |  <x-variable>  |  "(" <expression> ")"
```

where `<x-variable>` can be either a lower case or an upper case "X". This change in the BNF requires a change in the `factorTree()` subroutine.

7. This exercise builds on the previous exercise, Exercise 9.6. To understand it, you should have some background in Calculus. The derivative of an expression that involves the variable `x` can be defined by a few recursive rules:

- The derivative of a constant is 0.
- The derivative of `x` is 1.
- If `A` is an expression, let `dA` be the derivative of `A`. Then the derivative of `-A` is `-dA`.
- If `A` and `B` are expressions, let `dA` be the derivative of `A` and let `dB` be the derivative of `B`. Then the derivative of `A+B` is `dA+dB`.
- The derivative of `A-B` is `dA-dB`.
- The derivative of `A*B` is `A*dB + B*dA`.
- The derivative of `A/B` is `(B*dA - A*dB) / (B*B)`.

For this exercise, you should modify your program from the previous exercise so that it can compute the derivative of an expression. You can do this by adding a derivative-computing method to each of the node classes. First, add another abstract method to the *ExpNode* class:

```
abstract ExpNode derivative();
```

Then implement this method in each of the four subclasses of *ExpNode*. All the information that you need is in the rules given above. In your main program, instead of printing the stack operations for the original expression, you should print out the stack operations that define the derivative. Note that the formula that you get for the derivative can be much more complicated than it needs to be. For example, the derivative of `3*x+1` will be computed as `(3*1+0*x)+0`. This is correct, even though it's kind of ugly, and it would be nice for it to be simplified. However, simplifying expressions is not easy.

As an alternative to printing out stack operations, you might want to print the derivative as a fully parenthesized expression. You can do this by adding a `printInfix()` routine to each node class. It would be nice to leave out unnecessary parentheses, but again, the problem of deciding which parentheses can be left out without altering the meaning of the expression is a fairly difficult one, which I don't advise you to attempt.

(There is one curious thing that happens here: If you apply the rules, as given, to an expression tree, the result is no longer a tree, since the same subexpression can occur at multiple points in the derivative. For example, if you build a node to represent `B*B` by saying "`new BinOpNode('*',B,B)`", then the left and right children of the new node are actually the same node! This is not allowed in a tree. However, the difference is harmless in this case since, like a tree, the structure that you get has no loops in it. Loops, on the other hand, would be a disaster in most of the recursive tree-processing subroutines that we have written, since it would lead to infinite recursion.)

Quiz on Chapter 9

1. Explain what is meant by a *recursive* subroutine.

2. Consider the following subroutine:

```
static void printStuff(int level) {
    if (level == 0) {
       System.out.print("*");
    }
    else {
       System.out.print("[");
       printStuff(level - 1);
       System.out.print(",");
       printStuff(level - 1);
       System.out.println("]");
    }
}
```

Show the output that would be produced by the subroutine calls `printStuff(0)`, `printStuff(1)`, `printStuff(2)`, and `printStuff(3)`.

3. Suppose that a linked list is formed from objects that belong to the class

```
class ListNode {
   int item;       // An item in the list.
   ListNode next;  // Pointer to next item in the list.
}
```

Write a subroutine that will count the number of zeros that occur in a given linked list of **ints**. The subroutine should have a parameter of type `ListNode` and should return a value of type **int**.

4. What are the three operations on a *stack?*

5. What is the basic difference between a stack and a queue?

6. What is an *activation record*? What role does a stack of activation records play in a computer?

7. Suppose that a binary tree of integers is formed from objects belonging to the class

```
class TreeNode {
   int item;       // One item in the tree.
   TreeNode left;  // Pointer to the left subtree.
   TreeNode right; // Pointer to the right subtree.
}
```

Write a recursive subroutine that will find the sum of all the nodes in the tree. Your subroutine should have a parameter of type `TreeNode`, and it should return a value of type **int**.

8. What is a *postorder traversal* of a binary tree?

9. Suppose that a `<multilist>` is defined by the BNF rule

```
<multilist>  ::=  <word>  |  "(" [ <multilist> ]... ")"
```
1

where a `<word>` can be any sequence of letters. Give five different `<multilist>'s` that can be generated by this rule. (This rule, by the way, is almost the entire syntax of the programming language `LISP`! `LISP` is known for its simple syntax and its elegant and powerful semantics.) 2

10. Explain what is meant by *parsing* a computer program. 3